

Dynamical Systems Theory in CR3BP

Naoki Hiraiwa

2022/06/03

2024/3/21 Update: Soi Yamaguchi

1. Dynamical Model
 - [CR3BP](#) ([Inertial frame](#) and [Rotating frame](#))
 - [ER3BP](#)
 - [BCR4BP](#)
2. [Coordinate Frame Transformation](#)
(Rotating frame to Inertial frame)
3. [Equilibrium points](#)
4. Zero velocity curve
 - 4.1. [Jacobi constant](#)
 - 4.2. [Zero velocity curve](#)

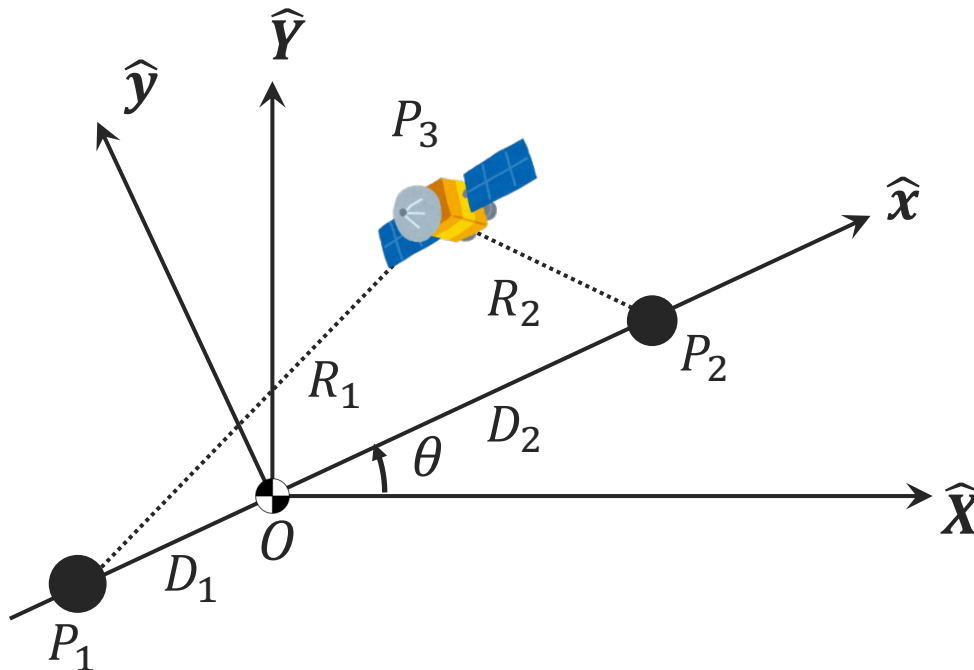
5. State Transition Matrix
 - [The concept of STM](#)
 - [Monodromy Matrix](#)
6. [Differential Correction](#) (Single Shooting)
7. [Invariant Manifold](#)
8. [Multiple shooting](#)
9. [Poincaré map](#)
10. [Natural parameter continuation](#)
11. [Pseudo-arclength continuation](#)

12. Introduction of periodic orbit family

- [Northern halo family](#)
- [Southern halo family](#)
- [Vertical Lyapunov family](#)
- [Axial family](#)

Circular Restricted Three-Body Problem (CR3BP)

- The motion of two gravitational bodies P_1, P_2 and a spacecraft P_3 .
- The mass of the spacecraft M_3 is much smaller than the masses of the primaries M_1, M_2 ($M_1 > M_2 \gg M_3$).
- The primaries P_1, P_2 rotate around their barycenter in a **circular orbit**.
 - The rotating rate $\dot{\theta}$ is constant. $\dot{\theta} = N = \text{const.}$ (N : mean motion)



O : Barycenter

$\hat{x}-\hat{y}-\hat{z}$: Rotating frame

$\hat{X}-\hat{Y}-\hat{Z}$: Inertial frame

Nondimensionalization

- The values in CR3BP are generally **nondimensionalized** by the following characteristics values.

Variable	Characteristic value	Nondimensional value
Length	$l^* = D_1 + D_2$	$d_1 = \frac{D_1}{l^*}, d_2 = \frac{D_2}{l^*}$
Mass	$m^* = M_1 + M_2$	$m_1 = \frac{M_1}{m^*}, m_2 = \frac{M_2}{m^*}$
Time	$t^* = 1/N = ((l^*)^3 / Gm^*)^{1/2}$	$t = \frac{T}{t^*}$

- Then, these nondimensional values are obtained when $\mu = \frac{M_2}{M_1 + M_2}$:

Mass

$$m_1 = 1 - \mu$$

$$m_2 = \mu$$

Distance

$$d_1 = \mu$$

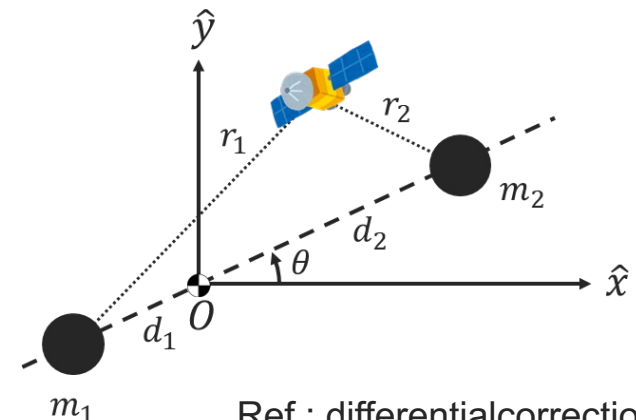
$$d_2 = 1 - \mu$$

Rotating rate

$$\dot{\theta} \rightarrow 1$$

Graviatational constant

$$G \rightarrow 1$$

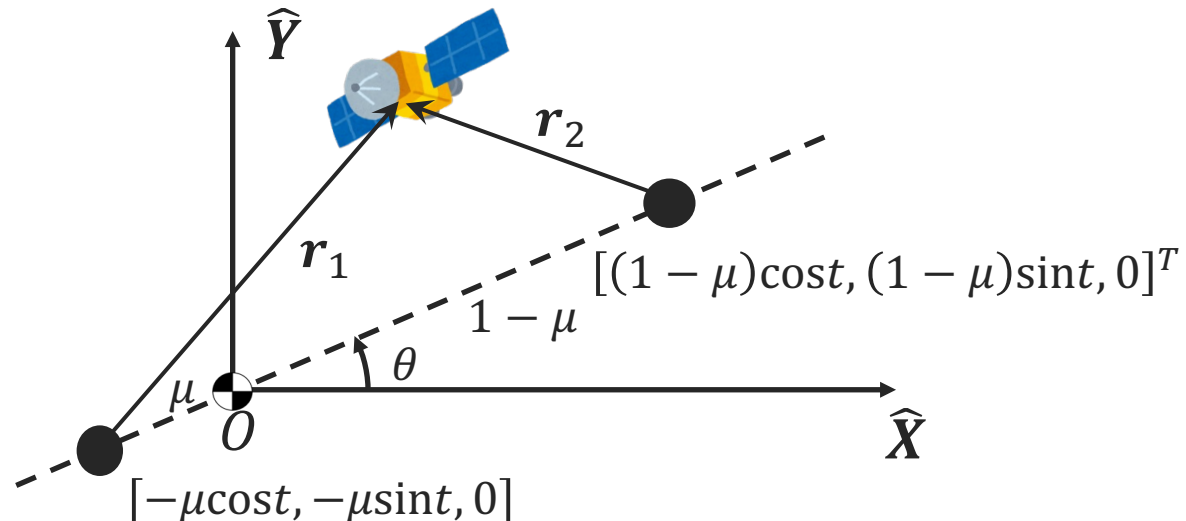


CR3BP in the inertial frame

- The positions of the primaries in the inertial frame are defined by
 $P_1 : [-\mu \cos t, -\mu \sin t, 0]^T$
 $P_2 : [(1 - \mu) \cos t, (1 - \mu) \sin t, 0]^T$
- The positions of the spacecraft relative to the primaries are calculated by

$$\mathbf{r}_1 = [X - \mu \cos t, Y - \mu \sin t, Z]^T$$

$$\mathbf{r}_2 = [X - (1 - \mu) \cos t, Y - (1 - \mu) \sin t, Z]^T$$



Equations of Motion in the inertial frame

Equations of motion of the CR3BP in the inertial frame are obtained by

$$\left\{ \begin{array}{l} \ddot{X} = -\frac{(1-\mu)(X + \mu \cos t)}{r_1^3} - \frac{\mu(X - (1-\mu)\cos t)}{r_2^3} \\ \ddot{Y} = -\frac{(1-\mu)(Y + \mu \sin t)}{r_1^3} - \frac{\mu(Y - (1-\mu)\sin t)}{r_2^3} \\ \ddot{Z} = -\frac{(1-\mu)Z}{r_1^3} - \frac{\mu Z}{r_2^3} \end{array} \right.$$

where

$$r_1 = \sqrt{(X + \mu \cos t)^2 + (Y + \mu \sin t)^2 + Z^2}$$

$$r_2 = \sqrt{(X - (1-\mu)\cos t)^2 + (Y - (1-\mu)\sin t)^2 + Z^2}$$

Sample Code : CR3BP in the inertial frame

fun_cr3bp_IF.m

```
function dx = fun_cr3bp_IF(t,x,mu)
% t : non-dimensional time
% x : non-dimensional position and velocity, x = [x y z vx vy vz]'
% mu : mass ratio of the primaries

% Primary body position
r1      = [x(1)+mu*cos(t);x(2)+mu*sin(t);x(3)];
r2      = [x(1)-(1-mu)*cos(t);x(2)-(1-mu)*sin(t);x(3)];
R1      = norm(r1);
R2      = norm(r2);

%
%      Ux      = -(1-mu)*(x(1)+mu*cos(t))/R1^3 - mu*(x(1)-(1-mu)*cos(t))/R2^3;
%      Uy      = -(1-mu)*(x(2)+mu*sin(t))/R1^3 - mu*(x(2)-(1-mu)*sin(t))/R2^3;
%      Uz      = -(1-mu)*x(3)/R1^3 - mu*x(3)/R2^3;

Ux      = -(1-mu)*r1(1)/R1^3 - mu*r2(1)/R2^3;
Uy      = -(1-mu)*r1(2)/R1^3 - mu*r2(2)/R2^3;
Uz      = -(1-mu)*r1(3)/R1^3 - mu*r2(3)/R2^3;
dx      = [x(4);x(5);x(6);Ux;Uy;Uz];
```

Equations of Motion in the rotating frame

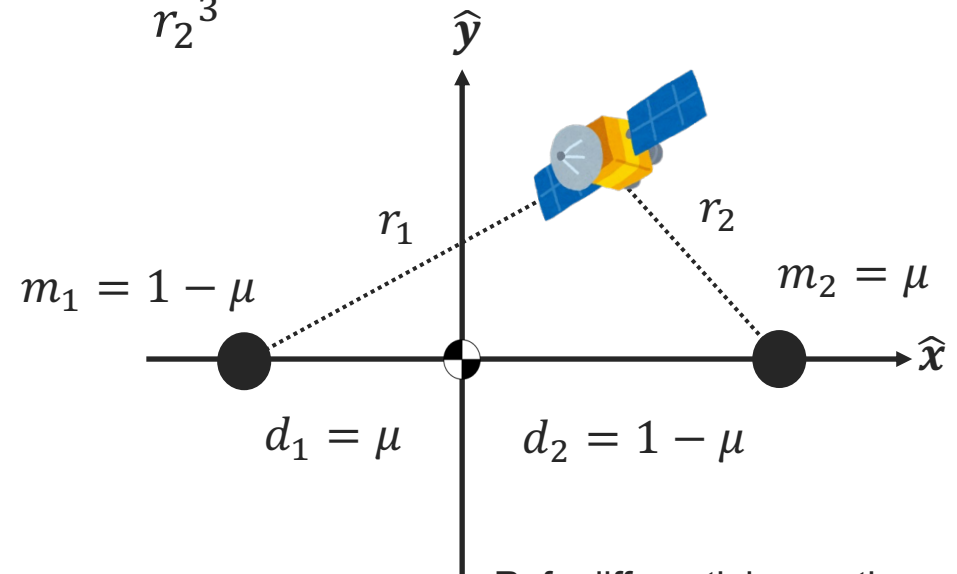
Equations of motion of the CR3BP in the rotating frame are obtained by

$$\left\{ \begin{array}{l} \ddot{x} - 2\dot{y} - x = -\frac{(1-\mu)(x+\mu)}{r_1^3} - \frac{\mu(x-1+\mu)}{r_2^3} \\ \ddot{y} + 2\dot{x} - y = -\frac{(1-\mu)y}{r_1^3} - \frac{\mu y}{r_2^3} \\ \ddot{z} = -\frac{(1-\mu)z}{r_1^3} - \frac{\mu z}{r_2^3} \end{array} \right.$$

where

$$r_1 = \sqrt{(x+\mu)^2 + y^2 + z^2}$$

$$r_2 = \sqrt{(x-1+\mu)^2 + y^2 + z^2}$$



Sample Code : CR3BP in the rotating frame

fun_cr3bp.m

```
function dx = fun_cr3bp(t,x,mu)
% t : non-dimensional time
% x : non-dimensional position and velocity, y = [x y z vx vy vz]'
% mu : mass ratio of the primaries

%the distances
r1 = sqrt((mu+x(1))^2+(x(2))^2+(x(3))^2);
r2 = sqrt((x(1)-1+mu)^2+(x(2))^2+(x(3))^2);

dx = [x(4);
      x(5);
      x(6);
      2*x(5) + x(1) - (1-mu)*(x(1)+mu)/r1^3 - mu*(x(1)-1+mu)/r2^3;
      -2*x(4) + x(2) - (1-mu)*x(2)/r1^3 - mu*x(2)/r2^3;
      - (1-mu)*x(3)/r1^3 - mu*x(3)/r2^3];
end
```

Elliptic Restricted Three-Body Problem (ER3BP)

- Consider the motion of two gravitational bodies P_1, P_2 and a spacecraft P_3 .
- The mass of the spacecraft M_3 is much smaller than the masses of the primaries M_1, M_2 ($M_1 > M_2 \gg M_3$).
- The primaries P_1, P_2 rotate around their barycenter as an **elliptic orbit**.
 - The distance between primaries changes periodically.

Nondimensionalization

Characteristic values

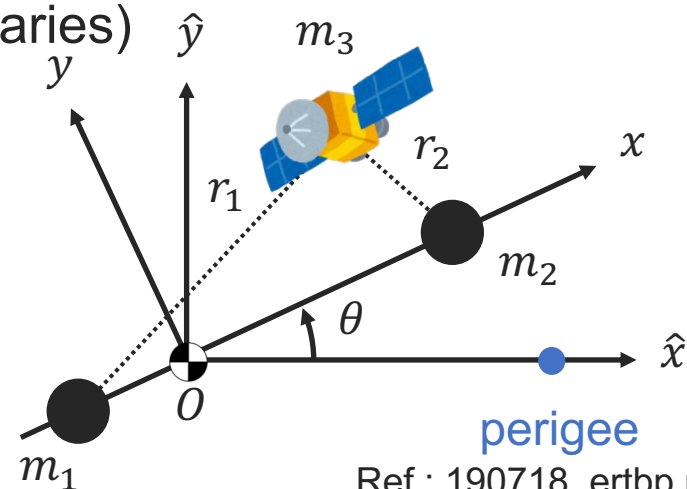
- Length : $L = D$ (the distance between primaries)
- Mass : $M = M_1 + M_2$
- Time : $T = \frac{1}{N}$ (N : mean motion)

O : Barycenter = **Focus**

θ : **True anomaly**

x - y - z : Rotating frame

$\hat{x}$ - $\hat{y}$ - $\hat{z}$: Inertial frame



Equations of Motion in the rotating frame

Equations of motion of the ER3BP in the rotating frame are obtained by

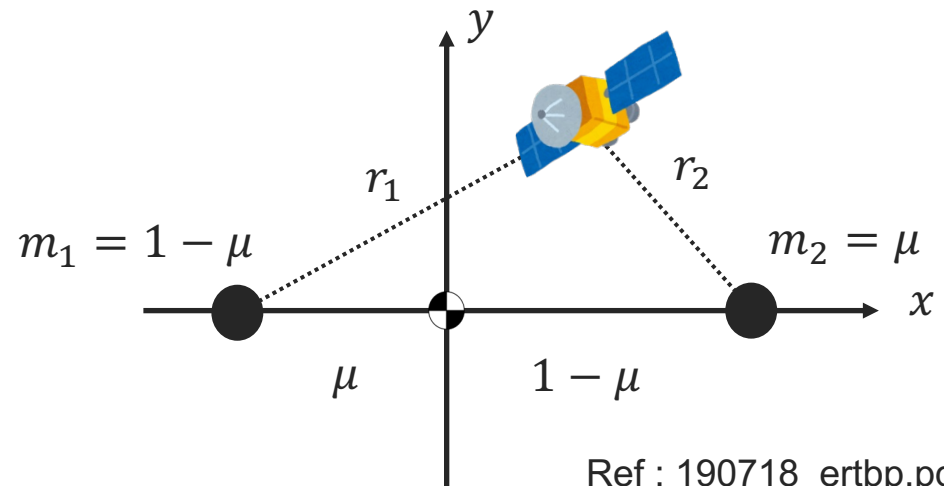
$$\begin{cases} \frac{d^2x}{d\theta^2} - 2\frac{dy}{d\theta} = \frac{1}{1 + e \cos \theta} \left(x - \frac{(1 - \mu)(x + \mu)}{r_1^3} - \frac{\mu(x - 1 + \mu)}{r_2^3} \right) \\ \frac{d^2y}{d\theta^2} + 2\frac{dx}{d\theta} = \frac{1}{1 + e \cos \theta} \left(y - \frac{(1 - \mu)y}{r_1^3} - \frac{\mu y}{r_2^3} \right) \\ \frac{d^2z}{d\theta^2} + z = \frac{1}{1 + e \cos \theta} \left(z - \frac{(1 - \mu)z}{r_1^3} - \frac{\mu z}{r_2^3} \right) \end{cases}$$

where

$$\mu = \frac{M_2}{M_1 + M_2}$$

$$r_1 = \sqrt{(x + \mu)^2 + y^2 + z^2}$$

$$r_2 = \sqrt{(x - 1 + \mu)^2 + y^2 + z^2}$$



Sample Code : ER3BP

fun_er3bp.m

```
function dx = fun_er3bp(t,x,mu,e)
% t : non-dimensional true anomaly
% x : non-dimensional position and velocity, x = [x y z vx vy vz]'
% mu : mass ratio of the primaries
% e : eccentricity

%the distances
r1 = ((mu+x(1))^2+(x(2))^2+(x(3))^2)^0.5;
r2 = ((1-mu-x(1))^2+(x(2))^2+(x(3))^2)^0.5;

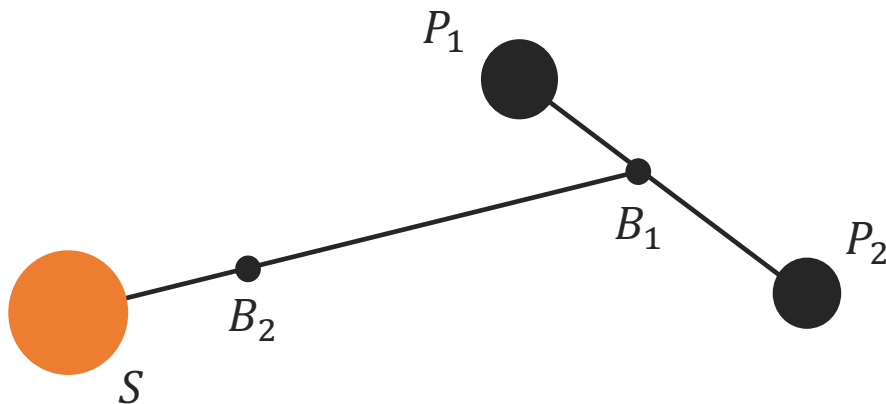
dUdx = x(1) - (1-mu)*(x(1)+mu)/r1^3 - mu*(x(1)-1+mu)/r2^3;
dUdy = x(2) - (1-mu)*x(2)/r1^3 - mu*x(2)/r2^3;
dUdz = x(3) - (1-mu)*x(3)/r1^3 - mu*x(3)/r2^3;

dx = [x(4);
      x(5);
      x(6);
      2*x(5) + dUdx/(1+e*cos(t));
      -2*x(4) + dUdy/(1+e*cos(t));
      - x(3) + dUdz/(1+e*cos(t))];

end
```

Bicircular Restricted Four-Body Problem (BCR4BP)

- The mass of the spacecraft M_3 is much smaller than the masses of the primaries M_1, M_2 and the Sun M_s ($M_s > M_1 > M_2 \gg M_3$).
- The primaries P_1, P_2 rotate around their barycenter B_1 as a **circular orbit**.
- The Sun and B_1 rotate around their barycenter B_2 as a **circular orbit**.
- The Sun does not influence on the motion of P_1 and P_2 .
- The Sun orbit is in the same plane as the $P_1 - P_2$ plane.
(Not always necessary)



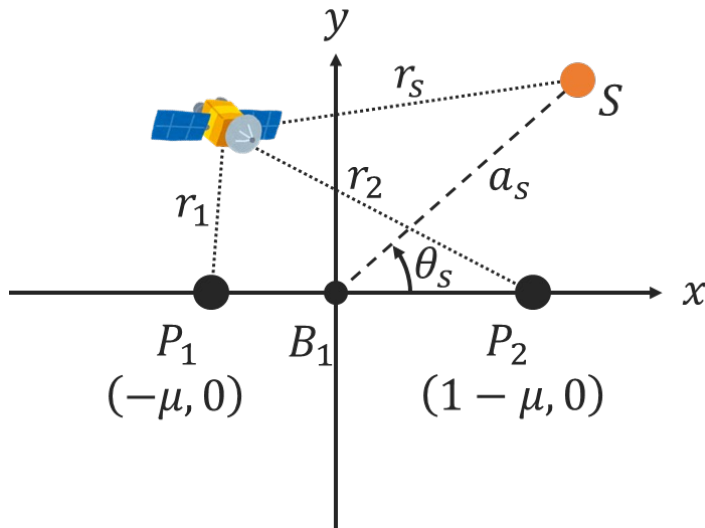
S : Sun
 P_1 : Primary
 P_2 : Secondary
 B_1 : Barycenter of P_1 and P_2
 B_2 : Barycenter of S , P_1 and P_2

Bicircular Restricted Four-Body Problem (BCR4BP)

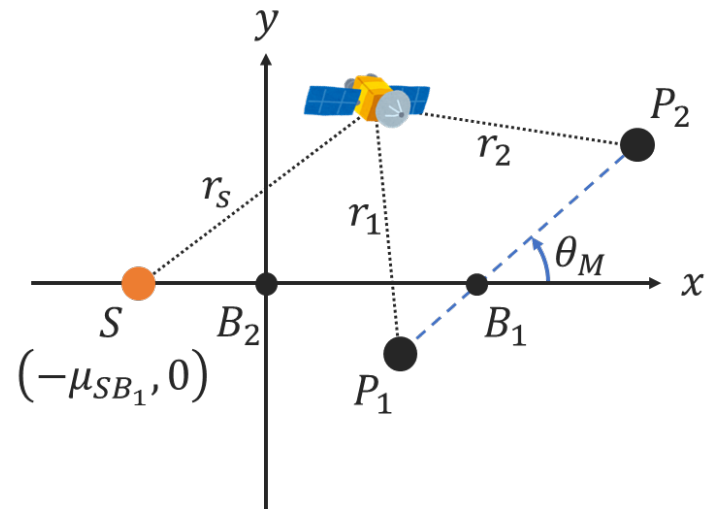
Note : Assumptions mean that this model is **not self-consistent** because the movement of the three primaries is not the solution of the three-body problem. [cf : quasi-bicircular_problem.pdf]

Two major reference frames

Earth-Moon rotating frame



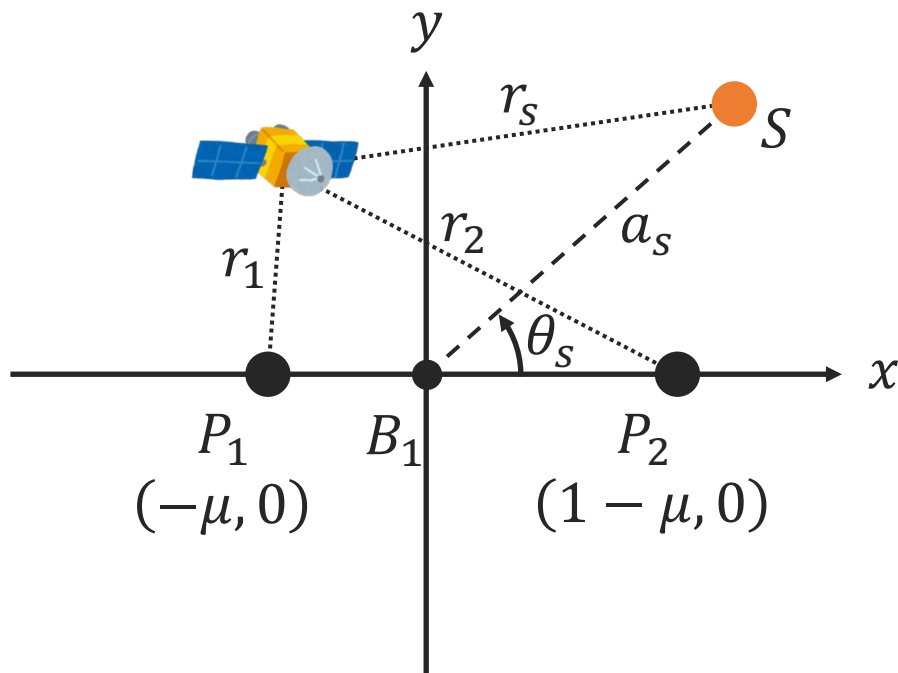
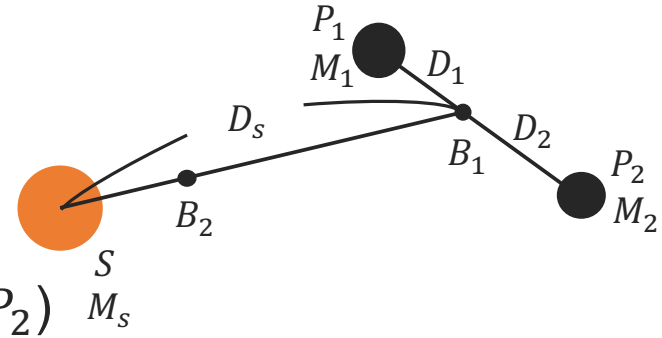
Sun- B_1 rotating frame



Nondimensionalization in the Earth-Moon rotating frame

Characteristic values (same with CR3BP)

- Length : $l^* = D_1 + D_2$
- Mass : $m^* = M_1 + M_2$
- Time : $t^* = \frac{1}{N}$ (N : mean motion of P_1 and P_2)



$$\mu = \frac{M_2}{M_1 + M_2}, \quad m_s = \frac{M_s}{M_1 + M_2}, \quad a_s = \frac{D_s}{L},$$

$$n_s = \sqrt{\frac{1 + m_s}{a_s^3}}, \quad \omega_s = n_s - 1,$$

$$\theta_s = \theta_{s,0} + \omega_s t$$

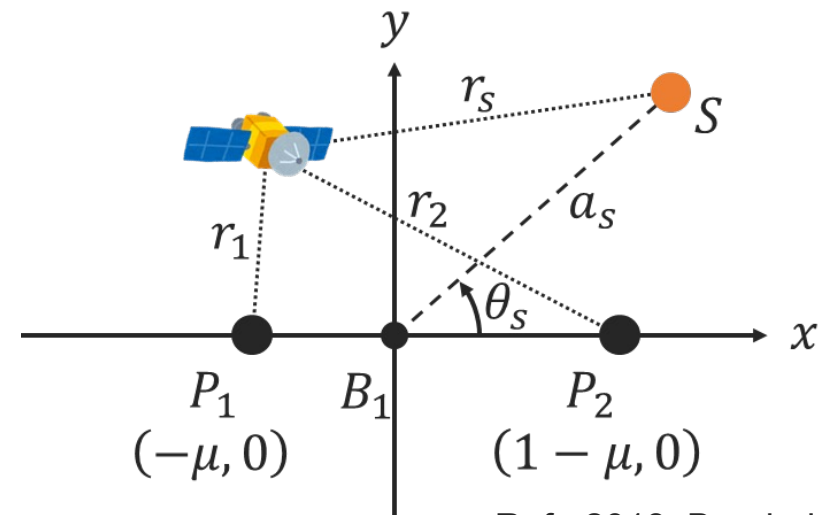
Equations of Motion in the Earth-Moon rotating frame

Equations of motion of the BCR4BP in the Earth-Moon rotating frame are obtained by

$$\begin{cases} \ddot{x} - 2\dot{y} - x = -\frac{(1-\mu)(x+\mu)}{r_1^3} - \frac{\mu(x-1+\mu)}{r_2^3} - \frac{m_s(x-x_s)}{r_s^3} - \frac{m_s}{a_s^3}x_s \\ \ddot{y} + 2\dot{x} - y = -\frac{(1-\mu)y}{r_1^3} - \frac{\mu y}{r_2^3} - \frac{m_s(y-y_s)}{r_s^3} - \frac{m_s}{a_s^3}y_s \\ \ddot{z} = -\frac{(1-\mu)z}{r_1^3} - \frac{\mu z}{r_2^3} - \frac{m_s(z-z_s)}{r_s^3} - \frac{m_s}{a_s^3}z_s \end{cases}$$

where

$$\text{Sun position : } \begin{bmatrix} x_s \\ y_s \\ z_s \end{bmatrix} = \begin{bmatrix} a_s \cos \theta_s \\ a_s \sin \theta_s \\ 0 \end{bmatrix}$$



Sample Code : BCR4BP in the Earth-Moon rotating frame

fun_bcr4bp_EMS.m

```
function dx = fun_bcr4bp_EMS(t,x,mu,mS,aS,wS,thetaS0)
% Earth - Moon rotating frame
%   t : non-dimensional time
%   x : non-dimensional position and velocity, x = [x y z vx vy vz]'
%   mu : mass ratio of Moon, mu = m2 / (m1 + m2)
%   mS : nondimensional mass of Sun
%   aS : nondimensional Sun orbit radius
%   wS : nondimensional Sun angular velocity
% thetaS0 : Sun angle

%the distances
r1      = ((x(1)+mu)^2 + x(2)^2 + x(3)^2)^0.5;
r2      = ((x(1)-1+mu)^2 + x(2)^2 + x(3)^2)^0.5;
Xs      = aS*cos(thetaS0+wS*t);
Ys      = aS*sin(thetaS0+wS*t);
Zs      = 0;
r3      = ((x(1)-Xs)^2 + (x(2)-Ys)^2 + (x(3)-Zs)^2)^0.5;

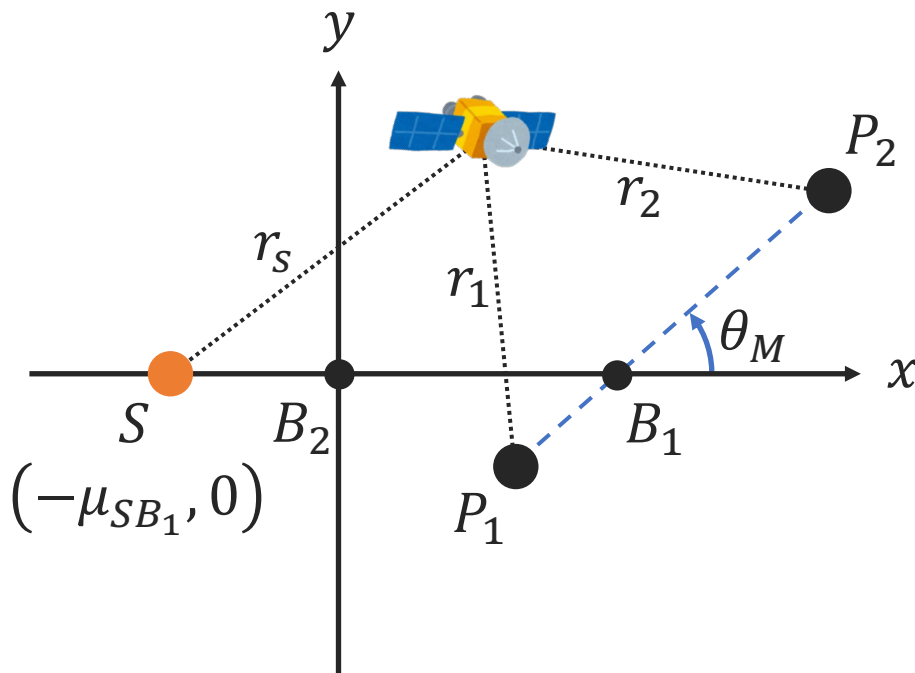
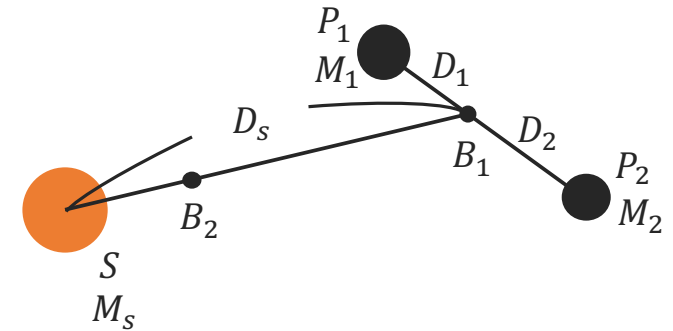
dx      = [x(4);
           x(5);
           x(6);
           2*x(5) + x(1) - (1-mu)*(x(1)+mu)/r1^3 - mu*(x(1)-1+mu)/r2^3 - mS*(x(1)-Xs)/r3^3 - mS/aS^3*Xs;
           -2*x(4) + x(2) - (1-mu)*x(2)/r1^3 - mu*x(2)/r2^3 - mS*(x(2)-Ys)/r3^3 - mS/aS^3*Ys;
           - (1-mu)*x(3)/r1^3 - mu*x(3)/r2^3 - mS*(x(3)-Zs)/r3^3 - mS/aS^3*Zs];

end
```

Nondimensionalization in the Sun- B_1 rotating frame

Characteristic values

- Length : $L = D_s$
- Mass : $M = M_1 + M_2 + M_s$
- Time : $T = \frac{1}{N_s}$ (N_s : mean motion of Sun)



$$\mu_{EM} = \frac{M_2}{M_1 + M_2}, \quad \mu_{SB_1} = \frac{M_1 + M_2}{M_1 + M_2 + M_s},$$

$$a_{EM} = \frac{D_1 + D_2}{L}, \quad a_s = \frac{D_s}{L} = 1,$$

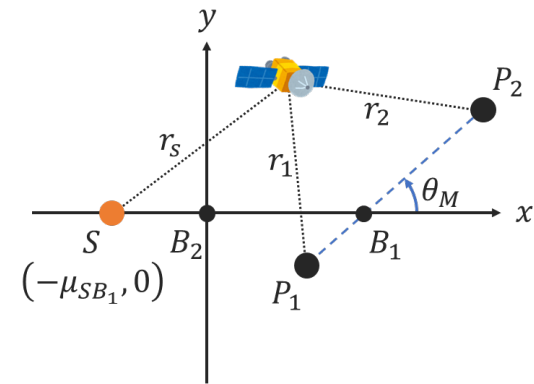
$$\theta_M = \theta_{M,0} + \omega_M t = \pi - \theta_s$$

BCR4BP in the Sun-B₁ rotating frame

$$\begin{cases} \ddot{x} - 2\dot{y} - x = -\frac{(1 - \mu_{SB_1})(x + \mu_{SB_1})}{r_s^3} - \frac{\mu_{SB_1}(1 - \mu_{EM})(x - x_E)}{r_1^3} - \frac{\mu_{SB_1}\mu_{EM}(x - x_M)}{r_2^3} \\ \ddot{y} + 2\dot{x} - y = -\frac{(1 - \mu_{SB_1})y}{r_s^3} - \frac{\mu_{SB_1}(1 - \mu_{EM})(y - y_E)}{r_1^3} - \frac{\mu_{SB_1}\mu_{EM}(y - y_M)}{r_2^3} \\ \ddot{z} = -\frac{(1 - \mu_{SB_1})z}{r_s^3} - \frac{\mu_{SB_1}(1 - \mu_{EM})(z - z_E)}{r_1^3} - \frac{\mu_{SB_1}\mu_{EM}(z - z_M)}{r_2^3} \end{cases}$$

Earth position : $\begin{bmatrix} x_E \\ y_E \\ z_E \end{bmatrix} = \begin{bmatrix} 1 - \mu_{SB_1} - \frac{a_{EM}}{a_s} \mu_{EM} \cos \theta_M \\ -\frac{a_{EM}}{a_s} \mu_{EM} \sin \theta_M \\ 0 \end{bmatrix}$

Moon position : $\begin{bmatrix} x_M \\ y_M \\ z_M \end{bmatrix} = \begin{bmatrix} 1 - \mu_{SB_1} + \frac{a_{EM}}{a_s} (1 - \mu_{EM}) \cos \theta_M \\ \frac{a_{EM}}{a_s} (1 - \mu_{EM}) \sin \theta_M \\ 0 \end{bmatrix}$



Sample Code : BCR4BP in the Sun-B₁ rotating frame

fun_bcr4bp_SB1.m

```
function dx = fun_bcr4bp_SB1(t,x,muSB1,muEM,aS,aEM,wM,thetaM0)
% Sun - B1 rotating frame
%   t : non-dimensional time
%   x : non-dimensional position and velocity, x = [x y z vx vy vz]'
%   muSB1 : mass ratio in Sun - B1 frame, muSB1 = (m1 + m2) / (m1 + m2 + ms)
%   muEM : mass ratio in Earth - Moon frame, muEM = m2 / (m1 + m2)
%   aS : nondimensional Sun orbit radius
%   aEM : nondimensional distance between Earth and Moon
%   wM : nondimensional Moon angular velocity
% thetaM0 : the initial value of Moon angle

%the distances
thata = thetaM0 + wM*t;
rs = ((x(1)+muSB1)^2 + x(2)^2 + x(3)^2)^0.5;
rE = [(-muEM)*cos(thata), (-muEM)*sin(thata), 0]*(aEM/aS)+[1-muSB1, 0, 0];
rM = [(1-muEM)*cos(thata), (1-muEM)*sin(thata), 0]*(aEM/aS)+[1-muSB1, 0, 0];
r1 = norm([x(1)-rE(1), x(2)-rE(2), x(3)-rE(3)]);
r2 = norm([x(1)-rM(1), x(2)-rM(2), x(3)-rM(3)]);

dx = [x(4);
      x(5);
      x(6);
      2*x(5) + x(1) - (1-muSB1)*(x(1)+muSB1)/rs^3 - muSB1*(1-muEM)*(x(1)-rE(1))/r1^3 - muSB1*muEM*(x(1)-rM(1))/r2^3;
      -2*x(4) + x(2) - (1-muSB1)*x(2)/rs^3 - muSB1*(1-muEM)*(x(2)-rE(2))/r1^3 - muSB1*muEM*(x(2)-rM(2))/r2^3;
      - (1-muSB1)*x(3)/rs^3 - muSB1*(1-muEM)*(x(3)-rE(3))/r1^3 - muSB1*muEM*(x(3)-rM(3))/r2^3];

end
```

Example: trajectories in various systems

Ex1_Equations_of_motion.m

This code calculates a L_2 Lyapunov orbit in Earth-Moon CR3BP, and then trajectories in Earth-Moon ER3BP and BCR4BP with **the same initial condition**.

L_2 Lyapunov orbit by fun_cr3bp.m



fun_er3bp.m



fun_bcr4bp_EMS.m



fun_bcr4bp_SB1.m

Example: trajectories in various systems

```
%% initial conditions
Ms = 1.9885e+30; %[kg], mass of Sun, from https://nssdc.gsfc.nasa.gov/planetary/factsheet/sunfact.html
M1 = 5.9724e+24; %[kg], mass of Earth, from https://nssdc.gsfc.nasa.gov/planetary/factsheet/
M2 = 7.346e+22; %[kg], mass of Moon, from https://nssdc.gsfc.nasa.gov/planetary/factsheet/
G = 6.673e-11; %[m^3/(kg*s^2)], gravitational constant

% Earth-Moon CR3BP
chara_length_CR3BP = 3.8440e+8; %[m]
chara_mass_CR3BP = M1 + M2; %[kg]
chara_time_CR3BP = sqrt( chara_length_CR3BP^3 / (G * chara_mass_CR3BP) ); % == period / (2*pi)
mu_EM = M2 / (M1 + M2);
N_CR3BP = sqrt( G*chara_mass_CR3BP / chara_length_CR3BP^3 );

% Earth-Moon ER3BP
a = 3.8440e+8; %[m], semimajor axis, from https://nssdc.gsfc.nasa.gov/planetary/factsheet/
N_ER3BP = sqrt( G*(M1+M2)/(a^3) ); %[1/s], mean motion
e = 0.0549; % eccentricity, from https://nssdc.gsfc.nasa.gov/planetary/factsheet/

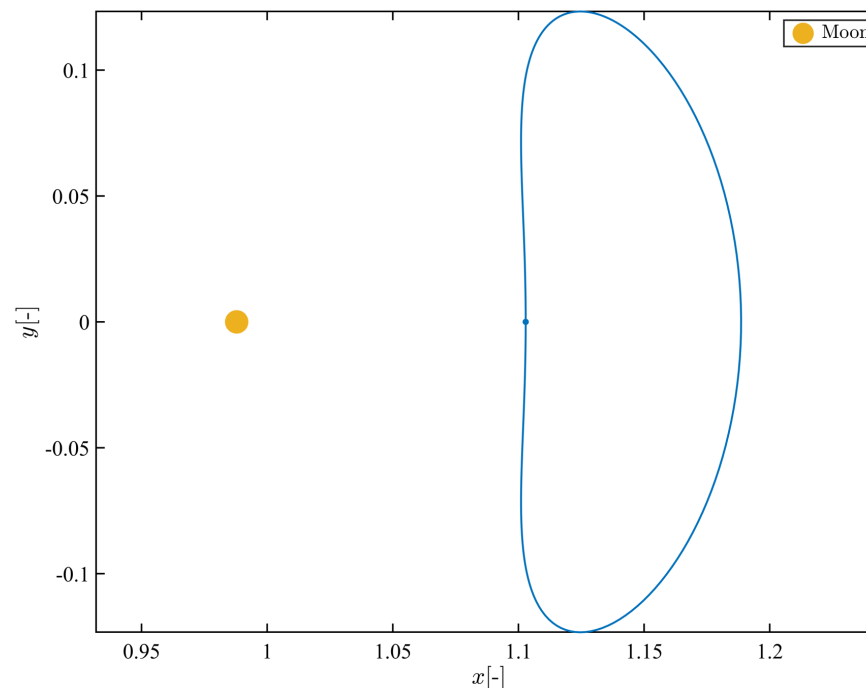
% Earth-Moon-Sun BCR4BP
chara_length_SB1 = 1.4960e+11; %[m]
chara_mass_SB1 = M1 + M2 + Ms; %[kg]
chara_time_SB1 = sqrt( chara_length_SB1^3 / (G * chara_mass_SB1) ); % == period / (2*pi)
mS = Ms / (M1 + M2); % nondimensional mass of Sun
aS = chara_length_SB1 / chara_length_CR3BP; % nondimensional Sun orbit radius
wS = sqrt( (1+mS) / (aS^3) ) - 1; % nondimensional Sun angular velocity
thetaS0 = pi; %[rad], when a total eclipse of the moon occurs
mu_SB1 = (M1 + M2) / (M1 + M2 + Ms); % mass ratio in Sun - B1 frame
aEM = chara_length_CR3BP / chara_length_SB1; % nondimensional distance between Earth and Moon
wM = chara_time_SB1/chara_time_CR3BP - 1; % nondimensional Moon angular velocity
thetaM0 = 0; %[rad], when a total eclipse of the sun occurs

% calculations
options_ODE = odeset('RelTol',1e-14, 'AbsTol',1e-14);
```

The parameters of the systems are declared in this part.
Some values are obtained from
<https://nssdc.gsfc.nasa.gov/planetary/planetfact.html>.

Example 1 : CR3BP

```
%% L2 Lyapunov orbit in Earth-Moon CR3BP
load('estimated_initial_values.mat','x_n_CR3BP','t_n_CR3BP');
t_CR3BP = t_n_CR3BP;
x0_CR3BP = x_n_CR3BP;
tspan1 = [0 t_CR3BP];
[~, x_CR3BP] = ode113(@(t,x) fun_cr3bp(t,x,mu_EM), tspan1, x0_CR3BP, options_ODE);
```



Example 2 : ER3BP

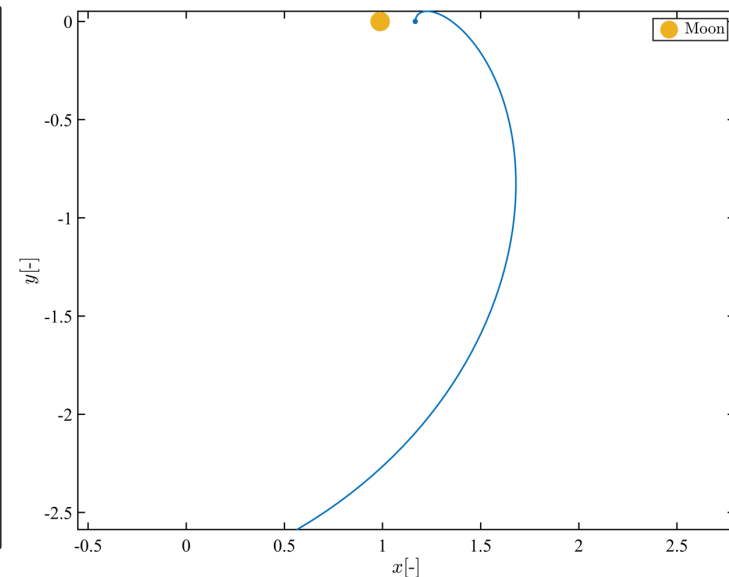
- The initial values of this trajectory is the same to those of the previous trajectory in CR3BP.
- The initial values in CR3BP is transformed to the values in ER3BP.

```
% trajectory in Earth-Moon ER3BP
t_ER3BP = t_CR3BP * chara_time_CR3BP * N_ER3BP;

% transformation from CR3BP reference frame to the inertial frame
X0_CR3BP = [x0_CR3BP(1:3).*chara_length_CR3BP; x0_CR3BP(4:6).*chara_length_CR3BP./chara_time_CR3BP];
C_CR3BP = [cos(0) -sin(0) 0;
           sin(0) cos(0) 0;
           0      0 1];
dtheta_dt_CR3BP = N_CR3BP;
rotating_matrix_CR3BP = fun_rotating_to_inertial_matrix(C_CR3BP, dtheta_dt_CR3BP);
X0_inertial = rotating_matrix_CR3BP * X0_CR3BP;

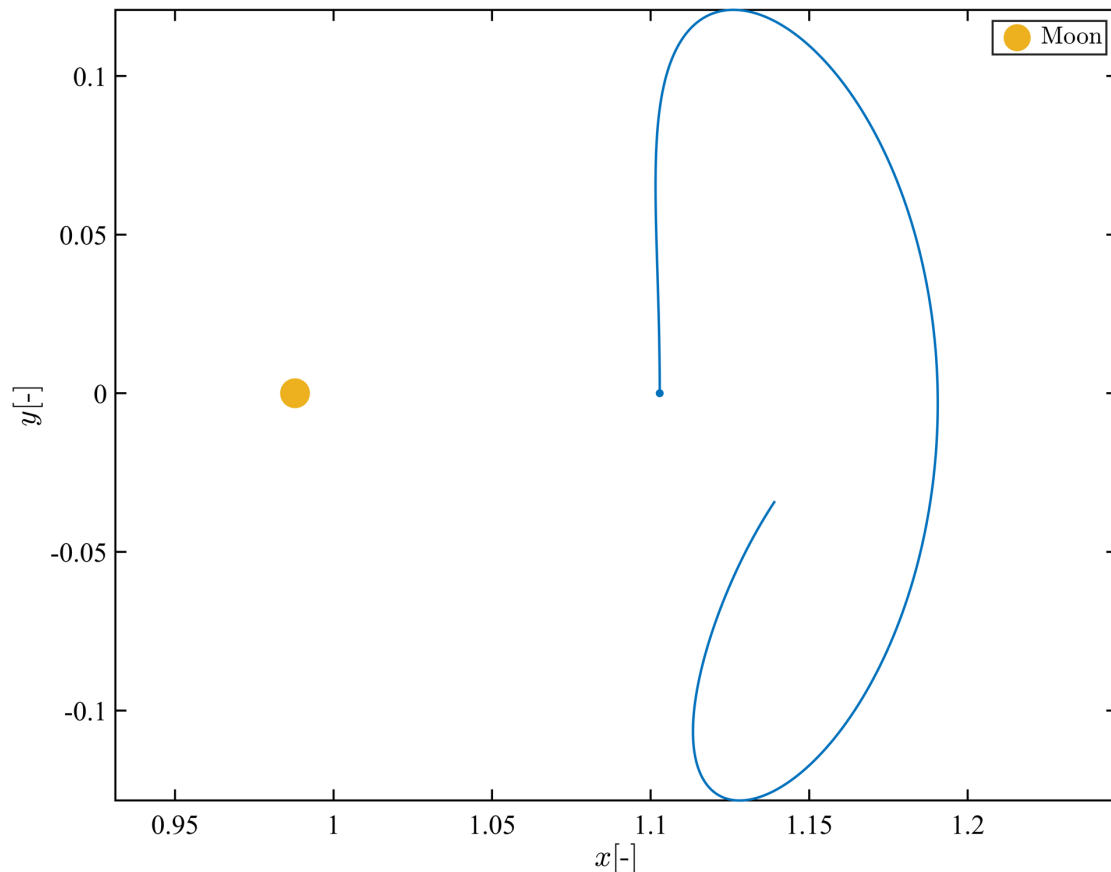
% transformation from the inertial frame to ER3BP reference frame
C_ER3BP = [cos(0) -sin(0) 0;
           sin(0) cos(0) 0;
           0      0 1];
dtheta_dt_ER3BP = sqrt( G*(M1+M2) * (1+e*cos(0))^4 / (a*(1-e^2))^3 );
rotating_matrix_ER3BP = fun_rotating_to_inertial_matrix(C_ER3BP, dtheta_dt_ER3BP);
X0_ER3BP = rotating_matrix_ER3BP \ X0_inertial;
x0_ER3BP = [X0_ER3BP(1:3)./(a*(1-e)); X0_ER3BP(4:6)./(a*(1-e))./N_ER3BP];

tspan2 = [0 t_ER3BP];
[~, x_ER3BP] = ode113(@(t,x) fun_er3bp(t,x,mu_EM,e), tspan2, x0_ER3BP, options_ODE);
```



Example 3 : BCR4BP in the Earth-Moon rotating frame

```
%% trajectory in Earth-Moon_Sun BCR4BP in the Earth-Moon rotating frame
tspan3 = [0 t_CR3BP];
[~, x_bcr4bp_EMS] = ode113(@(t,x) fun_bcr4bp_EMS(t,x,mu_EM,mS,aS,wS,thetaS0), tspan3, x0_CR3BP, options_ODE);
```



Example 4 : BCR4BP in the Sun- B_1 rotating frame

- The initial values of this trajectory is the same to those of the previous trajectory in CR3BP.
- The initial values in CR3BP is transformed to the values in the Sun- B_1 rotating frame.

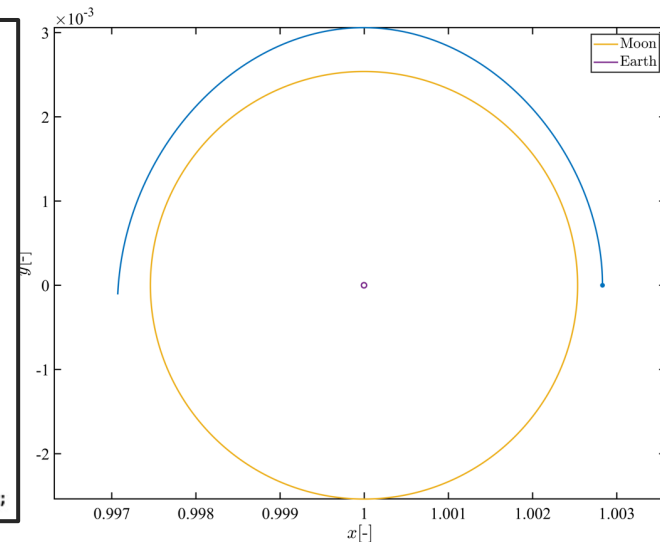
```
% trajectory in Earth-Moon-Sun BCR4BP in the Sun-B1 rotating frame
theta = linspace(0, 2*pi, 1001)';
x_moon = 1 - mu_SB1 + aEM*(1 - mu_EM)*cos(theta);
y_moon = aEM*(1 - mu_EM)*sin(theta);
x_Earth = 1 - mu_SB1 + aEM*mu_EM*cos(theta);
y_Earth = aEM*mu_EM*sin(theta);

t_bcr4bp_SB1 = t_CR3BP * chara_time_CR3BP / chara_time_SB1;

% transformation from the inertial frame to SB1 reference frame
C_SB1 = [cos(theta) -sin(theta) 0;
         sin(theta) cos(theta) 0;
         0 0 1];
dtheta_dt_SB1 = 1/chara_time_SB1;
rotating_matrix_SB1 = fun_rotating_to_inertial_matrix(C_SB1, dtheta_dt_SB1);
X0_bcr4bp_SB1 = rotating_matrix_SB1 \ X0_inertial;
x0_bcr4bp_SB1 = [X0_bcr4bp_SB1(1:3) ./ chara_length_SB1; X0_bcr4bp_SB1(4:6) ./ chara_length_SB1.*chara_time_SB1];

% parallel translation
x0_bcr4bp_SB1(1) = x0_bcr4bp_SB1(1) + (1 - mu_SB1);

tspan4 = [0 t_bcr4bp_SB1];
[~, x_bcr4bp_EMS] = ode113(@(t,x) fun_bcr4bp_SB1(t,x,mu_SB1,mu_EM,1,aEM,wM,thetaM0), tspan4, x0_bcr4bp_SB1, options_ODE);
```



Rotating frame to Inertial frame

When the angular velocity of the rotating frame ${}^I\boldsymbol{\omega}^R$ is expressed as

$${}^I\boldsymbol{\omega}^R = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta} \end{bmatrix}_I$$

I : the inertial frame

R : the rotating frame

and the rotating matrix is defined as

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_I = {}^I C^R \begin{bmatrix} x \\ y \\ z \end{bmatrix}_R \quad \text{where } {}^I C^R = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix},$$

the transformation matrix is written as

$$\begin{bmatrix} x \\ y \\ z \\ \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}_I = \begin{bmatrix} C_{11} & C_{12} & C_{13} & 0 & 0 & 0 \\ C_{21} & C_{22} & C_{23} & 0 & 0 & 0 \\ C_{31} & C_{32} & C_{33} & 0 & 0 & 0 \\ \dot{\theta} C_{12} & -\dot{\theta} C_{11} & 0 & C_{11} & C_{12} & C_{13} \\ \dot{\theta} C_{22} & -\dot{\theta} C_{21} & 0 & C_{21} & C_{22} & C_{23} \\ \dot{\theta} C_{32} & -\dot{\theta} C_{31} & 0 & C_{31} & C_{32} & C_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}_R$$

Rotating frame to Inertial frame

In CR3BP, the position vector is transformed with a simple rotation matrix.

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_I = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}_R$$

Since $\dot{\theta} = n$ (n : mean motion), the transformation matrix is written as

$$\begin{bmatrix} x \\ y \\ z \\ \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}_I = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ -n \sin \theta & -n \cos \theta & 0 & \cos \theta & -\sin \theta & 0 \\ n \cos \theta & -n \sin \theta & 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}_R$$

Sample Code : the transformation matrix

fun_rotating_to_inertial_matrix.m

```
function rotating_matrix = fun_rotating_to_inertial_matrix(C, dtheta_dt)
% C      : the rotating matrix of the position from a rotating frame to the inertial frame, 3x3
% dtheta_dt ; the time variation of theta, theta is the rotating angle, [1/s]

    rotating_matrix = [C(1,1) C(1,2) C(1,3) 0 0 0;
                       C(2,1) C(2,2) C(2,3) 0 0 0;
                       C(3,1) C(3,2) C(3,3) 0 0 0;
                       dtheta_dt*C(1,2) -dtheta_dt*C(1,1) 0 C(1,1) C(1,2) C(1,3);
                       dtheta_dt*C(2,2) -dtheta_dt*C(2,1) 0 C(2,1) C(2,2) C(2,3);
                       dtheta_dt*C(3,2) -dtheta_dt*C(3,1) 0 C(3,1) C(3,2) C(3,3)];
end
```

Concept of equilibrium points

- There are five equilibrium points in CR3BP, satisfying $\dot{x} = \dot{y} = \dot{z} = 0$ and $\ddot{x} = \ddot{y} = \ddot{z} = 0$ in the rotating frame.
- These points are called the **libration points** or **Lagrangian points**, and usually denoted as L_i , where $i = 1, 2, 3, 4, 5$.

Pseudo potential function

The equations of motion are expressed as

$$\left\{ \begin{array}{l} \ddot{x} - 2\dot{y} = \frac{\partial U}{\partial x} \\ \ddot{y} + 2\dot{x} = \frac{\partial U}{\partial y} \\ \ddot{z} = \frac{\partial U}{\partial z} \end{array} \right. \quad \text{where} \quad U = \frac{1}{2}(x^2 + y^2) + \frac{1-\mu}{r_1} + \frac{\mu}{r_2}$$

U : pseudo potential function

Linear part of the equations of motion

- When the equations of motion are expressed as

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \frac{\partial U}{\partial x} + 2\dot{y} \\ \frac{\partial U}{\partial y} - 2\dot{x} \\ \frac{\partial U}{\partial z} \end{bmatrix} \quad \text{where } \mathbf{x} = [x \ y \ z \ \dot{x} \ \dot{y} \ \dot{z}]^T$$

- A is the linear part of $\mathbf{f}(\mathbf{x})$ around the libration point L_i and expressed as

$$A = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \Big|_{L_i} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ \frac{\partial^2 U}{\partial x^2} \Big|_{L_i} & \frac{\partial^2 U}{\partial x \partial y} \Big|_{L_i} & \frac{\partial^2 U}{\partial x \partial z} \Big|_{L_i} & 0 & 2 & 0 \\ \frac{\partial^2 U}{\partial y \partial x} \Big|_{L_i} & \frac{\partial^2 U}{\partial y^2} \Big|_{L_i} & \frac{\partial^2 U}{\partial y \partial z} \Big|_{L_i} & -2 & 0 & 0 \\ \frac{\partial^2 U}{\partial z \partial x} \Big|_{L_i} & \frac{\partial^2 U}{\partial z \partial y} \Big|_{L_i} & \frac{\partial^2 U}{\partial z^2} \Big|_{L_i} & 0 & 0 & 0 \end{bmatrix}$$

Manifolds around the libration points

Eigenvalues of A around the colinear libration points $L_i (i = 1, 2, 3)$ consist of

$\pm\lambda_{c1}, \pm\lambda_{c2}$ (λ_{c1} and λ_{c2} are pure imaginary numbers.)

λ_s (The real part of λ_s is negative.)

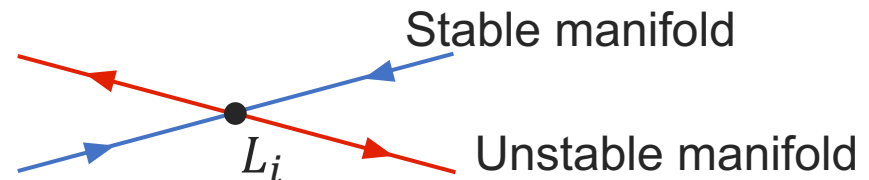
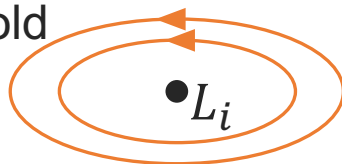
λ_u (The real part of λ_u is positive.)

$\pm\lambda_{c1}, \pm\lambda_{c2}$ $\longleftrightarrow$ **Center manifold** : The set of trajectories which neither converge on nor diverge from L_i

λ_s $\longleftrightarrow$ **Stable manifold** : The set of trajectories which converge on L_i
The eigenvector of λ_s shows the direction of the stable manifold.

λ_u $\longleftrightarrow$ **Unstable manifold** : The set of trajectories which diverge from L_i
The eigenvector of λ_u shows the direction of the unstable manifold.

Center manifold



The triangular libration points $L_i (i = 4, 5)$ have only center manifolds.

Jacobi Constant

- CR3BP has a constant of integration called the **Jacobi Constant** C .

$$\frac{dU}{dt} = \frac{\partial U}{\partial x} \dot{x} + \frac{\partial U}{\partial y} \dot{y} + \frac{\partial U}{\partial z} \dot{z} = (\ddot{x} - 2y)\dot{x} + (\ddot{y} + 2x)\dot{y} + \ddot{z}\dot{z} = \dot{x}\ddot{x} + \dot{y}\ddot{y} + \dot{z}\ddot{z}$$

where U is the pseudo potential function. Then,

$$U = \frac{1}{2}(\dot{x}^2 + \dot{y}^2 + \dot{z}^2 + C)$$

$$C = 2U - (\dot{x}^2 + \dot{y}^2 + \dot{z}^2)$$

- The Jacobi Constant C is constant in a trajectory and serves as a mean to check the accuracy of the numerical integration.

Sample Code: Jacobi Constant

Jacobi_const.m

```
function C = Jacobi_const(x,mu)
% x : non-dimensional position and velocity, x = [x y z vx vy vz]'
% mu : mass ratio of the primaries

%File computes Jacobi Energy for a given state (x,v)

%the distances
r1 = sqrt((mu+x(1))^2+(x(2))^2+(x(3))^2);
r2 = sqrt((1-mu-x(1))^2+(x(2))^2+(x(3))^2);

%Compute the Jacobi Energy
U = (x(1)^2 + x(2)^2)/2 + (1-mu)/r1 + mu/r2;
v = sqrt(x(4)^2 + x(5)^2+x(6)^2);
C = 2*U - v^2;
end
```

Hill's Regions

Since $v^2 = \dot{x}^2 + \dot{y}^2 + \dot{z}^2 \geq 0$,

$$2U - C \geq 0 \Leftrightarrow C \leq 2U$$

Trajectories must satisfy $C \leq 2U$. So, the allowable regions of motion exist in CR3BP, depending on the value of C .

These allowable regions are called **Hill's Regions**.

Zero velocity curve

The boundary surface of Hill's Regions is called zero velocity surface.

In the planar problem, their boundary is called a **zero velocity curve**.

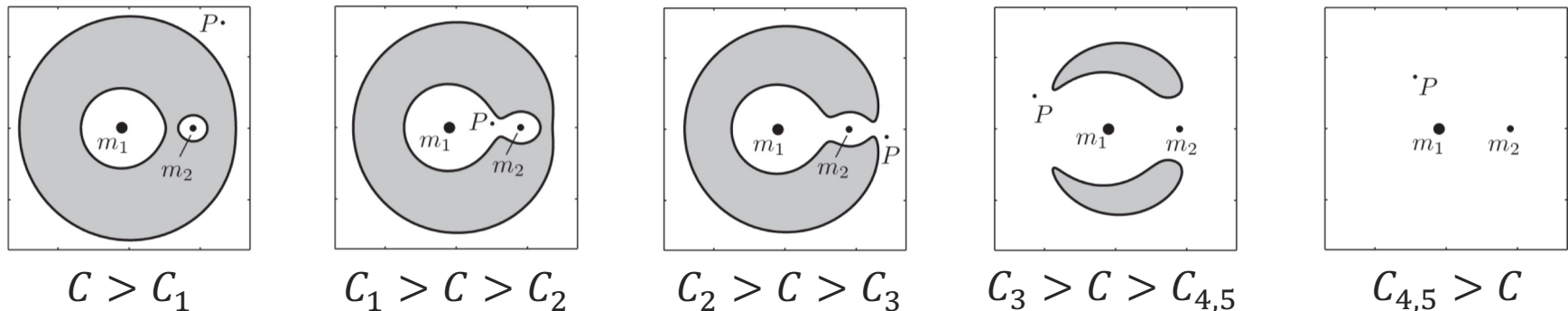


Fig. Forbidden area (Gray area), C_i is the Jacobi Constant of L_i

Example 1: Zero velocity curve

Ex2_Zero_velocity_curve.m

```
%% zero velocity curve
[mu,~,~] = parameter(2); % Earth-Moon,
[L1,L2,L3,L4,L5] = librationPoints(mu);

% co-rotating frame (x,y,z)
[x,y] = meshgrid(-1.5:1e-3:1.5);
[a,b] = size(x);
z = zeros(a,b);

x = reshape(x, [a*b,1]);
y = reshape(y, [a*b,1]);
z = reshape(z, [a*b,1]);

r1 = sqrt((x+mu).^2+y.^2+z.^2);
r2 = sqrt((x-(1-mu)).^2+y.^2+z.^2);
U = 1/2.*(x.^2+y.^2) + (1-mu)./r1 + mu./r2;
C = 2.*U;

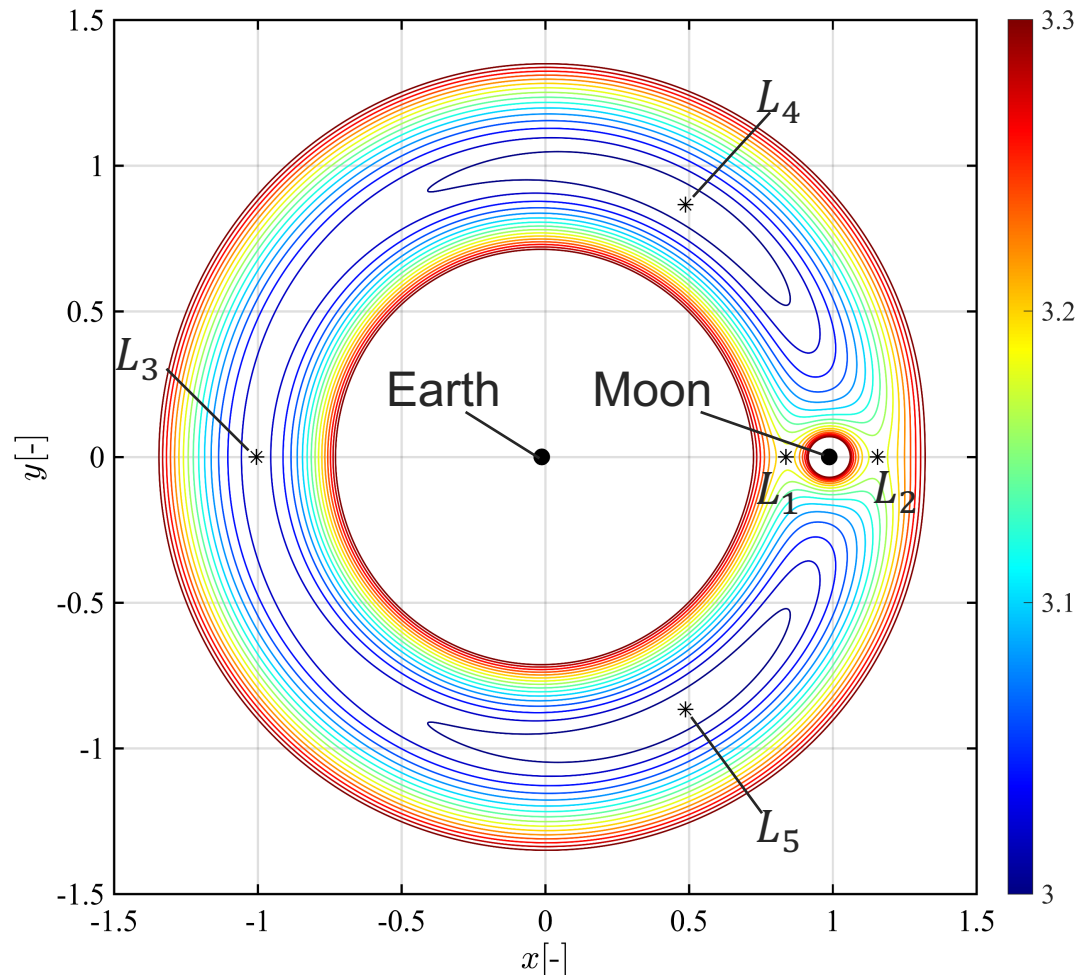
x = reshape(x, [a,b]);
y = reshape(y, [a,b]);
C = reshape(C, [a,b]);

%% show result
Clevel = [3.000 3.020 3.040 3.060 3.080 3.100 3.120 3.140 3.160 3.180 3.200 3.220 3.240 3.260 3.280 3.300];

f1 = figure;
hold on;
contour(x, y, C, Clevel,'LineWidth',1);
colormap jet
colorbar('Ticks',[3.0 3.1 3.2 3.3],'Ticksmode','manual','Limits',[3.0 3.3],'Limitsmode','manual')
plot(-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(1-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L1(1),L1(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L2(1),L2(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L3(1),L3(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L4(1),L4(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L5(1),L5(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
axis equal
xlabel('$x[-]$');
ylabel('$y[-]$');
grid on;
hold off;
f1_name = strcat('Ex2_Zero_Velocity_Curve_mu',num2str(mu));
f1_name = strrep(f1_name, '.', ',');
save_fig_thin(f1,f1_name,[0 90]);
```

Example 1: Zero velocity curve

Zero velocity curve in the Earth-Moon system



Example 2: Stable and unstable manifold of L_i

Ex3_manifold_of_Li.m

```
%% main
[mu,~,~,~] = parameter(2); % Earth-Moon
[L1,L2,L3,L4,L5] = librationPoints(mu);

options_ODE = odeset('RelTol',1e-14, 'AbsTol',1e-14);

% zero velocity curve
[xc,yc] = meshgrid(-1.5:1e-3:1.5);
[a,b] = size(xc);
zc = zeros(a,b);

xc = reshape(xc, [a*b,1]);
yc = reshape(yc, [a*b,1]);
zc = reshape(zc, [a*b,1]);

r1 = sqrt((xc+mu).^2+yc.^2+zc.^2);
r2 = sqrt((xc-(1-mu)).^2+yc.^2+zc.^2);
U = 1/2.*(xc.^2+yc.^2) + (1-mu)./r1 + mu./r2;
C = 2.*U;

xc = reshape(xc, [a,b]);
yc = reshape(yc, [a,b]);
C = reshape(C, [a,b]);
```

parameter.m

```
function [mu, a_1, a_s, w_1] = parameter(y)
switchplanets = y; % 1:Earth, 2:Moon, 3:Mars, 4:Phobos, 5:Deimos

% selection of axis
switch switchplanets
case 1
    % Sun-Earth
    mu = 3.00245E-06; % mass ratio
    a_1 = 1.49598E+08; % primary-secondary distance[km]
    a_s = 4.25874E-05; % secondary radius[nonD]
    w_1 = 1.99097E-07; % ang vel[rad/s]
case 2
    % Earth-Moon
    mu = 1.21536E-02; % mass ratio
    a_1 = 3.84400E+05; % primary-secondary distance[km]
    a_s = 4.52003E-03; % secondary radius[nonD]
    w_1 = 2.66167E-06; % ang vel[rad/s]
case 3
    % Sun-Mars
    mu = 3.22605E-07; % mass ratio
    a_1 = 2.27943824E+08; % primary-secondary distance[km]
    a_s = 1.48699E-05; % secondary radius[nonD]
    w_1 = 1.0585760E-07; % ang vel[rad/s]
case 4
    % Mars-Phobos
    mu = 1.66100E-08; % mass ratio
    a_1 = 9.37600E+03; % primary-secondary distance[km]
    a_s = 1.18387E-03; % secondary radius[nonD]
    w_1 = 2.28040E-04; % ang vel[rad/s]
case 5
    % Mars-Deimos
    mu = 2.30046E-09; % mass ratio
    a_1 = 2.34580E+04; % primary-secondary distance[km]
    a_s = 2.34580E+04; % secondary radius[nonD]
    w_1 = 2.34580E+04; % ang vel[rad/s]
case 6
    % Sun-Jupitar
    mu = 9.5479E-04; % mass ratio
    a_1 = 7.7841E+08; % primary-secondary distance[km]
    a_s = 9.1844E-04; % secondary radius[nonD]
    w_1 = 1.6800E-08; % ang vel[rad/s]
end
```

Example 2: Stable and unstable manifold of L_1

Ex3_manifold_of_Li.m

```
% L1
tf1 = 15;
C1 = Jacobi_const([L1;zeros(3,1)],mu);

sigma = (1-mu)/(abs(L1(1)+mu)^3) + mu/(abs(L1(1)-1+mu)^3);
A = [
    0      0      0      1      0      0;
    0      0      0      0      1      0;
    0      0      0      0      0      1;
    2*sigma+1  0      0      0      2      0;
    0      1-sigma  0      -2      0      0;
    0      0      -sigma  0      0      0];

[V,D] = eig(A);
for i = 1:6
    if imag(D(i,i)) ~= 0
        continue;
    end

    if D(i,i) < 0
        Vs = V(:,i);
    end

    if D(i,i) > 0
        Vu = V(:,i);
    end
end

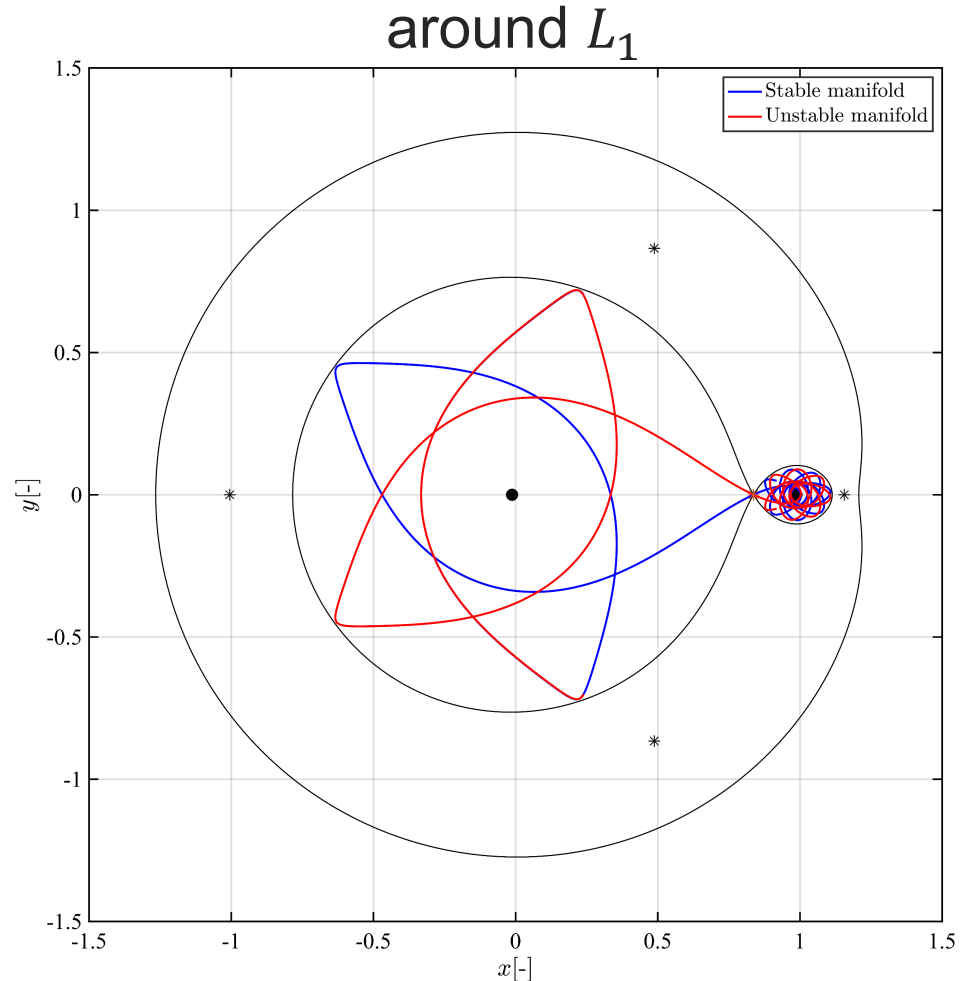
tspan_s = [tf1 0];
x0_sp = [L1;zeros(3,1)] + 1e-10.*Vs/norm(Vs);
[~, xsp_L1] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, x0_sp, options_0DE);
xsp_L1 = flipud(xsp_L1);

x0_sm = [L1;zeros(3,1)] - 1e-10.*Vs/norm(Vs);
[~, xsm_L1] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, x0_sm, options_0DE);
xsm_L1 = flipud(xsm_L1);

tspan_u = [0 tf1];
x0_up = [L1;zeros(3,1)] + 1e-10.*Vu/norm(Vu);
[~, xup_L1] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, x0_up, options_0DE);

x0_um = [L1;zeros(3,1)] - 1e-10.*Vu/norm(Vu);
[~, xum_L1] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, x0_um, options_0DE);

f1 = figure;
hold on;
contour(xc, yc, C1, 'k', 'LineWidth', 1);
plot(-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(1-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
stable1 = plot(xsp_L1(:,1), xsp_L1(:,2), 'b');
plot(xsm_L1(:,1), xsm_L1(:,2), 'b');
unstable1 = plot(xup_L1(:,1), xup_L1(:,2), 'r');
plot(xum_L1(:,1), xum_L1(:,2), 'r');
plot(L1(1),L1(2),'*','MarkerFaceColor','#D95319','MarkerEdgeColor','#D95319','MarkerSize',10);
plot(L2(1),L2(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L3(1),L3(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L4(1),L4(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L5(1),L5(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
axis equal;
xlabel('x[-]');
ylabel('y[-]');
grid on;
legend([stable1, unstable1], {'Stable manifold', 'Unstable manifold'});
hold off;
f1_name = strcat('Ex3_manifold_of_L1_mu=', num2str(mu), '_C1=', num2str(C1), '_tf=', num2str(tf1));
f1_name = strrep(f1_name, ',', '');
save_fig(f1,f1_name,[0 90]);
```



Example 2: Stable and unstable manifold of L_i

Ex3_manifold_of_Li.m

around L_2

```
% L2
tf2 = 25;
C2 = Jacobi_const([L2;zeros(3,1)],mu);

sigma = (1-mu)/(abs(L2(1)+mu)^3) + mu/(abs(L2(1)-1+mu)^3);
A = [
    0      0      0      1      0      0;
    0      0      0      0      1      0;
    0      0      0      0      0      1;
    2*sigma+1  0      0      0      2      0;
    0 1-sigma  0      0      -2      0;
    0      0      -sigma  0      0      0];

[V,D] = eig(A);
for i = 1:6
    if imag(D(i,i)) ~= 0
        continue;
    end

    if D(i,i) < 0
        Vs = V(:,i);
    end

    if D(i,i) > 0
        Vu = V(:,i);
    end
end

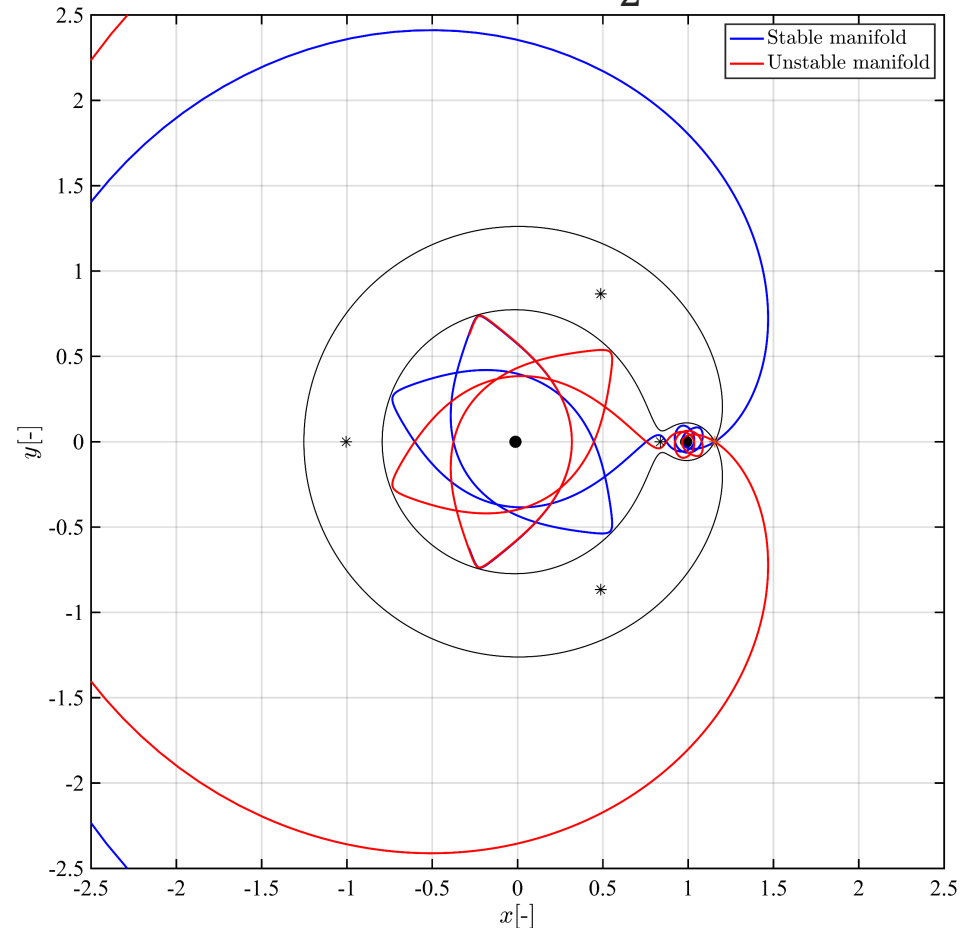
tspan_s = [tf2 0];
x0_sp = [L2;zeros(3,1)] + 1e-10.*Vs/norm(Vs);
[~, xsp_L2] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, x0_sp, options_ODE);
xsp_L2 = flipud(xsp_L2);

x0_sm = [L2;zeros(3,1)] - 1e-10.*Vs/norm(Vs);
[~, xsm_L2] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, x0_sm, options_ODE);
xsm_L2 = flipud(xsm_L2);

tspan_u = [0 tf2];
x0_up = [L2;zeros(3,1)] + 1e-10.*Vu/norm(Vu);
[~, xup_L2] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, x0_up, options_ODE);

x0_um = [L2;zeros(3,1)] - 1e-10.*Vu/norm(Vu);
[~, xum_L2] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, x0_um, options_ODE);

f2 = figure;
hold on;
contour(xc, yc, C, [C2 C2], 'k', 'LineWidth', 1);
plot(-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(1-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
stable2 = plot(xsp_L2(:,1), xsp_L2(:,2), 'b');
plot(xsm_L2(:,1), xsm_L2(:,2), 'b');
unstable2 = plot(xup_L2(:,1), xup_L2(:,2), 'r');
plot(xum_L2(:,1), xum_L2(:,2), 'r');
plot(L2(1),L2(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L1(1),L1(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L3(1),L3(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L4(1),L4(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L5(1),L5(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
axis equal
xlabel('x[-]);
ylabel('y[-]);
grid on;
xlim([-2.5 2.5]);
ylim([-2.5 2.5]);
legend([stable2, unstable2], {'Stable manifold', 'Unstable manifold'});
hold off;
f2_name = strcat('Ex3_manifold_of_L2_mu=', num2str(mu), '_C2=', num2str(C2), '_tf=', num2str(tf2));
f2_name = strrep(f2_name, ',', '');
save_fig(f2, f2_name, [0 90]);
```



Example 2: Stable and unstable manifold of L_i

Ex3_manifold_of_Li.m

around L_3

```
% L3
tf3 = 145;
C3 = Jacobi_const([L3;zeros(3,1)],mu);

sigma = (1-mu)/(abs(L3(1)+mu)^3) + mu/(abs(L3(1)-mu)^3);
A = [
    0      0      0      1      0      0;
    0      0      0      0      1      0;
    0      0      0      0      0      1;
    2*sigma+1  0      0      0      2      0;
    0 1-sigma  0      0      0      0;
    0      0      0 -sigma  0      0];

[V,D] = eig(A);
for i = 1:6
    if imag(D(i,i)) ~= 0
        continue;
    end

    if D(i,i) < 0
        Vs = V(:,i);
    end

    if D(i,i) > 0
        Vu = V(:,i);
    end
end

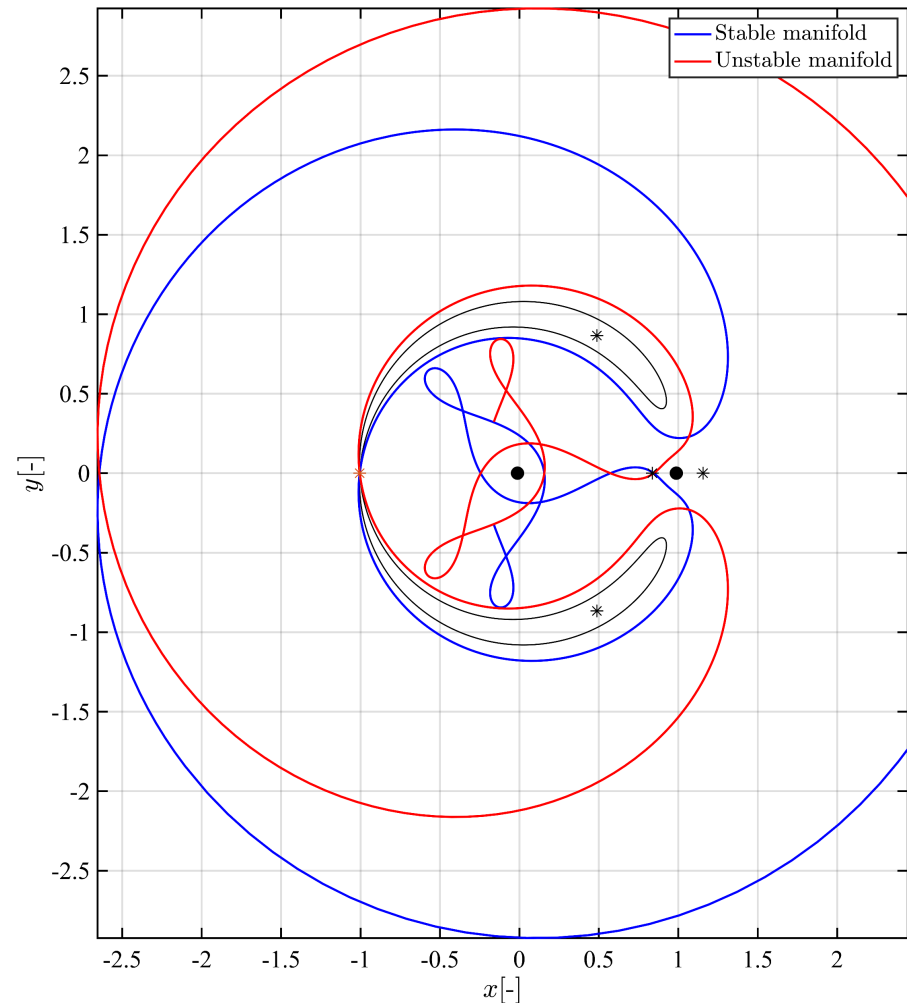
tspan_s = [tf3 0];
x0_sp = [L3;zeros(3,1)] + 1e-10.*Vs/norm(Vs);
[~, xsp_L3] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, x0_sp, options_ODE);
xsp_L3 = flipud(xsp_L3);

x0_sm = [L3;zeros(3,1)] - 1e-10.*Vs/norm(Vs);
[~, xsm_L3] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, x0_sm, options_ODE);
xsm_L3 = flipud(xsm_L3);

tspan_u = [0 tf3];
x0_up = [L3;zeros(3,1)] + 1e-10.*Vu/norm(Vu);
[~, xup_L3] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, x0_up, options_ODE);

x0_um = [L3;zeros(3,1)] - 1e-10.*Vu/norm(Vu);
[~, xum_L3] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, x0_um, options_ODE);

f3 = figure;
hold on;
contour(xc, yc, C, [C3 C3], 'k', 'LineWidth', 1);
plot(-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(1-mu,0,'o','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
stable3 = plot(xsp_L3(:,1), xsp_L3(:,2), 'b');
plot(xsm_L3(:,1), xsm_L3(:,2), 'b');
unstable3 = plot(xup_L3(:,1), xup_L3(:,2), 'r');
plot(xum_L3(:,1), xum_L3(:,2), 'r');
plot(L3(1),L3(2),'*', 'MarkerFaceColor','d','MarkerEdgeColor','d','MarkerSize',10);
plot(L1(1),L1(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L2(1),L2(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L4(1),L4(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
plot(L5(1),L5(2),'*', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
axis equal
xlabel('x[-]');
ylabel('y[-]');
grid on;
legend([stable3, unstable3], {'Stable manifold', 'Unstable manifold'});
hold off;
f3_name = strcat('Ex3_manifold_of_L3_mu=', num2str(mu), '_C3=', num2str(C3), '_tf=', num2str(tf3));
f3_name = strrep(f3_name, ',', '');
save_fig(f3,f3_name,[0 90]);
```

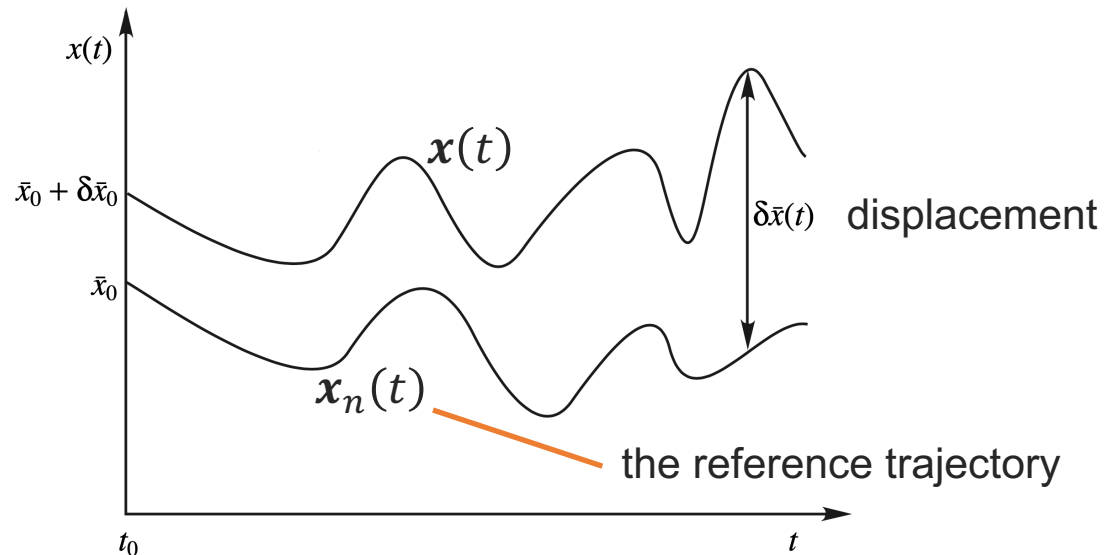


Concept of the state transition matrix (STM)

- The state transition matrix $\Phi(t, t_0)$ is defined as $\Phi(t, t_0) = \frac{\delta x(t)}{\delta x(t_0)}$.
- STM gives the linear relationship between small initial and final displacements as

$$\delta x(t) = \Phi(t, t_0) \delta x(t_0)$$

- Therefore, STM is useful in the differential correction.



Computation of STM

The equations of motion linearized along the reference trajectory is expressed as

$$\delta \dot{\mathbf{x}} = \mathbf{A} \delta \mathbf{x}$$

where

$$\mathbf{A} = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{x}_n} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ \left. \frac{\partial^2 U}{\partial x^2} \right|_{\mathbf{x}_n} & \left. \frac{\partial^2 U}{\partial x \partial y} \right|_{\mathbf{x}_n} & \left. \frac{\partial^2 U}{\partial x \partial z} \right|_{\mathbf{x}_n} & 0 & 2 & 0 \\ \left. \frac{\partial^2 U}{\partial y \partial x} \right|_{\mathbf{x}_n} & \left. \frac{\partial^2 U}{\partial y^2} \right|_{\mathbf{x}_n} & \left. \frac{\partial^2 U}{\partial y \partial z} \right|_{\mathbf{x}_n} & -2 & 0 & 0 \\ \left. \frac{\partial^2 U}{\partial z \partial x} \right|_{\mathbf{x}_n} & \left. \frac{\partial^2 U}{\partial z \partial y} \right|_{\mathbf{x}_n} & \left. \frac{\partial^2 U}{\partial z^2} \right|_{\mathbf{x}_n} & 0 & 0 & 0 \end{bmatrix}$$

Computation of STM

- Since STM satisfies $\delta \mathbf{x}(t) = \Phi(t, t_0) \delta \mathbf{x}(t_0)$,

$$\dot{\Phi}(t, t_0) \delta \mathbf{x}(t_0) = A(t) \Phi(t, t_0) \delta \mathbf{x}(t_0)$$

$$\dot{\Phi}(t, t_0) = A(t) \Phi(t, t_0)$$

- STM is calculated by solving this ODE with the initial value $\Phi(t_0, t_0) = I$.
- In the numerical computation, ODE of STM and the equations of motion are calculate at the same time.

Note :

STM has the following properties:

$$\Phi(t_2, t_0) = \Phi(t_2, t_1) \Phi(t_1, t_0)$$

$$\Phi(t_0, t_1) = \Phi^{-1}(t_1, t_0)$$

Sample Code: State transition matrix

fun_stm_cr3bp.m

```
function dx = fun_stm_cr3bp(t,x,mu)
% t : non-dimensional time
% x : non-dimensional position and velocity, x = [x y z vx vy vz]'
% mu : mass ratio of the primaries

%% equations of motion
r1 = sqrt((mu+x(1))^2+(x(2))^2+(x(3))^2);
r2 = sqrt((1-mu-x(1))^2+(x(2))^2+(x(3))^2);
dx(1:6,1) = [x(4);
             x(5);
             x(6);
             2*x(5) + x(1) - (1-mu)*(x(1)+mu)/r1^3 - mu*(x(1)-1+mu)/r2^3;
             -2*x(4) + x(2) - (1-mu)*x(2)/r1^3 - mu*x(2)/r2^3;
             - (1-mu)*x(3)/r1^3 - mu*x(3)/r2^3];

%% stm
A = fun_A_cr3bp(x(1:6),mu);
phi = reshape(x(7:end),6,6);
dphi = A*phi;
dx(7:42,1) = dphi(:);
end
```

fun_A_cr3bp.m

```
function A = fun_A_cr3bp(X, mu)
% X : non-dimensional position and velocity, X = [x y z vx vy vz]'
% mu : mass ratio of the primaries

%% Generate A Matrix
r1 = sqrt((mu+X(1))^2+(X(2))^2+(X(3))^2);
r2 = sqrt((1-mu-X(1))^2+(X(2))^2+(X(3))^2);

Uxx      = 1 - (1-mu)/r1^3 - mu/r2^3 + 3*(1-mu)*X(1)+mu)^2/r1^5 + 3*mu*(X(1)-1+mu)^2/r2^5;
Uyy      = 1 - (1-mu)/r1^3 - mu/r2^3 + 3*(1-mu)*X(2)^2/r1^5 + 3*mu*X(2)^2/r2^5;
Uzz      = - (1-mu)/r1^3 - mu/r2^3 + 3*(1-mu)*X(3)^2/r1^5 + 3*mu*X(3)^2/r2^5;
Uxy      = 3*(1-mu)*(X(1)+mu)*X(2)/r1^5 + 3*mu*(X(1)-1+mu)*X(2)/r2^5;
Uxz      = 3*(1-mu)*(X(1)+mu)*X(3)/r1^5 + 3*mu*(X(1)-1+mu)*X(3)/r2^5;
Uyz      = 3*(1-mu)*X(2)*X(3)/r1^5 + 3*mu*X(2)*X(3)/r2^5;
B        = [Uxx,Uxy,Uxz;
            Uxy,Uyy,Uyz;
            Uxz,Uyz,Uzz];

C        = [0 2 0; -2 0 0; 0 0 0];

A        = [zeros(3), eye(3); B, C];
end
```

Monodromy matrix

- STM of a periodic orbit after one period T is $\Phi(T) = \Phi(T, 0)$.
- This matrix is called the **monodromy matrix** M of the periodic orbit.
- The eigenvalues λ_i ($i = 1, 2, \dots, 6$) of the monodromy matrix M of a periodic orbit in CR3BP have some characteristics.
 1. At least two eigenvalues **equal to one**.
 2. If $\lambda_i \in \mathbb{C}$ is an eigenvalue of M , then $\frac{1}{\lambda_i}$, $\bar{\lambda}_i$, $\frac{1}{\bar{\lambda}_i}$ are also eigenvalues of M .
- Since the monodromy matrix M is a discrete mapping, the stability boundary is the unit circle, such that

$$\|\lambda_i\| = \begin{cases} < 1 & \text{stable} \\ = 1 & \text{center} \\ > 1 & \text{unstable} \end{cases}$$

Differential correction (Single shooting)

- The goal of the differential corrections process is to adjust the estimated initial condition and period until those values for a periodic orbit are acquired.
- Therefore, the objective function $F(X)$ is expressed as

$$F(X) = x(t_f; x_0) - x(t_0; x_0)$$

where $X = \begin{bmatrix} x_0 \\ t_f \end{bmatrix}$, and $x(t; x_0)$ indicates the state variables at time t with the initial condition x_0 . (This means $x(t_0; x_0) = x_0$.)
 $x(t_0; x_0)$ can be replaced with any desired final state $x_d(t_f; x_0)$.

- If the ideal value that satisfies the above equation is expressed as X^* , a first order Taylor expansion yields

$$F(X^*) = F(X_0) + \mathbf{DF}(X_0)(X^* - X_0) = 0$$

where $\mathbf{DF}(X) = \frac{\partial F}{\partial X}(X)$ is the $m \times n$ Jacobian matrix.

Differential correction (Single shooting)

- $\mathbf{DF}(X)$ is expressed as

$$\mathbf{DF}(X) = [\Phi(t_f, t_0) - I \mid \dot{x}(t_f; x_0)]$$

where Φ is the state transition matrix.

- Then,

$$X^* = X_0 - \mathbf{DF}^{-1}(X_0)F(X_0)$$

- Based on the Newton method, X^* is obtained by the iteration of i .

$$X_{i+1} = X_i - \mathbf{DF}^{-1}(X_i)F(X_i) \quad (m = n)$$

$$X_{i+1} = X_i - [\mathbf{DF}^T(X_i)\mathbf{DF}(X_i)]^{-1}\mathbf{DF}^T(X_i)F(X_i) \quad (m > n)$$

$$X_{i+1} = X_i - \mathbf{DF}^T(X_i)[\mathbf{DF}(X_i)\mathbf{DF}^T(X_i)]^{-1}F(X_i) \quad (m < n)$$

- The iteration ends when $\|F(X_i)\| \ll 1$.

Differential correction for a Lyapunov orbit

- In CR3BP, there exists the symmetry about $y = 0$ plane.
- Considering this symmetry, the initial state $\mathbf{x}_0$ and desired final state $\mathbf{x}_d$ are expressed as

$$\mathbf{x}_0 = [x_0 \quad 0 \quad 0 \quad 0 \quad \dot{y}_0 \quad 0]^T$$

$$\mathbf{x}_d = [x_d \quad 0 \quad 0 \quad 0 \quad \dot{y}_d \quad 0]^T$$

where $t_0 = 0$ and $t_f = \frac{1}{2}T$ (T is the estimated period of a Lyapunov orbit.).

- If $\mathbf{x}(t_f; \mathbf{x}_0) = [x_f \quad y_f \quad 0 \quad \dot{x}_f \quad \dot{y}_f \quad 0]^T$, the objective function is defined as

$$\mathbf{X} = \begin{bmatrix} \dot{y}_0 \\ t_f \end{bmatrix}, \quad \mathbf{F}(\mathbf{X}) = \begin{bmatrix} y_f - 0 \\ \dot{x}_f - 0 \end{bmatrix}$$

- Here, $\mathbf{x}_0$ is fixed and only $\dot{y}_0, t_f$ are corrected in the calculation.

Differential correction for a Lyapunov orbit

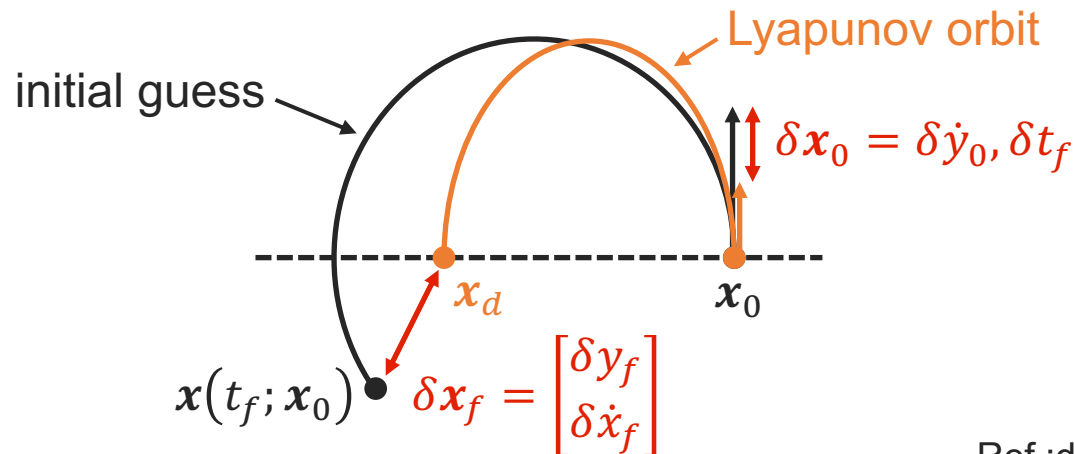
- If $\Phi(t_f, t_0)$ is expressed as $\Phi(t_f, t_0) = [\Phi_{i,j}]$,

$$\mathbf{DF}(X) = \begin{bmatrix} \Phi_{2,5} & \dot{y}_f \\ \Phi_{4,5} & \ddot{x}_f \end{bmatrix} \quad \left(\begin{bmatrix} \delta y_f \\ \delta \dot{x}_f \end{bmatrix} = \begin{bmatrix} \Phi_{2,5} & \dot{y}_f \\ \Phi_{4,5} & \ddot{x}_f \end{bmatrix} \begin{bmatrix} \delta y_0 \\ \delta t_f \end{bmatrix} \right)$$

- Then, the differential correction process is expressed as

$$X_{i+1} = X_i - \mathbf{DF}^{-1}(X_i)F(X_i)$$

- The initial condition for a Lyapunov orbit of **period** $2t_f$ is obtained through this iteration.



Differential correction for a halo orbit

- Considering the symmetry about $y = 0$ plane, the initial state $\mathbf{x}_0$ and desired final state $\mathbf{x}_d$ are expressed as

$$\mathbf{x}_0 = [x_0 \quad 0 \quad \mathbf{z}_0 \quad 0 \quad \dot{y}_0 \quad 0]^T$$

$$\mathbf{x}_d = [x_d \quad 0 \quad \mathbf{z}_d \quad 0 \quad \dot{y}_d \quad 0]^T$$

where $t_0 = 0$ and $t_f = \frac{1}{2}T$ (T is the estimated period of a halo orbit.).

- If $\mathbf{x}(t_f; \mathbf{x}_0) = [x_f \quad y_f \quad z_f \quad \dot{x}_f \quad \dot{y}_f \quad \dot{z}_f]^T$, the objective function is defined as

$$\mathbf{X} = \begin{bmatrix} z_0 \\ \dot{y}_0 \\ t_f \end{bmatrix}, \quad \mathbf{F}(\mathbf{X}) = \begin{bmatrix} y_f - 0 \\ \dot{x}_f - 0 \\ \dot{z}_f - 0 \end{bmatrix}$$

- Here, $\mathbf{x}_0$ is fixed and only $z_0, \dot{y}_0, t_f$ are corrected in the calculation.

Differential correction for a halo orbit

- If $\Phi(t_f, t_0)$ is expressed as $\Phi(t_f, t_0) = [\Phi_{i,j}]$,

$$\mathbf{DF}(X) = \begin{bmatrix} \Phi_{2,3} & \Phi_{2,5} & \dot{y}_f \\ \Phi_{4,3} & \Phi_{4,5} & \ddot{x}_f \\ \Phi_{6,3} & \Phi_{6,5} & \ddot{z}_f \end{bmatrix} \quad \left(\begin{bmatrix} \delta y_f \\ \delta \dot{x}_f \\ \delta \dot{z}_f \end{bmatrix} = \begin{bmatrix} \Phi_{2,3} & \Phi_{2,5} & \dot{y}_f \\ \Phi_{4,3} & \Phi_{4,5} & \ddot{x}_f \\ \Phi_{6,3} & \Phi_{6,5} & \ddot{z}_f \end{bmatrix} \begin{bmatrix} \delta z_0 \\ \delta \dot{y}_0 \\ \delta t_f \end{bmatrix} \right)$$

- Then, the differential correction process is expressed as

$$X_{i+1} = X_i - \mathbf{DF}^{-1}(X_i)F(X_i)$$

- The initial condition for a halo orbit of **period $2t_f$** is obtained through this iteration.

Sample Code: Differential correction

fun_differential_correction_cr3bp.m

```
% differential correction for Lyapunov and halo orbits in CR3BP
function [x_n,t_n,C] = fun_differential_correction_cr3bp(x0,t0,mu,options)
% x0      : initial guess of x
% t0      : initial guess of period
% mu      : mass ratio of the primaries
% options : options for ode

X0 = [x0(1),x0(2),x0(3),x0(4),x0(5),x0(6),reshape(eye(6),1,[])];
tspan = [0,t0];
[~,Y] = ode113(@(t,x) fun_stm_cr3bp(t,x,mu),tspan,X0,options);

X = Y(end,1:6);
Phi = reshape(Y(end,7:end),6,6);
f_x = fun_cr3bp([],X,mu);
DF = [Phi(2,3), Phi(2,5), f_x(2);
      Phi(4,3), Phi(4,5), f_x(4);
      Phi(6,3), Phi(6,5), f_x(6)];
x_ast = [X0(3); X0(5); t0]-DF\[X(2); X(4); X(6)];

x_n = [x0(1);0;x_ast(1);0;x_ast(2);0];
t_n = x_ast(3);
C = Jacobi_const(x_n,mu);
end
```

Example: Differential correction

Ex4_differential_correction.m

```
% initial settings
[mu,~,~,~] = parameter(1); % Sun-Earth
[L1,L2,L3,L4,L5] = librationPoints(mu);

options_ODE = odeset('RelTol',3e-14, 'AbsTol',1e-14);

iteration_max = 100;
threshold = 1e-12;

% for Lyapunov orbit
x0 = [0.9889 0 0 0 0.008375 0]';
t0 = 2;

% for halo orbit
%x0 = [0.9919 0 0.002223 0 -0.01014 0]';
%t0 = pi/2;

if x0(3) == 0
    Lyapunov = true;
else
    Lyapunov = false;
end
```

```
% differential correction
% initial guess
tspan = [0 2*t0];
[t_initial, x_initial] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x0, options_ODE);

f1 = figure;
hold on
f1_pL = plot3(L1(1),L1(2),L1(3),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10);
f1_p1 = plot3(x_initial(:,1), x_initial(:,2),x_initial(:,3),'r');
plot3(x_initial(1,1), x_initial(1,2),x_initial(1,3),'o','MarkerFaceColor','r','MarkerEdgeColor','r','MarkerSize',4);
hold off

% correction
for iteration = 1:iteration_max
    [x_n,t_n,~] = fun_differential_correction_cr3bp(x0,t0,mu,options_ODE);

    tspan = [0 2*t_n];
    [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n, options_ODE);

    x_error = norm(x_corrected(end,:) - x_corrected(1,:));
    disp( strcat('x_error = ',num2str(x_error)) );
    if x_error < threshold
        break;
    end

    if x_error > 1e+3
        disp('calculation diverged');
        return;
    end

    if iteration == iteration_max
        disp('do not finish');
        return;
    end

    x0 = x_n;
    t0 = t_n;
end

figure(f1);
hold on
f1_p2 = plot3(x_corrected(:,1), x_corrected(:,2),x_corrected(:,3),'b');
plot3(x_corrected(1,1), x_corrected(1,2),x_corrected(1,3),'o','MarkerFaceColor','b','MarkerEdgeColor','b','MarkerSize',4);
if Lyapunov == true
    view_angle = [0 90];
else
    view_angle = [40 30];
end
view(gca,view_angle(1),view_angle(2));
axis image
xlabel('$x$[-]');
ylabel('$y$[-]');
zlabel('$z$[-]');
grid on
legend([f1_pL, f1_p1, f1_p2], {'$L_1$', 'initial', 'corrected'});
hold off
f1_name = strcat('Ex4_differential_correction_mu=',num2str(mu),'_x0=',num2str(x0(1)),'_z0=',num2str(x0(3)),'_ydot0=',num2str(x0(5)),'_t0=',num2str(t0));
f1_name = strrep(f1_name,','',' ');
save_fig(f1,f1_name,view_angle);
```

Example: Differential correction

Calculate the monodromy matrix of a periodic orbit after differential correction.

```
% monodromy matrix
X0 = [x_n', reshape(eye(6),1,[])];
tspan = [0,2*t_n];
[~,Y] = ode113(@(t,x) fun_stm_cr3bp(t,x,mu),tspan,X0,options_ODE);

monodromy = reshape(Y(end,7:end),6,6);
[V,D] = eig(monodromy);
disp(diag(D));

theta = linspace(0, 2*pi, 1001)';
x_circle = cos(theta);
y_circle = sin(theta);

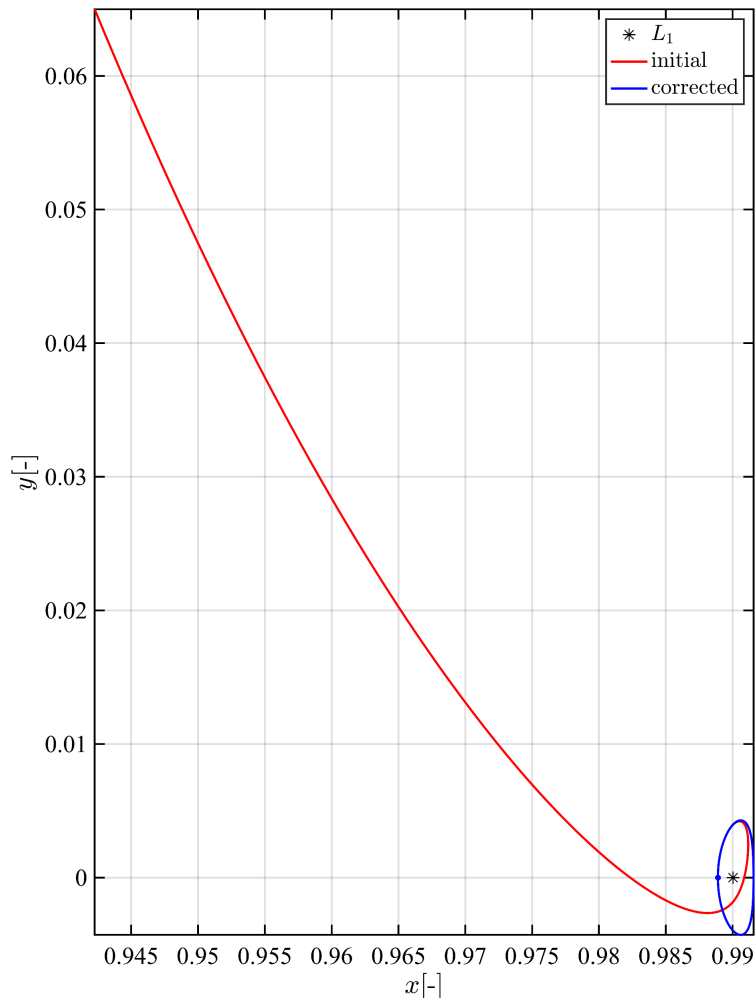
f2 = figure;
hold on
plot(x_circle,y_circle,'k');
f2_p1 = plot(diag(D),'o','MarkerFaceColor','none','MarkerEdgeColor','b','MarkerSize',10);
axis equal
xlabel('real part');
ylabel('imaginary part');
grid on
legend([f2_p1], {'eigenvalue'});
hold off

f2_name = strcat('Ex4_monodromy_mu=',num2str(mu),'_x0=',num2str(x0(1)),'_z0=',num2str(x0(3)),'_ydot0=',num2str(x0(5)),'_t0=',num2str(t0));
f2_name = strrep(f2_name, '.', ',');
save_fig(f2,f2_name,[0 90]);

figure(f2);
hold on
xlim([-1.5 1.5]);
ylim([-1.5 1.5]);
hold off
f3_name = strcat('Ex4_monodromy_origin_mu=',num2str(mu),'_x0=',num2str(x0(1)),'_z0=',num2str(x0(3)),'_ydot0=',num2str(x0(5)),'_t0=',num2str(t0));
f3_name = strrep(f3_name, '.', ',');
save_fig(f2,f3_name,[0 90]);
```

Example: Differential correction

Differential correction for a Lyapunov orbit

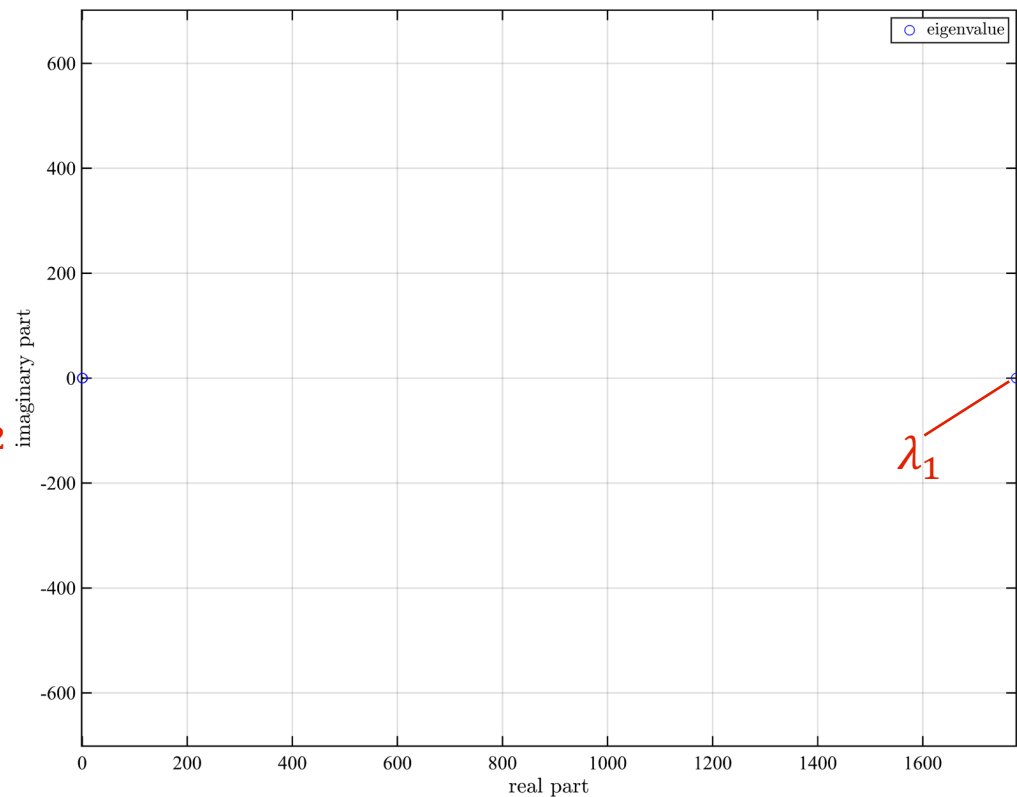
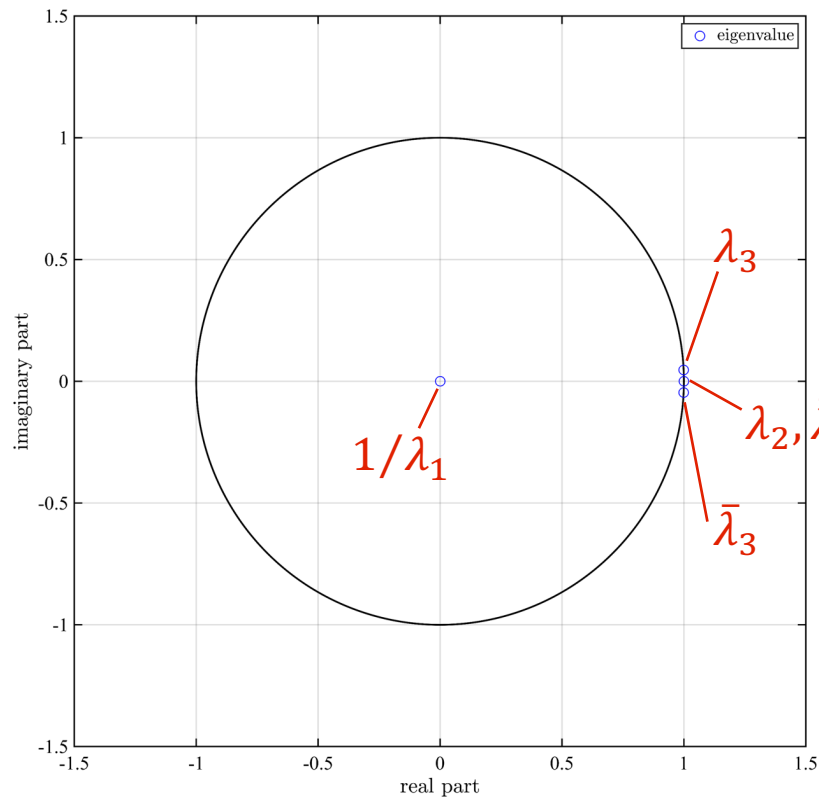


Eigenvalues of the monodromy matrix

$\lambda_1 (= 1/\lambda_2)$	1778.0
$\lambda_2 (= 1/\lambda_1)$	0.0005624
$\lambda_3 (= 1/\lambda_4 \approx 1)$	0.9999 + 0.000001881i
$\lambda_4 (= 1/\lambda_3 \approx 1)$	0.9999 - 0.000001881i
$\lambda_5 (= 1/\lambda_6)$	0.9989 + 0.04631i
$\lambda_6 (= 1/\lambda_5)$	0.9989 - 0.04631i

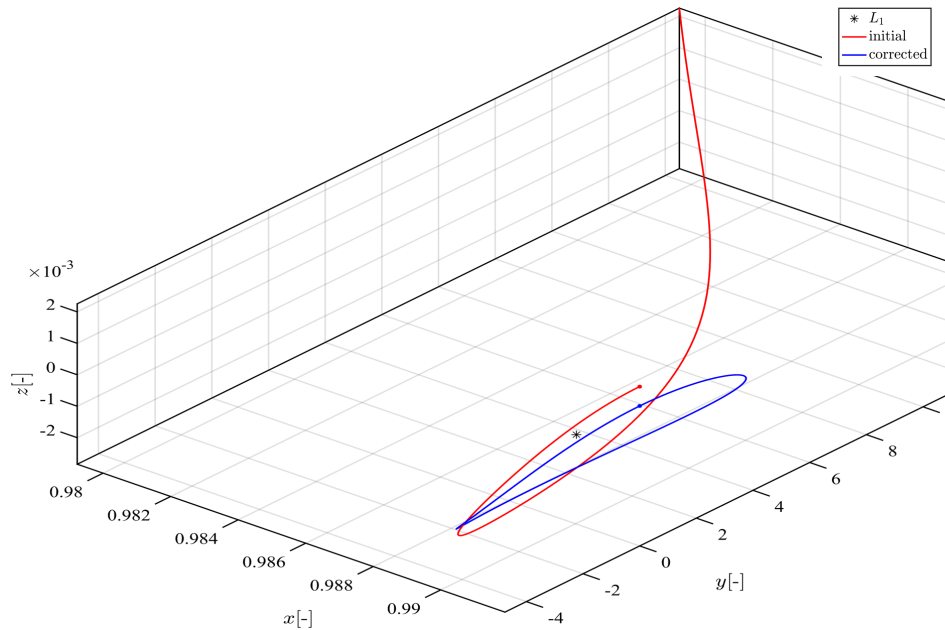
Example: Differential correction

Eigenvalues of the monodromy matrix of a Lyapunov orbit



Example: Differential correction

Differential correction for a halo orbit

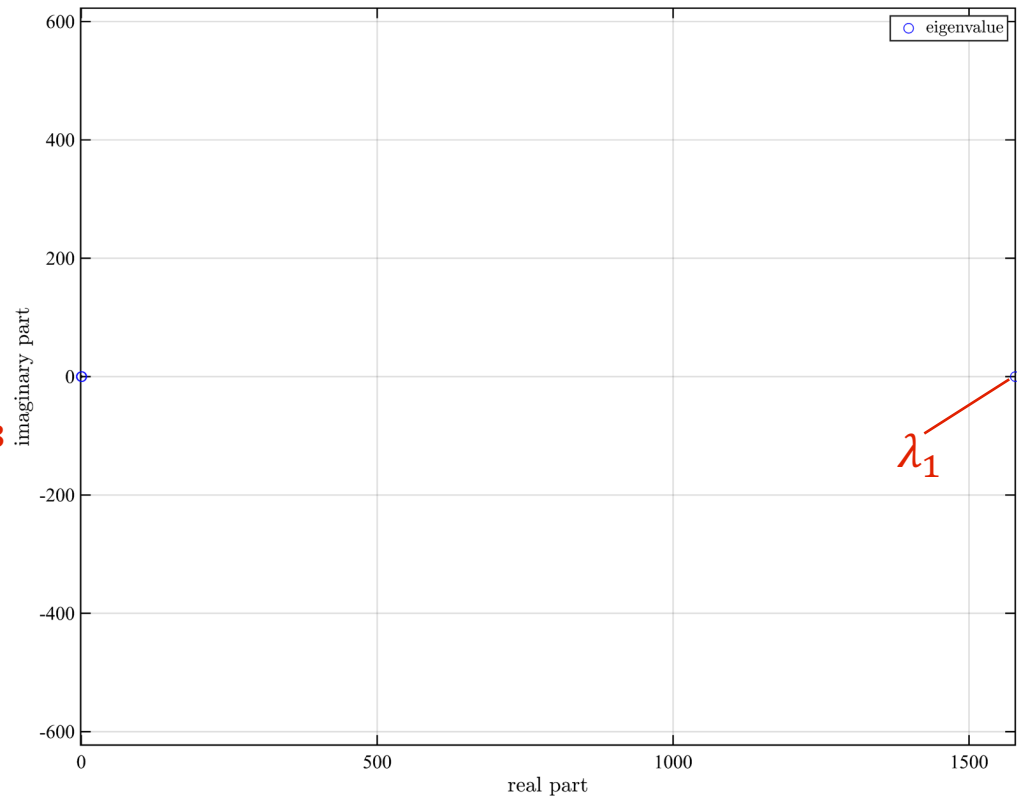
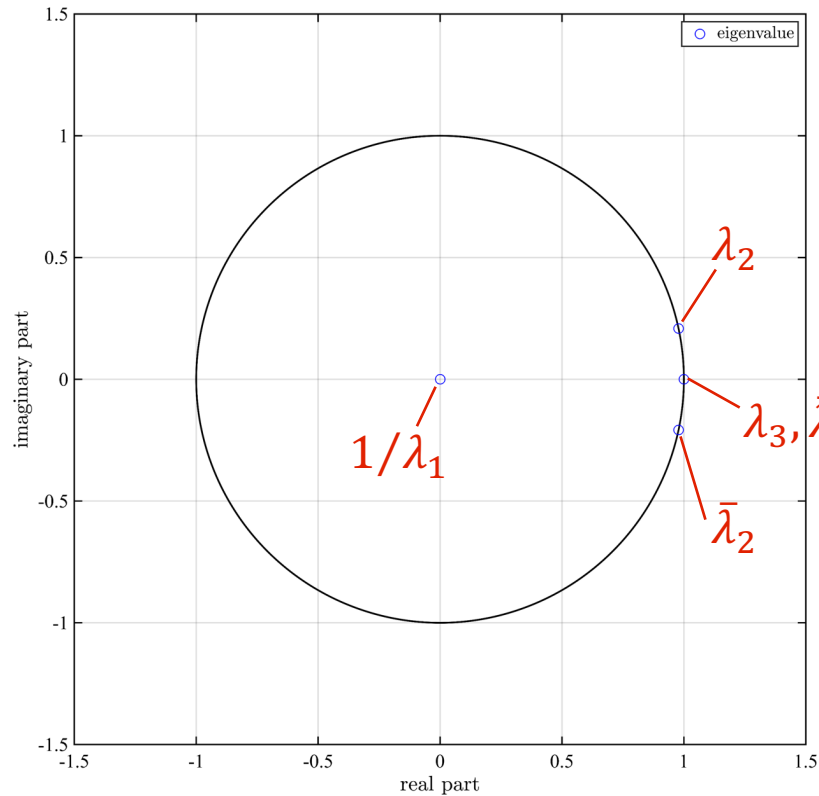


Eigenvalues of the monodromy matrix

$\lambda_1 (= 1/\lambda_2)$	1578.1
$\lambda_2 (= 1/\lambda_1)$	0.0006336
$\lambda_3 (= 1/\lambda_4)$	$0.9780 + 0.2082i$
$\lambda_4 (= 1/\lambda_3)$	$0.9780 - 0.2082i$
$\lambda_5 (= 1/\lambda_6 \approx 1)$	$0.9999 + 0.000002995i$
$\lambda_6 (= 1/\lambda_5 \approx 1)$	$0.9999 - 0.000002995i$

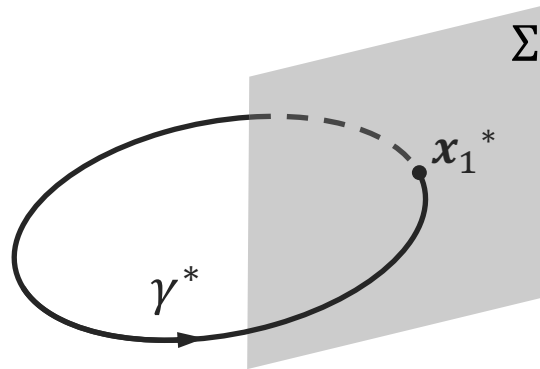
Example: Differential correction

Eigenvalues of the monodromy matrix of a Lyapunov orbit



Invariant manifolds of a periodic orbit

- Invariant manifolds are dynamical structures representing **the local and global flow** associated with equilibrium points as well as periodic orbits.
- If a periodic orbit γ^* has the minimum period T , γ^* may appear as a single **fixed point**, $x(t + nT) = x(t + (n + 1)T) = x_1^*$, on the mapping $P: \Sigma \rightarrow \Sigma$ (Poincaré map).



- Thus, a periodic orbit γ^* is discretized and represented by any one point along the orbit.

Invariant manifolds of a periodic orbit

- The map P is linearized relative to $\mathbf{x}^*$ such that

$$\delta \mathbf{x}(k+1) = \text{DP}(\mathbf{x}_1^*) \delta \mathbf{x}(k)$$

where $k = t + nT$, $k+1 = t + (n+1)T$, $n \in \mathbb{Z}$. $\text{DP}(\mathbf{x}_1^*) = \Phi(t+T, t)$ is the **monodromy matrix** associated with the fixed point $\mathbf{x}_1^*$.

- The eigenvalues $\lambda_{1,i}$ and eigenvectors $\mathbf{v}_{1,i}$ of $\text{DP}(\mathbf{x}_1^*) = \Phi(t+T, t)$ are used to compute stable and unstable manifolds.

$$\left\{ \begin{array}{ll} \text{Stable manifold} & \longleftrightarrow \text{The eigenvector } \mathbf{v}_{1,s} \ (\lambda_{1,s} < 1) \\ \text{Unstable manifold} & \longleftrightarrow \text{The eigenvector } \mathbf{v}_{1,u} \ (\lambda_{1,u} > 1) \end{array} \right.$$

Computation of the invariant manifolds

- It is assumed that $\mathbf{v}_{1,s}$ and $\mathbf{v}_{1,u}$ have a positive x component.
- The magnitude of the eigenvectors $\mathbf{v}_{1,s}$ and $\mathbf{v}_{1,u}$ is adjusted such that,

$$\mathbf{v}_1^{W^S} = \mathbf{v}_{1,s} / \|\mathbf{v}_{1,s}\|, \quad \mathbf{v}_1^{W^U} = \mathbf{v}_{1,u} / \|\mathbf{v}_{1,u}\|$$

Then,

$$\begin{cases} \mathbf{x}_1^{S+} = \mathbf{x}_1^* + d \cdot \mathbf{v}_1^{W^S} \\ \mathbf{x}_1^{S-} = \mathbf{x}_1^* - d \cdot \mathbf{v}_1^{W^S} \end{cases} \quad \begin{cases} \mathbf{x}_1^{U+} = \mathbf{x}_1^* + d \cdot \mathbf{v}_1^{W^U} \\ \mathbf{x}_1^{U-} = \mathbf{x}_1^* - d \cdot \mathbf{v}_1^{W^U} \end{cases}$$

where $d > 0$ is a perturbation parameter.

- Stable manifolds are calculated by propagating the states $\mathbf{x}_1^{S+}$ and $\mathbf{x}_1^{S-}$ backward in time.
- Unstable manifolds are calculated by propagating the states $\mathbf{x}_1^{U+}$ and $\mathbf{x}_1^{U-}$ forward in time.

Computation of the invariant manifolds

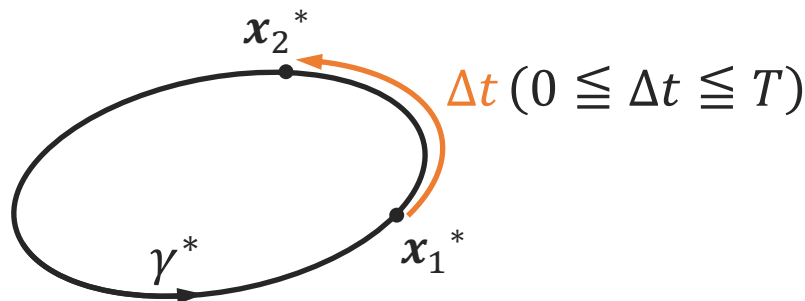
- If x_2^* is another fixed point on a periodic orbit γ^* such that

$$x(t + \Delta t + nT) = x(t + \Delta t + (n + 1)T) = x_2^*,$$

and $v_{2,s}$ and $v_{2,u}$ are the eigenvectors of $DP(x_2^*) = \Phi(t + \Delta t + T, t + \Delta t)$ associated with its stable and unstable eigenvalues,

$v_{1,s}$, $v_{1,u}$, $v_{2,s}$ and $v_{2,u}$ satisfy

$$\begin{cases} \Phi(t + T, t)v_{1,s} = \lambda_{1,s}v_{1,s} \\ \Phi(t + T, t)v_{1,u} = \lambda_{1,u}v_{1,u} \end{cases} \quad \begin{cases} \Phi(t + \Delta t + T, t + \Delta t)v_{2,s} = \lambda_{2,s}v_{2,s} \\ \Phi(t + \Delta t + T, t + \Delta t)v_{2,u} = \lambda_{2,u}v_{2,u} \end{cases}$$



Computation of the invariant manifolds

- Since $\Phi(t + \Delta t + T, t + \Delta t) = \Phi(t + \Delta t + T, t + T)\Phi(t + T, t)\Phi(t, t + \Delta t)$
 $= \Phi(t + \Delta t, t)\Phi(t + T, t)\Phi^{-1}(t + \Delta t, t),$

$$\begin{cases} \Phi(t + T, t)\{\Phi^{-1}(t + \Delta t, t)\mathbf{v}_{2,s}\} = \lambda_{2,s}\{\Phi^{-1}(t + \Delta t, t)\mathbf{v}_{2,s}\} \\ \Phi(t + T, t)\{\Phi^{-1}(t + \Delta t, t)\mathbf{v}_{2,u}\} = \lambda_{2,u}\{\Phi^{-1}(t + \Delta t, t)\mathbf{v}_{2,u}\} \end{cases}$$

- These equations imply the following relationships:

$$\begin{cases} \lambda_{2,s} = \lambda_{1,s} & \begin{cases} \Phi^{-1}(t + \Delta t, t)\mathbf{v}_{2,s} = \mathbf{v}_{1,s} \\ \Phi^{-1}(t + \Delta t, t)\mathbf{v}_{2,u} = \mathbf{v}_{1,u} \end{cases} \\ \lambda_{2,u} = \lambda_{1,u} \end{cases}$$

because $DP(\mathbf{x}_1^*) = \Phi(t + T, t)$ has only one stable eigenvalue and one unstable eigenvalue. Therefore,

$$\begin{cases} \mathbf{v}_{2,s} = \Phi(t + \Delta t, t)\mathbf{v}_{1,s} \\ \mathbf{v}_{2,u} = \Phi(t + \Delta t, t)\mathbf{v}_{1,u} \end{cases}$$

- These relationships are useful to compute stable and unstable manifolds of N points on a periodic orbit.

Sample Code: Stable and unstable manifolds

fun_manifold_cr3bp.m

```
% calculate initial conditions for stable and unstable manifolds of a periodic orbit
function [XS_left, XS_right, XU_left, XU_right, Y] = fun_manifold_cr3bp(mu, x0, t0, N, xpert, options)
% mu      : mass ratio of the primaries
% x0      : initial state of a periodic orbit
% t0      : the period of a periodic orbit
% N       : the number of points on a periodic orbit
% xpert   : weight for a position of the eigenvector
% options : options for ode

% calculate eigenvalues of the monodromy matrix
% Propagate orbit for one period and gather data at N points
tspan = linspace(t0, t0 + N, N);
X0 = [x0; reshape(eye(6), 36, 1)];
[~, Y] = ode113(@(t,x) fun_stm_cr3bp(t,x,mu), tspan, X0, options);

% Monodromy matrix
M = reshape(Y(end, 7:42), 6, 6);

% Eigenvector and eigenvalue analysis
[vec, val] = eig(M);
val = diag(val);

[~, index_s] = min(abs(val)); % stable
[~, index_u] = max(abs(val)); % unstable

if (imag(val(index_s)) ~= 0) || (imag(val(index_u)) ~= 0)
    error('Imag eigenvalues are dominant');
end

% Stable and unstable eigenvectors
vector_stable = vec(:, index_s);
if vector_stable(1) < 0
    vector_stable = - vector_stable;
end
vector_unstable = vec(:, index_u);
if vector_unstable(1) < 0
    vector_unstable = - vector_unstable;
end
```

```
% calculate manifolds
XS_left = zeros(6, N);
XS_right = zeros(6, N);
XU_left = zeros(6, N);
XU_right = zeros(6, N);

% Apply perturbations to each of N points of a periodic orbit
for iteration = 1:N
    % Grab state transition matrix at the fixed point
    Phi = reshape(Y(iteration, 7:42), 6, 6);

    % Grab state at the fixed point
    x_star = Y(iteration, 1:6)';

    % Map stable and unstable vectors forward
    S = Phi*vector_stable;
    S = S/norm(S);
    U = Phi*vector_unstable;
    U = U/norm(U);

    % Create perturbation vector
    vpert = xpert*norm(x_star(4:6))/norm(x_star(1:3));
    pert = [ones(3,1).*xpert; ones(3,1).*vpert];

    % Perturb conditions
    XS_left(:, iteration) = x_star - S.*pert;
    XS_right(:, iteration) = x_star + S.*pert;

    XU_left(:, iteration) = x_star - U.*pert;
    XU_right(:, iteration) = x_star + U.*pert;
end
end
```

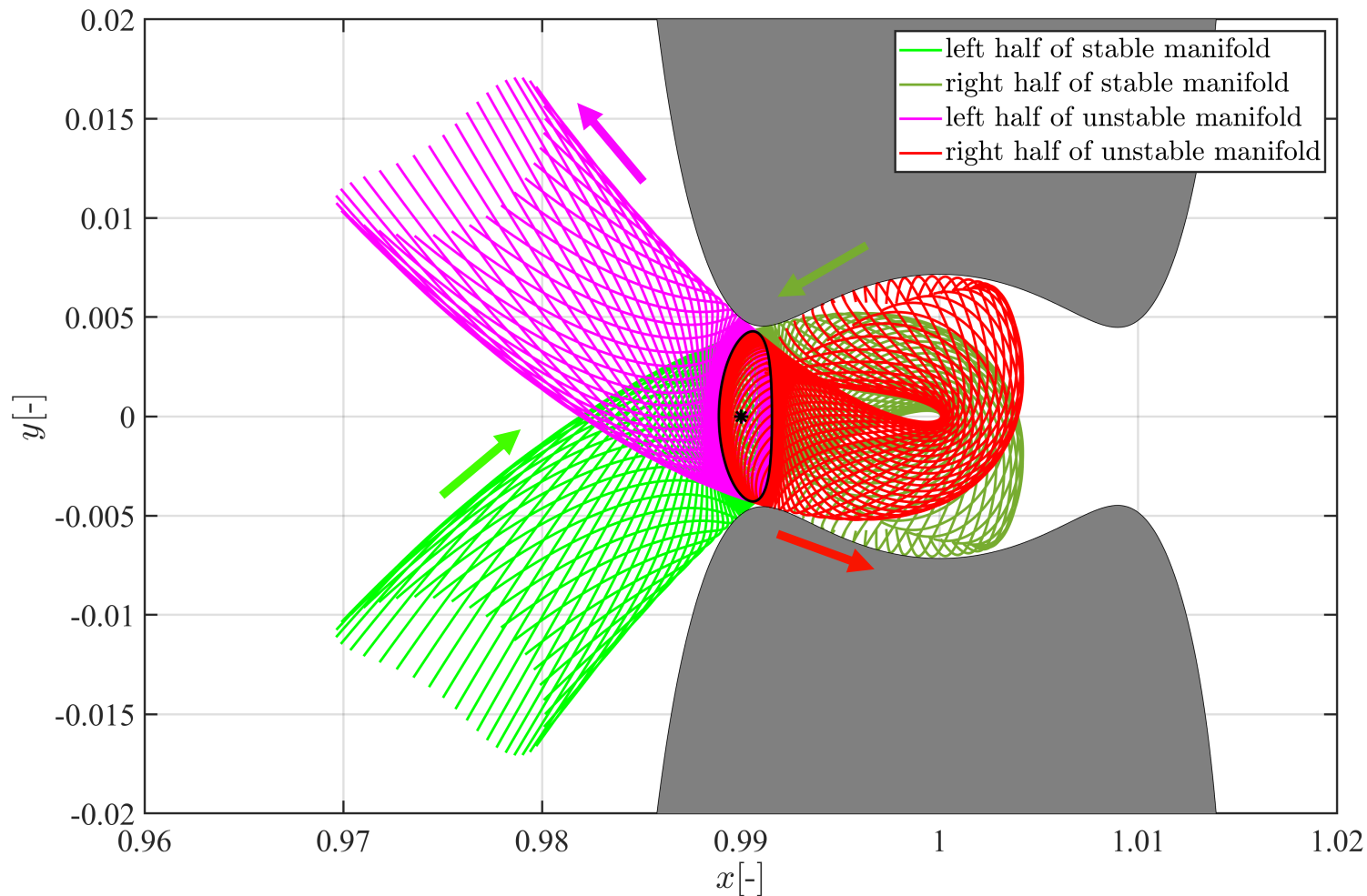
Example: Stable and unstable manifolds of a Lyapunov orbit

Ex5_manifold.m

<pre>% Initial settings [mu,~,~,~] = parameter(1); % Sun-Earth [L1,L2,L3,L4,L5] = librationPoints(mu); options_ODE = odeset('RelTol',3e-14, 'AbsTol',1e-14); iteration_max = 100; threshold = 1e-12; % for Lyapunov orbit x0 = [0.9889 0 0 0 0.008375 0]'; t0 = 2; if x0(3) == 0 Lyapunov = true; else Lyapunov = false; end % differential correction % initial guess tspan1 = [0 2*t0]; [t_initial, x_initial] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan1, x0, options_ODE); % correction for iteration = 1:iteration_max [x_n,t_n,C_xn] = fun_differential_correction_cr3bp(x0,t0,mu,options_ODE); tspan = [0 2*t_n]; [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n, options_ODE); x_error = norm(x_corrected(end,:) - x_corrected(1,:)); %disp('x_error = ', num2str(x_error)); if x_error < threshold break; end if x_error > 1e+3 disp('calculation diverged'); return; end if iteration == iteration_max disp('do not finish'); return; end x0 = x_n; t0 = t_n; end % zero velocity curve [x_curve,y_curve] = meshgrid(0.9:1e-4:1.1,-0.1:1e-4:0.1); [a,b] = size(x_curve); z_curve = zeros(a,b); x_curve = reshape(x_curve,[a,b,1]); y_curve = reshape(y_curve,[a,b,1]); z_curve = reshape(z_curve,[a,b,1]); r1 = sqrt((x_curve+mu).^2+y_curve.^2+z_curve.^2); r2 = sqrt((x_curve-1-mu).^2+y_curve.^2+z_curve.^2); U = 1/2.*(x_curve.^2+y_curve.^2) + (1-mu)./r1 + mu./r2; C = 2.*U; x_curve = reshape(x_curve,[a,b]); y_curve = reshape(y_curve,[a,b]); C = reshape(C,[a,b]);</pre>	<pre>% stable and unstable manifolds xpert = 1e-6; N = 40; [XS_left, XS_right, XU_left, XU_right, Y] = fun_manifold_cr3bp(mu, x_n, 2*t_n, N, xpert, options_ODE); tf = 4.8; tspan_s = [tf 0]; tspan_u = [0 tf]; f1 = figure; hold on contourf(x_curve, y_curve, -C, [-C_xn -C_xn], 'k'); colormap gray for i = 1:N [~,ys_left] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, XS_left(:,i), options_ODE); ys_left = flipud(ys_left); f1_p1 = plot3(ys_left(:,1),ys_left(:,2),ys_left(:,3),'g'); end for i = 1:N [~,ys_right] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, XS_right(:,i), options_ODE); ys_right = flipud(ys_right); f1_p2 = plot3(ys_right(:,1),ys_right(:,2),ys_right(:,3),'Color','#77AC30'); end for i = 1:N [~,yu_left] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, XU_left(:,i), options_ODE); f1_p3 = plot3(yu_left(:,1),yu_left(:,2),yu_left(:,3),'m'); end for i = 1:N [~,yu_right] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, XU_right(:,i), options_ODE); f1_p4 = plot3(yu_right(:,1),yu_right(:,2),yu_right(:,3),'r'); end plot3(x_corrected(:,1), x_corrected(:,2),x_corrected(:,3),'k'); plot3(L1(1),L1(2),L1(3),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',10); if Lyapunov == true view_angle = [0 90]; else view_angle = [40 30]; end view(gca,view_angle(1),view_angle(2)); axis image xlim([0.96 1.02]); ylim([-0.02 0.02]); xlabel('x[s]-1'); ylabel('y[s]-1'); zlabel('z[s]-1'); grid on legend([f1_p1, f1_p2, f1_p3, f1_p4], {'left half of stable manifold','right half of stable manifold','left half of unstable manifold','right half of unstable manifold'}); hold off f1_name = strcat('Ex5_manifold_mu=',num2str(mu),'_xm=',num2str(x_n(1)),'_zn=',num2str(x_n(3)),'_ydotn=',num2str(x_n(5)),'_t_n=',num2str(t_n),'_xpert=',num2str(xpert),'_tf=',num2str(tf)); f1_name = strrep(f1_name,' ','_'); save_fig(f1,f1_name,view_angle); if Lyapunov == false legend('off'); f1_name_xy = strcat(f1_name,'_xy'); save_fig(f1,f1_name_xy,[0 90]); f1_name_xz = strcat(f1_name,'_xz'); save_fig(f1,f1_name_xz,[0 0]); f1_name_yz = strcat(f1_name,'_yz'); save_fig(f1,f1_name_yz,[90 0]); end % End of script time = strcat('calculation time: ', num2str(toc(myTimer))); disp(time);</pre>
--	--

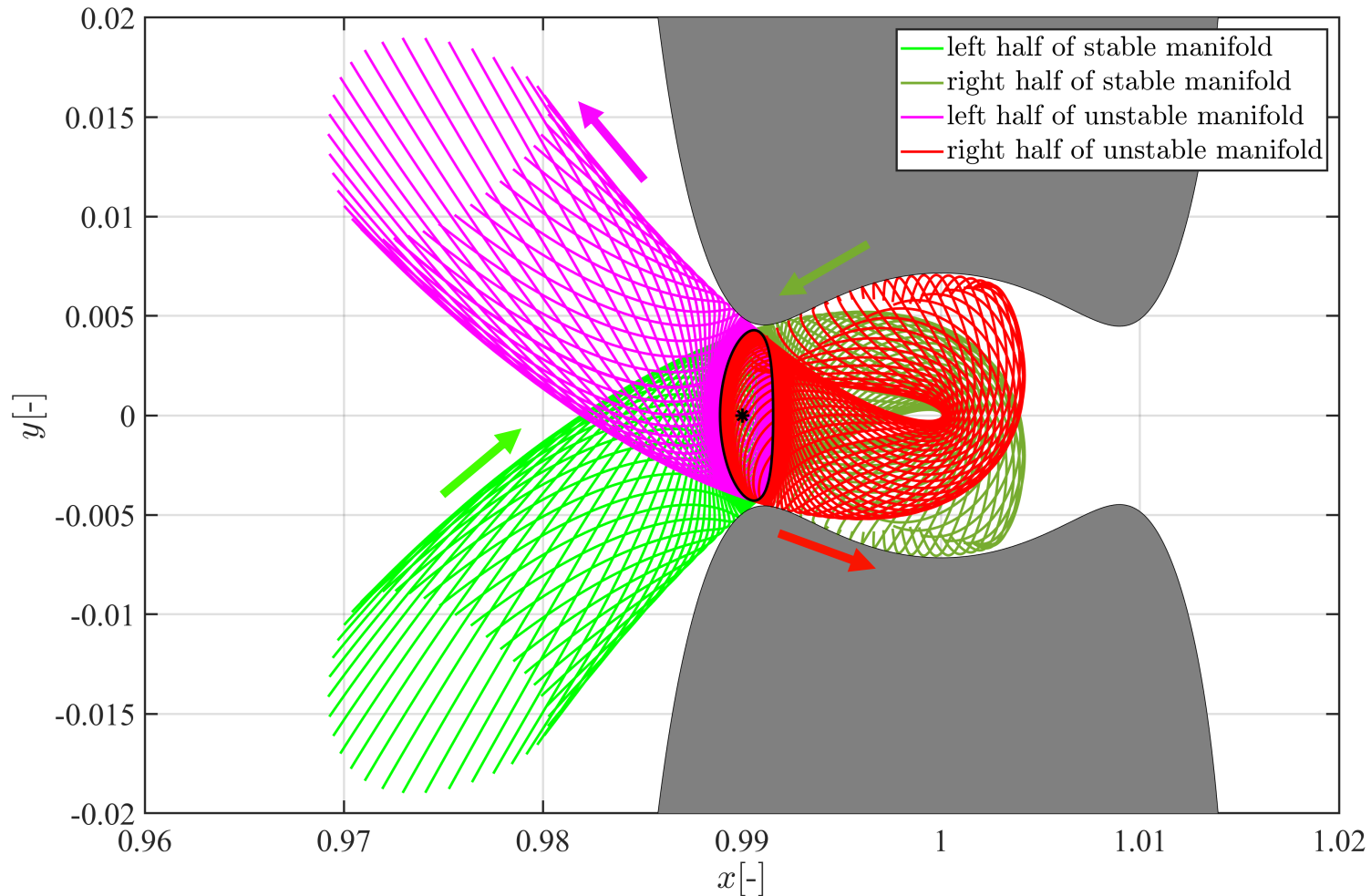
Example: Stable and unstable manifolds of a Lyapunov orbit

$x_{\text{pert}} = 1\text{e-}6$, $t_f = 4.8$



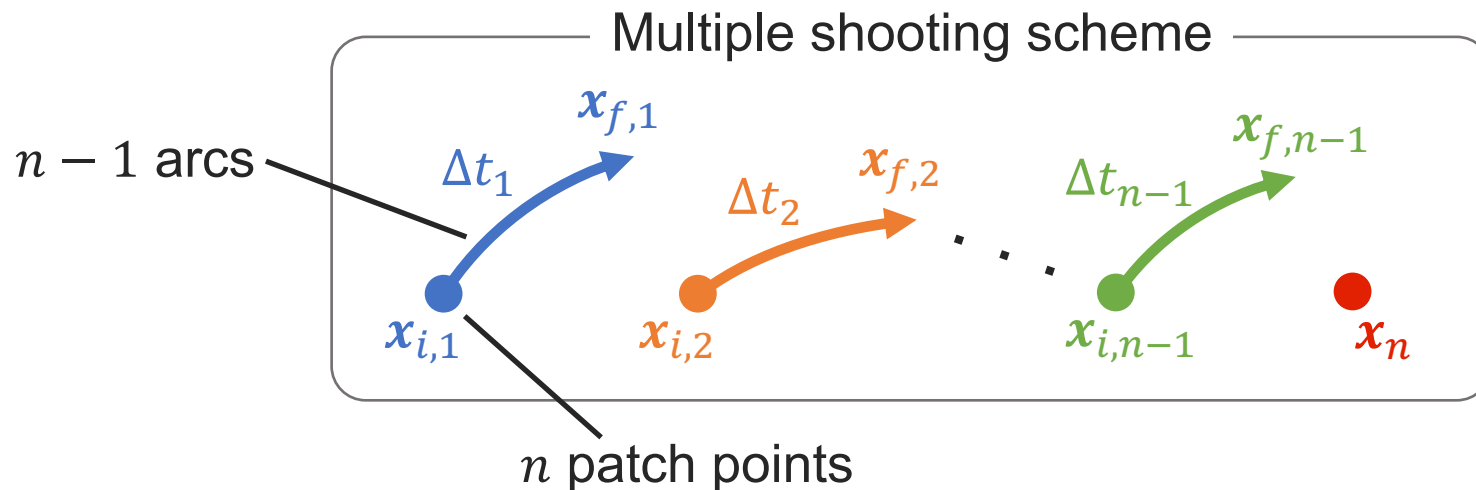
Example: Stable and unstable manifolds of a Lyapunov orbit

$x_{\text{pert}} = 1\text{e-}10$, $t_f = 8.6$



The concept of multiple shooting

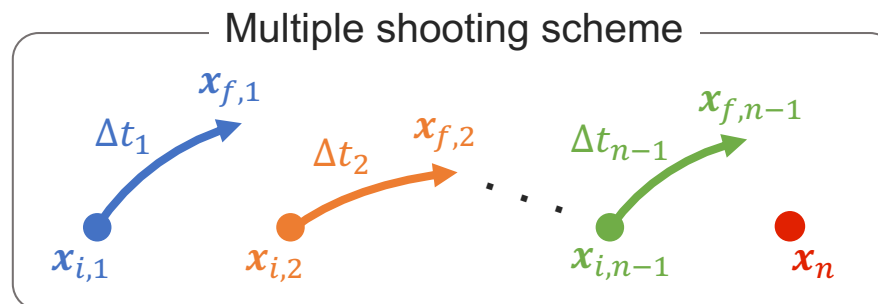
- A powerful extension of the single shooting (differential correction) scheme
- Adjust multiple distinct trajectory arcs **simultaneously**, forcing continuity between each arc
- Especially beneficial for trajectories with close passes near the primaries, a dynamically sensitive event



The calculation scheme of multiple shooting

- The design variable vector X and objective function $F(X)$ is expressed as

$$X = \begin{bmatrix} x_{i,1} \\ x_{i,2} \\ \vdots \\ x_{i,n-1} \\ x_{i,n} \\ \Delta t_1 \\ \Delta t_2 \\ \vdots \\ \Delta t_{n-1} \end{bmatrix}, \quad F(X) = \begin{bmatrix} x_{f,1} - x_{i,2} \\ x_{f,2} - x_{i,3} \\ \vdots \\ x_{f,n-1} - x_n \end{bmatrix}$$



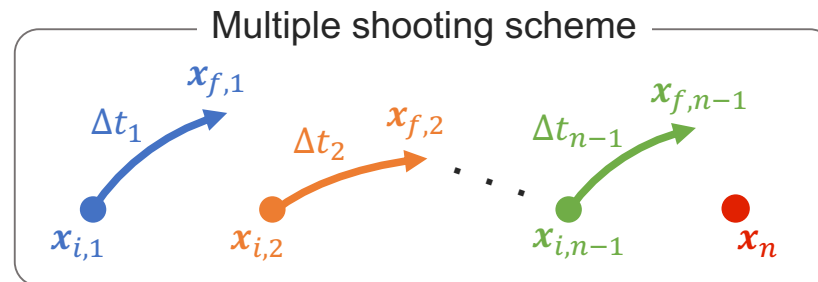
The calculation scheme of multiple shooting

- The calculation process is the same to that of [differential correction](#). However, the calculation of $\mathbf{DF}(X)$ is worth noting.
- $\mathbf{DF}(X)$ is expressed as bellow in the multiple shooting scheme.

$$X = \begin{bmatrix} \mathbf{x}_{i,1}^T & \mathbf{x}_{i,2}^T & \cdots & \mathbf{x}_{i,n-1}^T & \Delta t_1 & \Delta t_2 & \cdots & \Delta t_{n-1} \end{bmatrix}^T$$

$$\mathbf{DF}(X) = \begin{bmatrix} \Phi_1 & -I_{6 \times 6} & 0_{6 \times 6} & \cdots & 0_{6 \times 6} & \dot{\mathbf{x}}_{f,1} & \mathbf{0}_{6 \times 1} & \cdots & \mathbf{0}_{6 \times 1} \\ 0_{6 \times 6} & \Phi_2 & -I_{6 \times 6} & \cdots & \vdots & \mathbf{0}_{6 \times 1} & \dot{\mathbf{x}}_{f,2} & \cdots & \vdots \\ \vdots & \cdots & \cdots & \cdots & 0_{6 \times 6} & \vdots & \cdots & \cdots & \mathbf{0}_{6 \times 1} \\ 0_{6 \times 6} & \cdots & 0_{6 \times 6} & \Phi_{n-1} & -I_{6 \times 6} & \mathbf{0}_{6 \times 1} & \cdots & \mathbf{0}_{6 \times 1} & \dot{\mathbf{x}}_{f,n-1} \end{bmatrix}$$

where $\Phi_k = \frac{\partial \mathbf{x}_{f,k}}{\partial \mathbf{x}_{i,k}}$ is denoted as the state transition matrices at the end points of the k-th arc.



The calculation scheme of multiple shooting

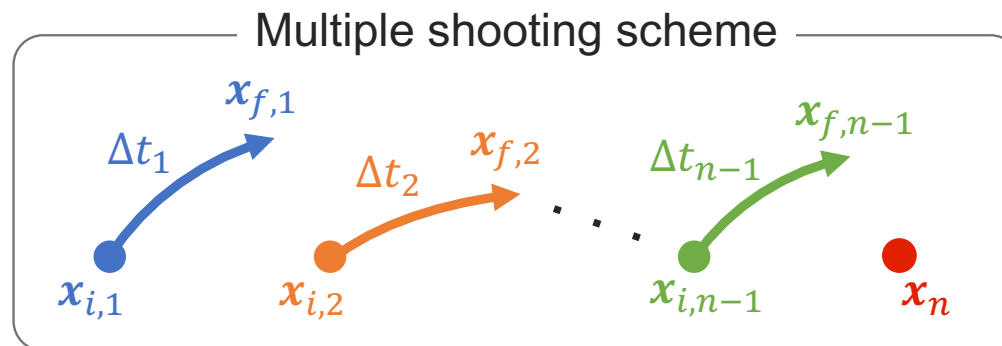
- Based on the Newton method, the optimal solution is obtained by the iteration of i .

$$X_{i+1} = X_i - \mathbf{D}\mathbf{F}^{-1}(X_i)\mathbf{F}(X_i) \quad (m = n)$$

$$X_{i+1} = X_i - [\mathbf{D}\mathbf{F}^T(X_i)\mathbf{D}\mathbf{F}(X_i)]^{-1}\mathbf{D}\mathbf{F}^T(X_i)\mathbf{F}(X_i) \quad (m > n)$$

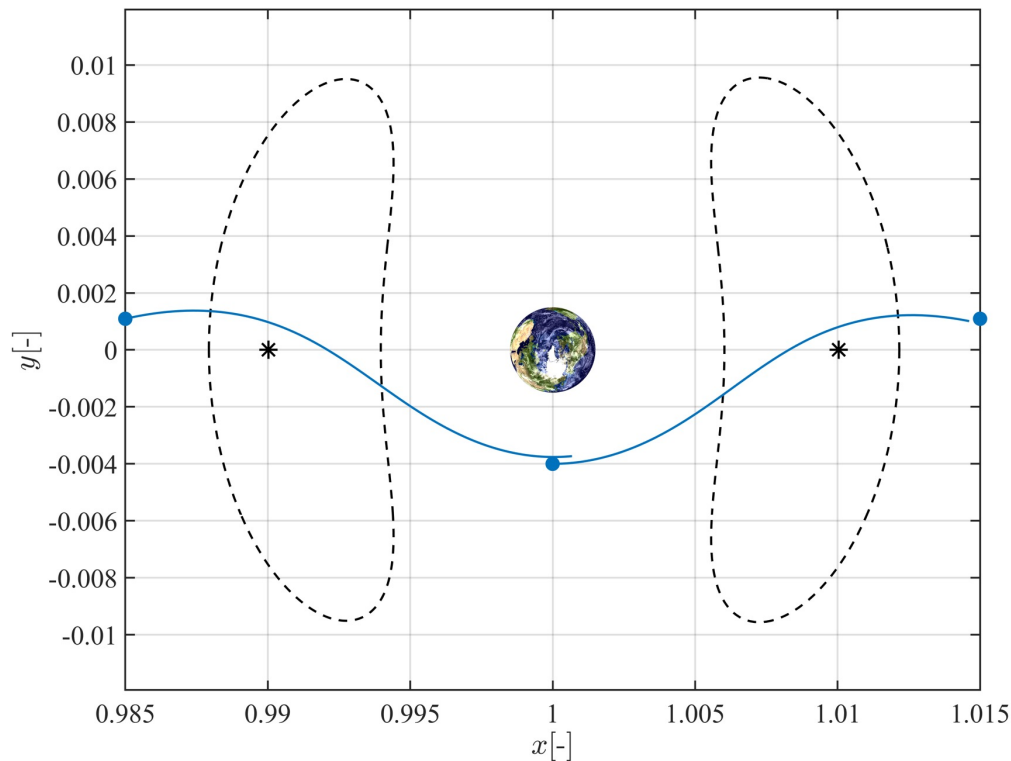
$$X_{i+1} = X_i - \mathbf{D}\mathbf{F}^T(X_i)[\mathbf{D}\mathbf{F}(X_i)\mathbf{D}\mathbf{F}^T(X_i)]^{-1}\mathbf{F}(X_i) \quad (m < n)$$

- The iteration ends when $\|F(X_i)\| \ll 1$.



Example 1: Multiple shooting

- Design a transfer trajectory in the Sun-Earth CR3BP.
- Initial and final positions are fixed through the calculation.



Initial guess for multiple shooting

Example 1: Multiple shooting

fun_multiple_shooting_cr3bp.m

```
% multiple shooting
function X_n = fun_multiple_shooting_cr3bp(X0,n,mu,options)
% X0      : initial guess for the design variable vector X
% n       : the number of the patch points
% mu      : mass ratio of the primaries
% options : options for ode

Xf = zeros(6*(n-1),1);
Phi = zeros(6,6,n-1);
for i = 1:(n-1)
    Y0 = [X0((6*i-5):6*i); reshape(eye(6),[],1)];
    tspan = [0 X0(6*n+i)];
    [~,Y] = ode113(@(t,x) fun_stm_cr3bp(t,x,mu),tspan,Y0,options);

    Xf((6*i-5):6*i) = Y(end,1:6)';
    Phi(:, :, i) = reshape(Y(end,7:end),6,6);
end

F = zeros(6*(n-1),1);
for i = 1:(n-1)
    F((6*i-5):6*i) = Xf((6*i-5):6*i) - X0((6*i+1):6*(i+1));
end

DF = zeros(6*(n-1),7*n-1);
I = eye(6);
for i = 1:(n-1)
    DF((6*i-5):6*i, (6*i-5):6*i) = Phi(:, :, i);
    DF((6*i-5):6*i, (6*i+1):6*(i+1)) = -I;

    DF((6*i-5):6*i, 6*n+i) = fun_cr3bp(X0(6*n+i),Xf((6*i-5):6*i),mu);
end
X_ast = [X0(4:6*(n-1)); X0((6*n-2):end)] - [DF(:,4:6*(n-1)) DF(:,(6*n-2):end)]\F;
X_n = [X0(1:3); X_ast(1:(6*n-9)); X0((6*n-5):(6*n-3)); X_ast((6*n-8):end)];
end
```

Example 1: Multiple shooting

EX6_multiple_shooting.m

```
% Initial settings
[mu,~,~,~] = parameter(1); % Sun-Earth
[L1,L2,L3,L4,L5] = librationPoints(mu);

options_ODE = odeset('RelTol',3e-14, 'AbsTol',1e-14);

options_planet3D.Position = [1-mu, 0, 0]';
options_planet3D.RefPlane = 'ecliptic';
options_planet3D.RotAngle = 135;
options_planet3D.Units = 6378.1363e3*1000/1.5;

%% differential correction
iteration_DC_max = 100;
threshold = 1e-12;

% L1 Lyapunov orbit
x0_1 = [0.987933580858119 0 0 0 0.018052674638026 0]';
t0_1 = 3.290337439058733/2;
for iteration = 1:iteration_DC_max
    [x_n1,t_n1,C_L1] = fun_differential_correction_cr3bp(x0_1,t0_1,mu,options_ODE);

    tspan = [0 2*t_n1];
    [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n1, options_ODE);

    x_error = norm(x_corrected(end,:) - x_corrected(1,:));
    %disp( strcat('x_error = ',num2str(x_error)) );
    if x_error < threshold
        break;
    end

    if x_error > 1e+3
        disp('calculation diverged');
        return;
    end

    if iteration == iteration_DC_max
        disp('do not finish');
        return;
    end

    x0_1 = x_n1;
    t0_1 = t_n1;
end
x_L1 = x_corrected; % C_L1 = 3.000599995882406
t_L1 = t_n1;

% L2 Lyapunov orbit
x0_2 = [1.012164676114102 0 0 0 -0.017947125645568 0]';
t0_2 = 3.319976286115385/2;
for iteration = 1:iteration_DC_max
    [x_n2,t_n2,C_L2] = fun_differential_correction_cr3bp(x0_2,t0_2,mu,options_ODE);

    tspan = [0 2*t_n2];
    [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n2, options_ODE);
```

```
x_error = norm(x_corrected(end,:) - x_corrected(1,:));
%disp( strcat('x_error = ',num2str(x_error)) );
if x_error < threshold
    break;
end

if x_error > 1e+3
    disp('calculation diverged');
    return;
end

if iteration == iteration_DC_max
    disp('do not finish');
    return;
end

x0_2 = x_n2;
t0_2 = t_n2;
end
x_L2 = x_corrected; % C_L2 = 3.000600000000000
t_L2 = t_n2;

%% initial trajectory
x1 = [0.985000000000000 0.001092112291766 0 0.022 0.005 0]';
x2 = [1-mu -4e-3 0 0.03 0 0]';
x3 = [1.015000000000000 0.001092400176687 0 0.020764610600732 -0.004967672676993 0]';
delta_t1 = 0.75;
delta_t2 = 0.75;

tspan1 = [0 delta_t1];
[~,x_traj_1] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan1,x1,options_ODE);

tspan2 = [0 delta_t2];
[~,x_traj_2] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan2,x2,options_ODE);

f1 = figure();
hold on
planet3D('Earth Cloudy',options_planet3D);
plot(x_L1(:,1),x_L1(:,2),'-k');
plot(x_L2(:,1),x_L2(:,2),'-k');
plot(x_traj_1(:,1),x_traj_1(:,2),'Color','0072BD');
plot(x_traj_2(:,1),x_traj_2(:,2),'Color','0072BD');
plot(x1(1),x1(2),'o','MarkerFaceColor','0072BD','MarkerEdgeColor','0072BD','MarkerSize',10);
plot(x2(1),x2(2),'o','MarkerFaceColor','0072BD','MarkerEdgeColor','0072BD','MarkerSize',10);
plot(x3(1),x3(2),'o','MarkerFaceColor','0072BD','MarkerEdgeColor','0072BD','MarkerSize',10);
plot(L1(1),L1(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
plot(L2(1),L2(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
view(gca,0,90);
axis equal
xlabel('$x$[-]');
ylabel('$y$[-]');
xlim([0.985 1.015]);
grid on
hold off
f1_name = strcat('Ex6_initial_guess_mu=',num2str(mu));
f1_name = strrep(f1_name,','.','');
save_fig(f1,f1_name,[0 90]);

f2 = figure();
hold on
planet3D('Earth Cloudy',options_planet3D);
plot(x_L1(:,1),x_L1(:,2),'-k');
plot(x_L2(:,1),x_L2(:,2),'-k');
f2_p1 = plot(x_traj_1(:,1),x_traj_1(:,2),'Color','0072BD');
plot(x_traj_2(:,1),x_traj_2(:,2),'Color','0072BD');
plot(L1(1),L1(2),'o','MarkerFaceColor','0072BD','MarkerEdgeColor','0072BD','MarkerSize',10);
plot(L2(1),L2(2),'o','MarkerFaceColor','0072BD','MarkerEdgeColor','0072BD','MarkerSize',10);
plot(x3(1),x3(2),'o','MarkerFaceColor','0072BD','MarkerEdgeColor','0072BD','MarkerSize',10);
plot(L1(1),L1(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
plot(L2(1),L2(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
hold off
```

Example 1: Multiple shooting

EX6_multiple_shooting.m (continued)

```
% multiple shooting
iteration_MS_max = 100;
X0 = [x1; x2; x3; delta_t1; delta_t2];
n = 3;

for iteration = 1:iteration_MS_max
    X_n = fun_multiple_shooting_cr3bp(X0,n,mu,options_ODE);

    Xf = zeros(6*(n-1),1);
    Phi = zeros(6,6*(n-1));
    for i = 1:(n-1)
        Y0 = [X_n((6*i-5):6*i); reshape(eye(6),[],1)];
        tspan = [0 X_n(6*(n+1))];
        [~,Y] = ode113(@(t,x) fun_stm_cr3bp(t,x,mu),tspan,Y0,options_ODE);

        Xf((6*i-5):6*i) = Y(end,1:6);
        Phi(i,:,:) = reshape(Y(end,7:end),6,6);
    end

    F = zeros(6*(n-1),1);
    for i = 1:(n-1)
        F((6*i-5):6*i) = Xf((6*i-5):6*i) - X0((6*i+1):6*(i+1));
    end

    X_error = norm(F);
    disp(strcat('X_error = ',num2str(X_error)) );
    if X_error < threshold
        break;
    end

    if X_error > 1e+3
        disp('calculation diverged in multiple shooting');
        return;
    end

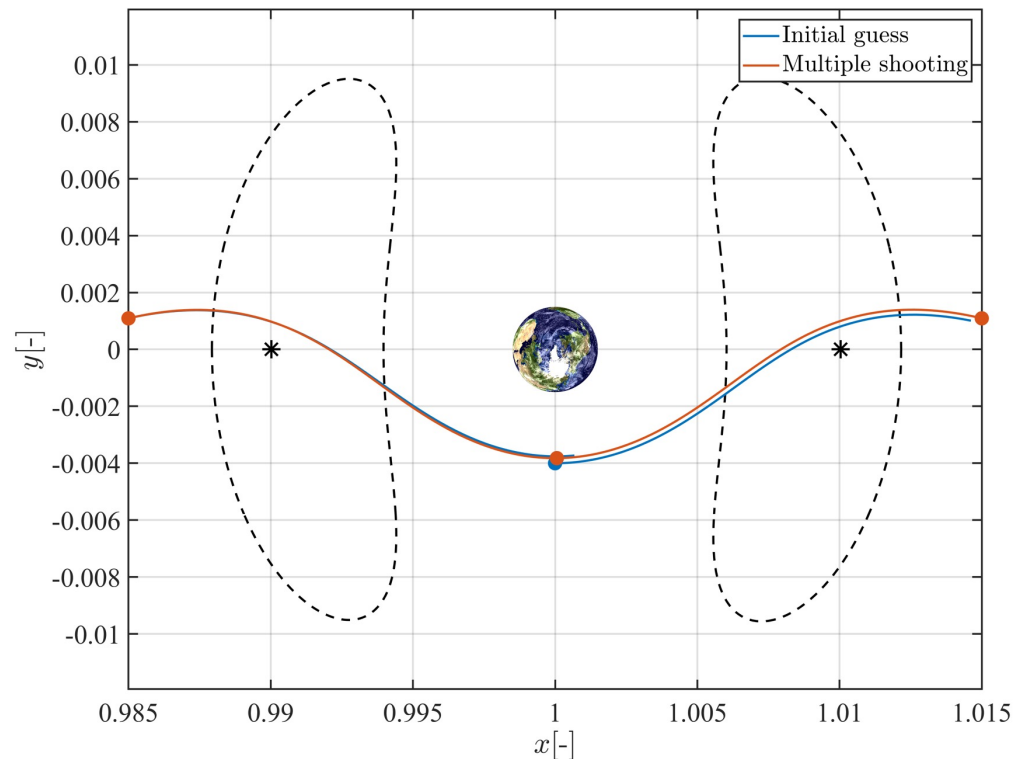
    if iteration == iteration_MS_max
        disp('do not finish in multiple shooting');
        return;
    end

    X0 = X_n;
end
save('Ex6_solution.mat','X_n','X_error');

figure(f2);
hold on
for i = 1:(n-1)
    tspan = [0 X_n(6*(n+1))];
    [~,X] = ode113(@(t,x) fun_cr3bp(t,x,mu),tspan,X_n((6*i-5):6*i),options_ODE);
    f2_p2 = plot(X(:,1),X(:,2),'Color','#D95319');
    plot(X(1,1),X(1,2),'o','MarkerFaceColor','#D95319','MarkerEdgeColor','#D95319','MarkerSize',10);
    plot(X(end,1),X(end,2),'o','MarkerFaceColor','#D95319','MarkerEdgeColor','#D95319','MarkerSize',10);
end
view(gca,0,90);
axis equal
xlabel('$x[-]$');
ylabel('$y[-]$');
grid on
legend([f2_p1, f2_p2], {'Initial guess','Multiple shooting'});
hold off
f2_name = strcat('Ex6_multiple_shooting_mu=',num2str(mu));
f2_name = strrep(f2_name,' ','');
save_fig(f2,f2_name,[0 90]);

% End of script
time = strcat('calculation time: ', num2str(toc(myTimer)));
disp(time);
```

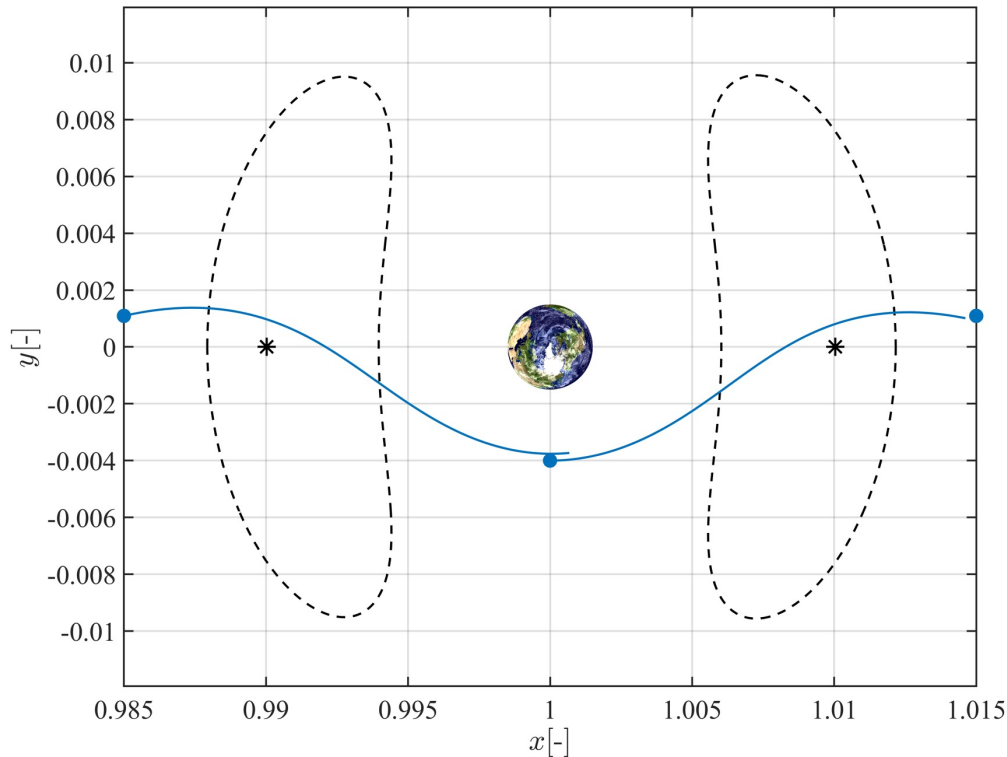
- In the end, $\|F\|$ becomes $5.1419\text{e-}14$.



Result of multiple shooting

Example 2: Multiple shooting by fmincon

- The initial guess is **the same** to that of the previous example.
- Initial and final positions are fixed through the calculation.
- Time of flight (ToF) is constrained in this example, but the converged solution in the previous example satisfies this condition.



Initial guess for multiple shooting

Example 2: Multiple shooting by fmincon

fun_MulShoot_const_CR3BP.m

```
% Constraint
function [c, ceq] = fun_MulShoot_const_CR3BP(X, r0, rf, n, ToF, mu, options)
% X      : design variable vector
% r0     : position of the initial point
% rf     : position of the final point
% n      : the number of the patch points
% ToF    : Time of Flight from the initial to final points
% mu     : mass ratio of the primaries
% options : options for ode

if(size(r0,2) > 1), r0 = r0'; end
if(size(rf,2) > 1), rf = rf'; end

% Equality constraint
ceq = [sum(X((end-n+2):end)) - ToF];

% Inequality constraint
c = [];
end
```

Example 2: Multiple shooting by fmincon

fun_MulShoot_objective_CR3BP.m

```
function J = fun_MulShoot_objective_CR3BP(X, r0, rf, n, mu,options)
% X      : design variable vector
% r0     : position of the initial point
% rf     : position of the final point
% n      : the number of the patch points
% mu     : mass ratio of the primaries
% options : options for ode

if(size(r0,2) > 1), r0 = r0'; end
if(size(rf,2) > 1), rf = rf'; end

X0 = [r0; X(1:(6*n-9)); rf; X((6*n-8):end)];
Xf = zeros(6*(n-1),1);
for i = 1:(n-1)
    tspan = [0 X0(6*n+i)];
    [~,Y] = ode113(@(t,x) fun_cr3bp(t,x,mu),tspan,X0((6*i-5):6*i),options);

    Xf((6*i-5):6*i) = Y(end,1:6)';
end

F = zeros(6*(n-1),1);
for i = 1:(n-1)
    F((6*i-5):6*i) = Xf((6*i-5):6*i) - X0((6*i+1):6*(i+1));
end

J = norm(F);
end
```

Example 2: Multiple shooting by fmincon

Ex7_multiple_shooting_fmincon.m

```

%% initial settings
[mu,~,~] = parameter(1); % Sun-Earth
[L1,L2,L3,L4,L5] = librationPoints(mu);

options_ODE = odeset('RelTol',3e-14, 'AbsTol',1e-14);

options_planet3D.Position = [1-mu, 0, 0]';
options_planet3D.RefPlane = 'ecliptic';
options_planet3D.RotAngle = 135;
options_planet3D.Units = 6378.1363e3*1000/1.5;

%% differential correction
iteration_DC_max = 100;
threshold = 1e-12;

% L1 Lyapunov orbit
x0_1 = [0.987933580858119 0 0 0 0.018052674638026 0]';
t0_1 = 3.290337439058733/2;
for iteration = 1:iteration_DC_max
    [x_n_1,t_n_1,C_L1] = fun_differential_correction_cr3bp(x0_1,t0_1,mu,options_ODE);

    tspan = [0 2*t_n_1];
    [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n_1, options_ODE);

    x_error = norm(x_corrected(end,:) - x_corrected(1,:));
    %disp( strcat('x_error = ',num2str(x_error)) );
    if x_error < threshold
        break;
    end

    if x_error > 1e+3
        disp('calculation diverged');
        return;
    end

    if iteration == iteration_DC_max
        disp('do not finish');
        return;
    end

    x0_1 = x_n_1;
    t0_1 = t_n_1;
end
x_L1 = x_corrected; % C_L1 = 3.000599995882406

% L2 Lyapunov orbit
x0_2 = [1.012164676114102 0 0 0 -0.017947125645568 0]';
t0_2 = 3.319976286115385/2;
for iteration = 1:iteration_DC_max
    [x_n_2,t_n_2,C_L2] = fun_differential_correction_cr3bp(x0_2,t0_2,mu,options_ODE);

    tspan = [0 2*t_n_2];
    [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n_2, options_ODE);

    x_error = norm(x_corrected(end,:) - x_corrected(1,:));
    %disp( strcat('x_error = ',num2str(x_error)) );
    if x_error < threshold
        break;
    end
end

```

```

if x_error > 1e+3
    disp('calculation diverged');
    return;
end

if iteration == iteration_DC_max
    disp('do not finish');
    return;
end

x0_2 = x_n_2;
t0_2 = t_n_2;
end
x_L2 = x_corrected; % C_L2 = 3.000600000000000
t_L2 = t_n_2;

%% initial trajectory
x1 = [0.985000000000000 0.001092112291766 0 0.022 0.005 0]';
x2 = [1-mu -4e-3 0 0.03 0 0]';
x3 = [1.015000000000000 0.001092400176687 0 0.020764610600732 -0.004967672676993 0]';
delta_t1 = 0.75;
delta_t2 = 0.75;

tspan1 = [0 delta_t1];
[~,x_traj_1] = ode113(@(t,x) fun_cr3bp(t,x,mu),tspan1,x1,options_ODE);

tspan2 = [0 delta_t2];
[~,x_traj_2] = ode113(@(t,x) fun_cr3bp(t,x,mu),tspan2,x2,options_ODE);

f1 = figure();
hold on
planet3D('Earth_Cloudy',options_planet3D);
plot(x_L1(1,:),x_L1(2,:), 'k');
plot(x_L2(1,:),x_L2(2,:), 'k');
plot(x_traj_1(1,:),x_traj_1(2,:), 'Color','r','MarkerSize',10);
plot(x_traj_2(1,:),x_traj_2(2,:), 'Color','r','MarkerSize',10);
plot(x1(1),x1(2), 'o', 'MarkerFaceColor','r','MarkerEdgeColor','r','MarkerSize',10);
plot(x2(1),x2(2), 'o', 'MarkerFaceColor','r','MarkerEdgeColor','r','MarkerSize',10);
plot(x3(1),x3(2), 'o', 'MarkerFaceColor','r','MarkerEdgeColor','r','MarkerSize',10);
plot(L1(1),L1(2), 'k', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
plot(L2(1),L2(2), 'k', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
view(gca,0,90);
axis equal
xlabel('x');
ylabel('y');
xlim([0.985 1.015]);
grid on
hold off
f1_name = strcat('Ex7_initial_guess_mu=',num2str(mu));
f1_name = strrep(f1_name, ',', '');
save_fig(f1,f1_name,[0 90]);

f2 = figure();
hold on
planet3D('Earth_Cloudy',options_planet3D);
plot(x_L1(1,:),x_L1(2,:), 'k');
plot(x_L2(1,:),x_L2(2,:), 'k');
f2_p1 = plot(x_traj_1(1,:),x_traj_1(2,:), 'Color','r','MarkerSize',10);
plot(x_traj_2(1,:),x_traj_2(2,:), 'Color','r','MarkerSize',10);
plot(x1(1),x1(2), 'o', 'MarkerFaceColor','r','MarkerEdgeColor','r','MarkerSize',10);
plot(x2(1),x2(2), 'o', 'MarkerFaceColor','r','MarkerEdgeColor','r','MarkerSize',10);
plot(x3(1),x3(2), 'o', 'MarkerFaceColor','r','MarkerEdgeColor','r','MarkerSize',10);
plot(L1(1),L1(2), 'k', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
plot(L2(1),L2(2), 'k', 'MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
hold off

```

Example 2: Multiple shooting by fmincon

Ex7_multiple_shooting_fmincon.m (continued)

```
%% multiple shooting
X0 = [x1(4:6); x2; x3(4:6); delta_t1; delta_t2];
r0 = x1(1:3);
rf = x3(1:3);
ToF = delta_t1 + delta_t2;
n = 3;

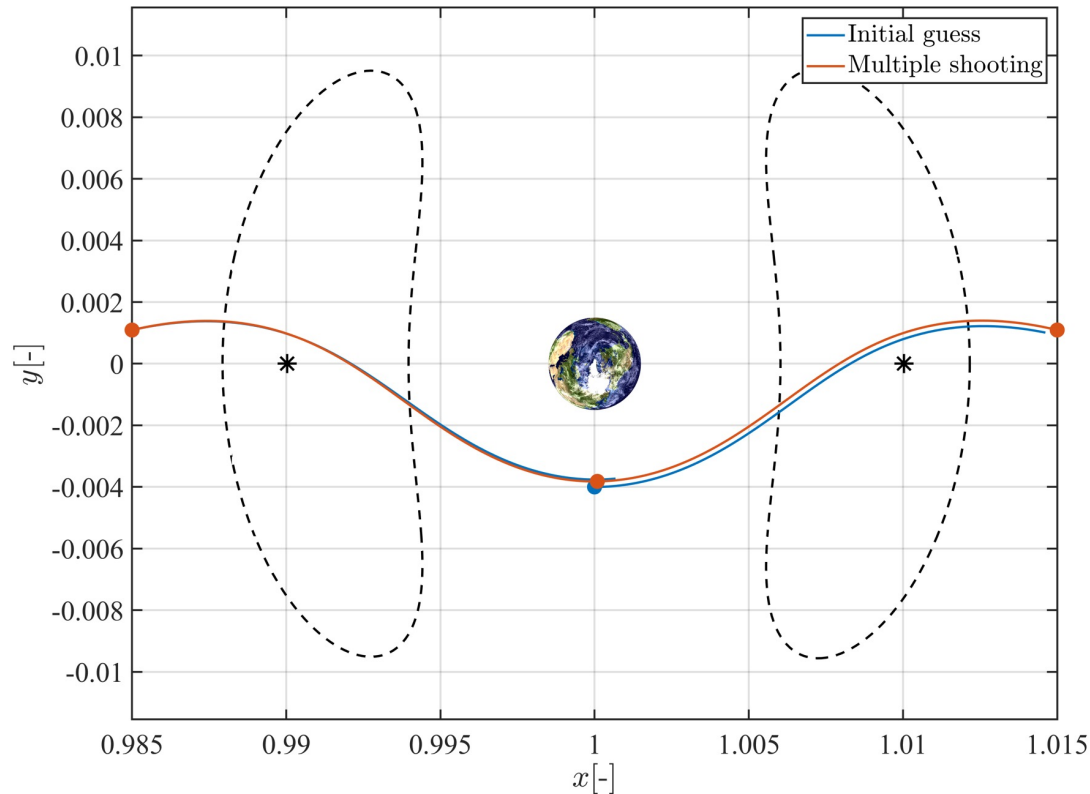
% Optimization with fmincon
problem.x0 = X0;
problem.A = [];
problem.b = [];
problem.Aeq = [];
problem.beq = [];
problem.ub = [];
problem.lb = [];
problem.nonlcon = @(x) fun_MulShoot_const_CR3BP(x, r0, rf, n, ToF, mu, options_ODE);
problem.objective = @(x) fun_MulShoot_objective_CR3BP(x, r0, rf, n, mu, options_ODE);
problem.solver = 'fmincon';
problem.options = optimoptions(@fmincon,'Algorithm','sqp', 'ConstraintTolerance',1e-14, 'Display','Iter', 'MaxFunctionEvaluations',1e+4, 'MaxIterations',1000, 'ObjectiveLimit',1e-12, 'StepTolerance',1e-14);
[x_ms, fval, exitflag, output] = fmincon(problem);
if exitflag <= 0
    disp('Error in fmincon');
    return;
end
X_n = [r0; x_ms(1:(6*n-9)); rf; x_ms((6*n-8):end)];
save('Ex7_solution.mat','X_n','fval');

figure(2);
hold on
for i = 1:(n-1)
    tspan = [0 X_n(6*n+i)];
    [~,X] = ode113(@(t,x) fun_cr3bp(t,x,mu),tspan,X_n((6*i-5):6*i),options_ODE);
    f2_p2 = plot(X(:,1),X(:,2),'Color','#D95319');
    plot(X(1,1),X(1,2),'o','MarkerFaceColor','#D95319','MarkerEdgeColor','#D95319','MarkerSize',10);
    plot(X(end,1),X(end,2),'o','MarkerFaceColor','#D95319','MarkerEdgeColor','#D95319','MarkerSize',10);
end
view(gca,0,90);
xlabel('$x[-]$');
ylabel('$y[-]$');
grid on
legend([f2_p1, f2_p2], {'Initial guess','Multiple shooting'}, 'Location','northeast');
hold off
f2_name = strcat('Ex7_multiple_shooting_mu=',num2str(mu));
f2_name = strrep(f2_name,','.','');
save_fig(f2,f2_name,[0 90]);

%% End of script
time = strcat('calculation time: ', num2str(toc(myTimer)));
disp(time);
```

Example 2: Multiple shooting by fmincon

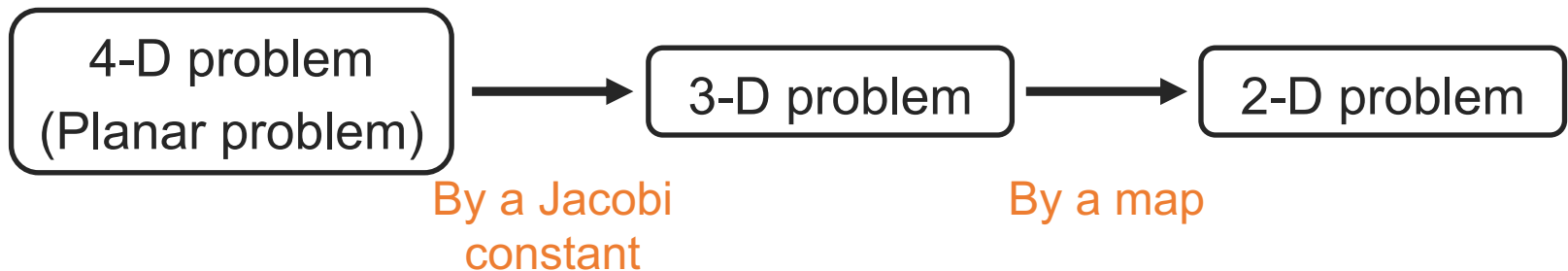
- In the end, $\|F\|$ becomes 4.3985e-06.
- The converged trajectory is similar to that in the previous example, but this trajectory contains larger discontinuity.



Result of multiple shooting

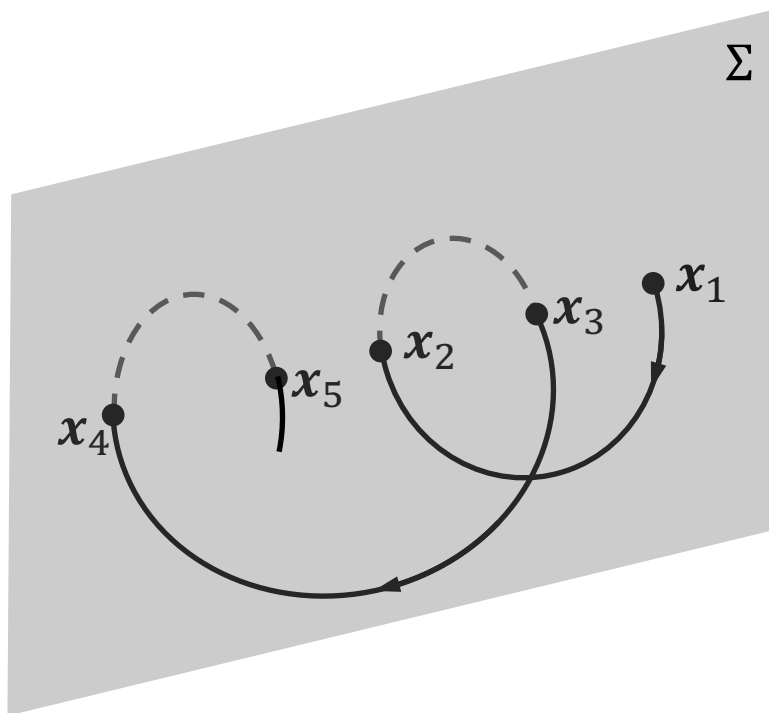
The introduction of Poincaré maps

- In Poincaré maps, points at which the trajectories cross a smooth **surface of section** (or hyperplane, Σ) are recorded and plotted.
- Using a map, the dimensionality of the problem is reduced by one.
- Imply **the dynamical structures** by the geometrical patterns
→ In a planar problem, Poincaré maps can fully represent the dynamics.



The introduction of Poincaré maps

- Two-sided map : Include crossings of the hyperplane Σ in both directions
- One-sided map : Include crossings of the hyperplane Σ in one direction

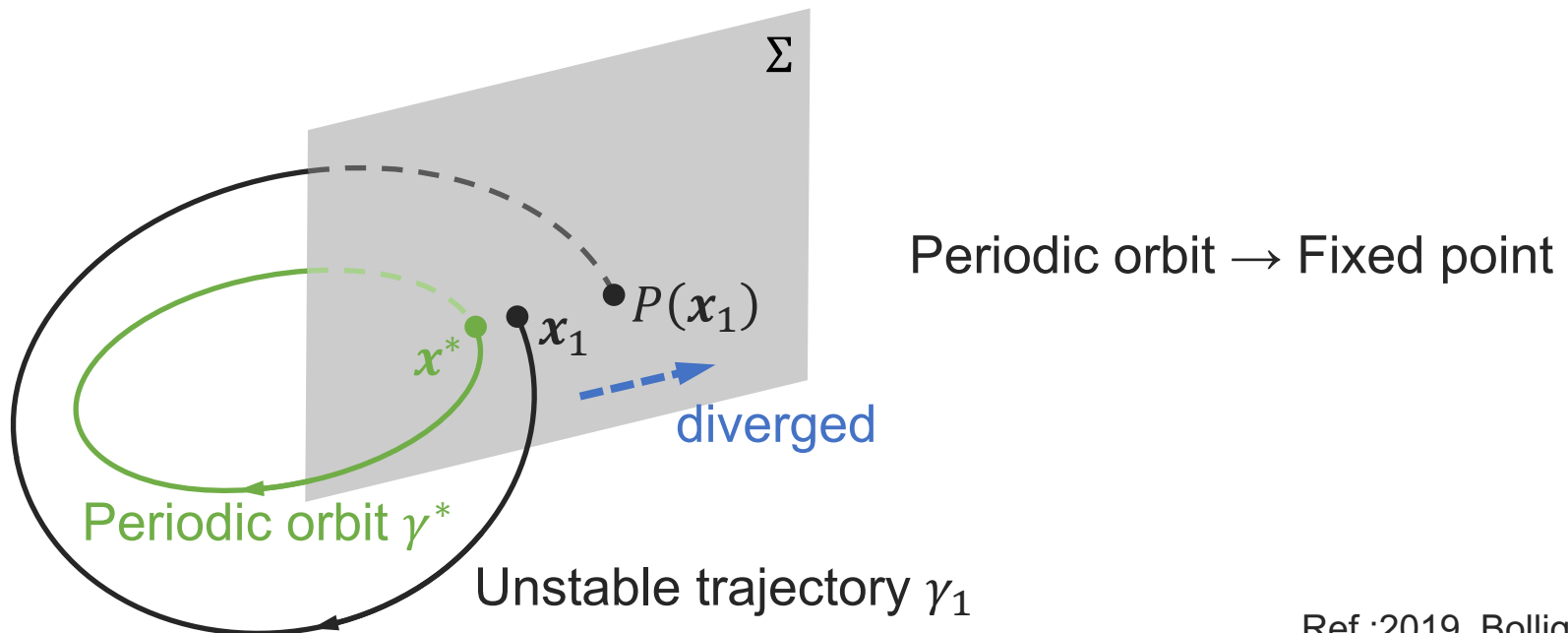


For example,

$$\left\{ \begin{array}{l} \text{A positive one-sided map: } A = \{x_1, x_3, x_5\} \\ \text{A negative one-sided map: } B = \{x_2, x_4\} \\ \text{Two-sided map: } A \cup B = \{x_1, x_2, x_3, x_4, x_5\} \end{array} \right.$$

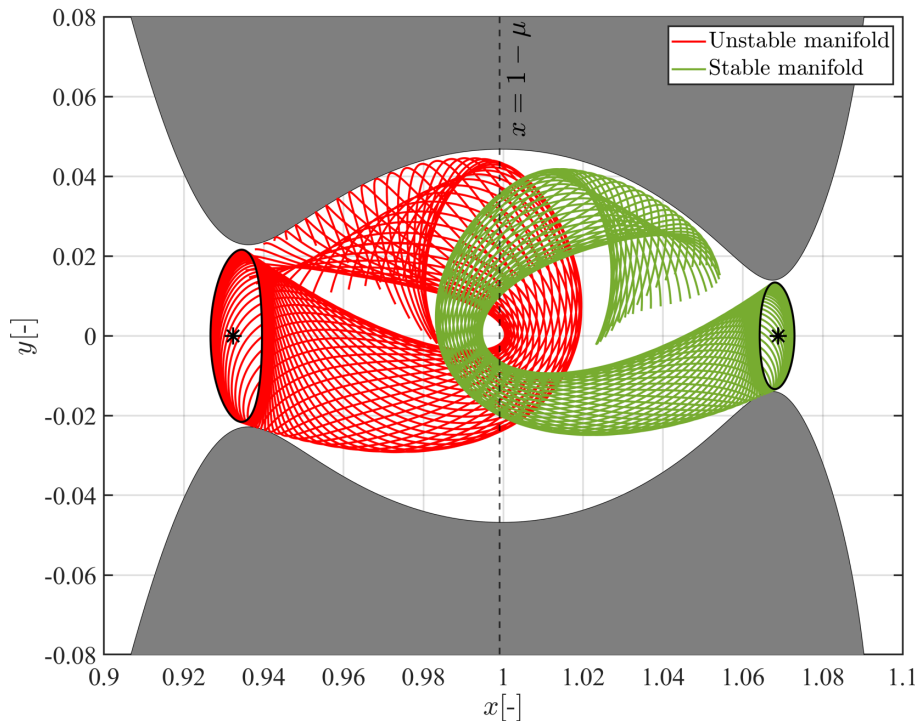
The introduction of Poincaré maps

- The hyperplane Σ is called **Poincaré section**.
- Poincaré section can be a physical plane (e.g., the x - y plane) or defined based on characteristic properties (e.g., periapse occurrences).
- Poincaré maps can preserve both **periodicity** and **stability** information.

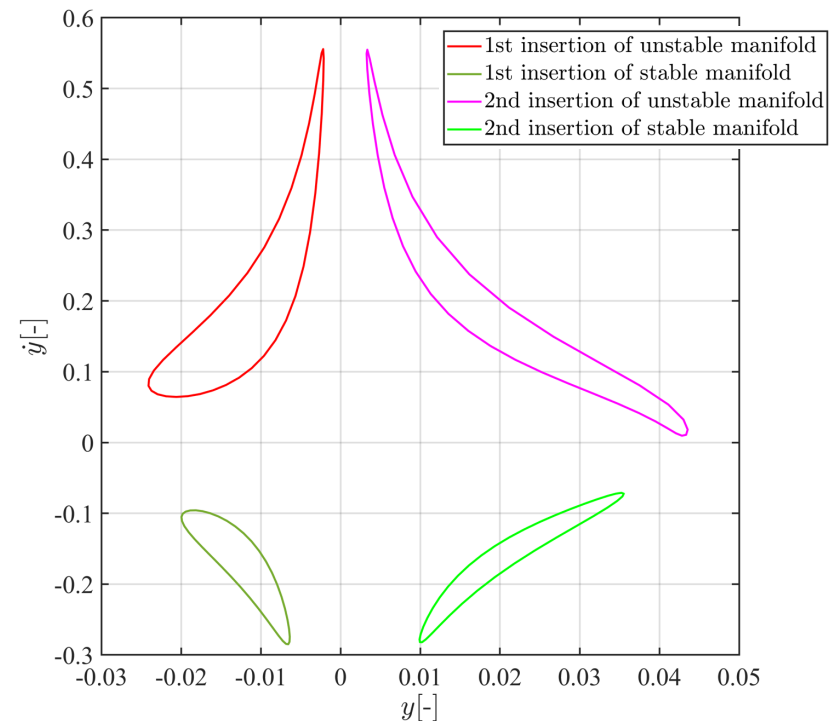


Example : Poincaré map

- Poincaré map of unstable manifolds of the L_1 Lyapunov orbit and stable manifolds of the L_2 Lyapunov orbit in the Sun-Jupiter system
- Poincaré section lies in $x = 1 - \mu$.



Poincaré map at $x = 1 - \mu$



Example : Poincaré map

fun_odestop_CR3BP_Body2_vertical.m

```
% poincare map where x = 1 - mu
function [value,isterminal,direction] = fun_odestop_CR3BP_Body2_vertical(t,x,mu)

    value = x(1) - (1-mu);
    isterminal = 0; % if isterminal == 1, the calculation will stop when the event occurs.
    direction = 0; % if direction == 0, all zeros are to be computed (the default)
                  % if direction == +1, only the zeros where the event function increases
                  % if direction == -1, only the zeros where the event function decreases.
end
```

Example : Poincaré map

Ex8_poincare_map.m

```
% Initial settings
[mu,~,~,~] = parameter(6); % Sun-Jupiter
[L1,L2,L3,L4,L5] = librationPoints(mu);

options_ODE = odeset('RelTol',3e-14, 'AbsTol',1e-14);
options_ODE_poincare = odeset('RelTol',3e-14, 'AbsTol',1e-14, 'Events', @(t,x) fun_odestop_CR3BP_Body2_vertical(t,x,mu) );

% differential correction
iteration_DC_max = 100;
threshold = 1e-12;

% L1 Lyapunov orbit
x0_1 = [0.926672517344211 0 0 0 0.045684284381193 0]';
t0_1 = 2.915640890603734/2;
for iteration = 1:iteration_DC_max
    [x_n_1,t_n_1,C_L1] = fun_differential_correction_cr3bp(x0_1,t0_1,mu,options_ODE);

    tspan = [0 2*t_n_1];
    [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n_1, options_ODE);

    x_error = norm(x_corrected(end,:) - x_corrected(1,:));
    %disp( strcat('x_error = ',num2str(x_error)) );
    if x_error < threshold
        break;
    end

    if x_error > 1e+3
        disp('calculation diverged');
        return;
    end

    if iteration == iteration_DC_max
        disp('do not finish');
        return;
    end

    x0_1 = x_n_1;
    t0_1 = t_n_1;
end
x_L1 = x_corrected; % C_L1 = 3.0369999999999999
t_L1 = t_n_1;

% L2 Lyapunov orbit
x0_2 = [1.072915577273646 0 0 0 -0.025945940908770 0]';
t0_2 = 3.184772418214781/2;
for iteration = 1:iteration_DC_max
    [x_n_2,t_n_2,C_L2] = fun_differential_correction_cr3bp(x0_2,t0_2,mu,options_ODE);

    tspan = [0 2*t_n_2];
    [t_corrected, x_corrected] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan, x_n_2, options_ODE);

    x_error = norm(x_corrected(end,:) - x_corrected(1,:));
    %disp( strcat('x_error = ',num2str(x_error)) );
    if x_error < threshold
        break;
    end

    if x_error > 1e+3
        disp('calculation diverged');
        return;
    end

    if iteration == iteration_DC_max
        disp('do not finish');
        return;
    end

    x0_2 = x_n_2;
    t0_2 = t_n_2;
end
x_L2 = x_corrected; % C_L2 = 3.036969030648278
t_L2 = t_n_2;
```

```
% zero velocity curve
[x_curve,y_curve] = meshgrid(0.9:1e-4:1.1,-0.1:1e-4:0.1);
[a,b] = size(x_curve);
z_curve = zeros(a,b);

x_curve = reshape(x_curve, [a*b,1]);
y_curve = reshape(y_curve, [a*b,1]);
z_curve = reshape(z_curve, [a*b,1]);

r1 = sqrt((x_curve+mu).^2+y_curve.^2+z_curve.^2);
r2 = sqrt((x_curve-(1-mu)).^2+y_curve.^2+z_curve.^2);
U = 1/2.*(x_curve.^2+y_curve.^2) + (1-mu)./r1 + mu./r2;
C = 2.*U;

x_curve = reshape(x_curve, [a,b]);
y_curve = reshape(y_curve, [a,b]);
C = reshape(C, [a,b]);

% manifold
tspan_u = [0 9];
tspan_s = [10 0];
xpert = 1e-10;
N = 40;

% L1
 [~,~,~, XU_right,~] = fun_manifold_cr3bp(mu, x_L1(1,:)', 2*t_L1, N, xpert, options_ODE);

% L2
 [XS_left,~,~,~,~] = fun_manifold_cr3bp(mu, x_L2(1,:)', 2*t_L2, N, xpert, options_ODE);
```

Example : Poincaré map

Ex8_poincare_map.m

```
%% poincare map
yu_right_f1 = zeros(N,6);
yu_right_f2 = NaN(N,6);
ys_left_f1 = zeros(N,6);
ys_left_f2 = NaN(N,6);

f1 = figure();
hold on
contourf(x_curve, y_curve, -C, [-C_L1 -C_L1], 'k');
colormap gray
for i = 1:N
    [~,yu_right,te,xe,ie] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_u, XU_right(:,i), options_ODE_poincare);
    f1_p1 = plot(yu_right(:,1),yu_right(:,2),'Color','r');
    yu_right_f1(i,:) = xe(1,:);

    if size(xe,1) < 2
        continue;
    end
    yu_right_f2(i,:) = xe(2,:);
end
for i = 1:N
    [~,ys_left,te,xe,ie] = ode113(@(t,x) fun_cr3bp(t,x,mu), tspan_s, XS_left(:,i), options_ODE_poincare);
    ys_left = flipud(ys_left);
    f1_p2 = plot(ys_left(:,1),ys_left(:,2),'Color','#77AC30');
    ys_left_f1(i,:) = xe(1,:);

    if size(xe,1) < 2
        continue;
    end
    ys_left_f2(i,:) = xe(2,:);
end
plot(x_L1(:,1),x_L1(:,2),'k');
plot(x_L2(:,1),x_L2(:,2),'k');
plot(L1(1),L1(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
plot(L2(1),L2(2),'*','MarkerFaceColor','k','MarkerEdgeColor','k','MarkerSize',15);
xlabel(1-mu, '-k', 'sx = 1 - \mu s', 'LineWidth',1.5,'FontSize',24,'Interpreter','latex');
view(gca,0,90);
axis equal
xlabel('$x[-]$');
ylabel('$y[-]$');
xlim([0.9 1.1]);
ylim([-0.08 0.08]);
grid on
legend([f1_p1, f1_p2], {'Unstable manifold','Stable manifold'});
hold off
f1_name = strcat('Ex8_poincare_map_tra_mu=',num2str(mu),'_xpert=',num2str(xpert));
f1_name = strrep(f1_name, ',', '');
save_fig(f1,f1_name,[0 90]);

f2 = figure();
hold on
f2_p1 = plot(yu_right_f1(:,2),yu_right_f1(:,5),'Color','r');
f2_p2 = plot(ys_left_f1(:,2),ys_left_f1(:,5),'Color','#77AC30');
f2_p3 = plot(yu_right_f2(:,2),yu_right_f2(:,5),'Color','m');
f2_p4 = plot(ys_left_f2(:,2),ys_left_f2(:,5),'Color','g');
axis square
xlabel('$y[-]$', 'Interpreter','latex');
ylabel('$\dot{y}[-]$', 'Interpreter','latex');
xlim([-0.03 0.05]);
ylim([-0.3 0.6]);
grid on
legend([f2_p1, f2_p2, f2_p3, f2_p4], {'1st insertion of unstable manifold','1st insertion of stable manifold',...
    '2nd insertion of unstable manifold','2nd insertion of stable manifold'}, 'Location','northeastoutside');
hold off
f2_name = strcat('Ex8_poincare_map_mu=',num2str(mu),'_xpert=',num2str(xpert));
f2_name = strrep(f2_name, ',', '');
save_fig(f2,f2_name,[0 90]);

%% End of script
time = strcat('calculation time: ', num2str(toc(myTimer)));
disp(time);
```

Natural Parameter Continuation (NPC)

- NPC is a straight-forward and **intuitive approach** to compute a family of solutions.
- The process is **initiated with some converged solution** from a single-shooting or multiple-shooting algorithm.
- The next solution is initialized by perturbing or varying a parameter a associated with the converged solution as follows:

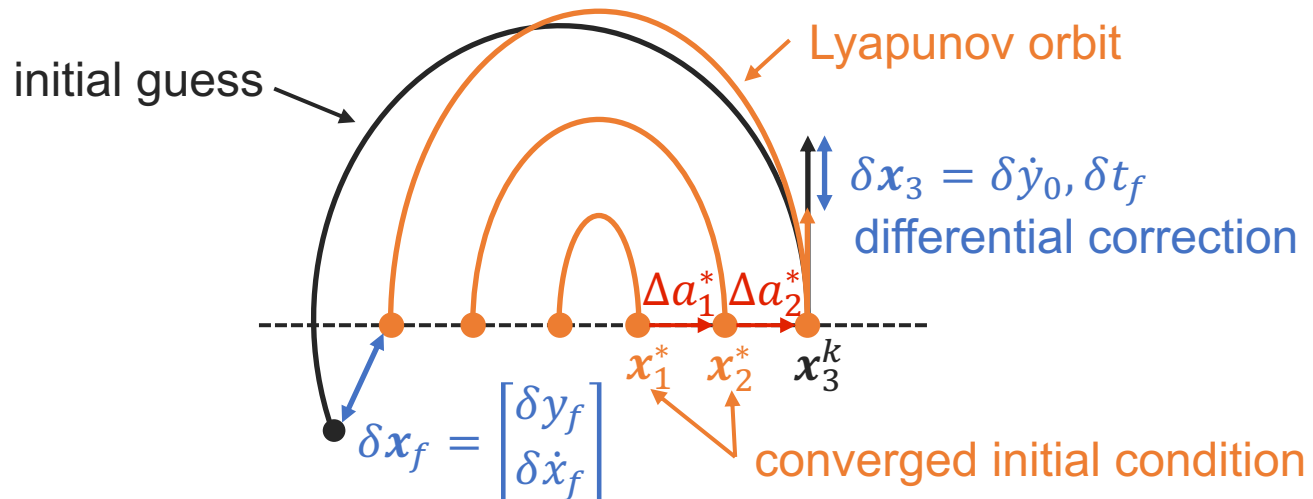
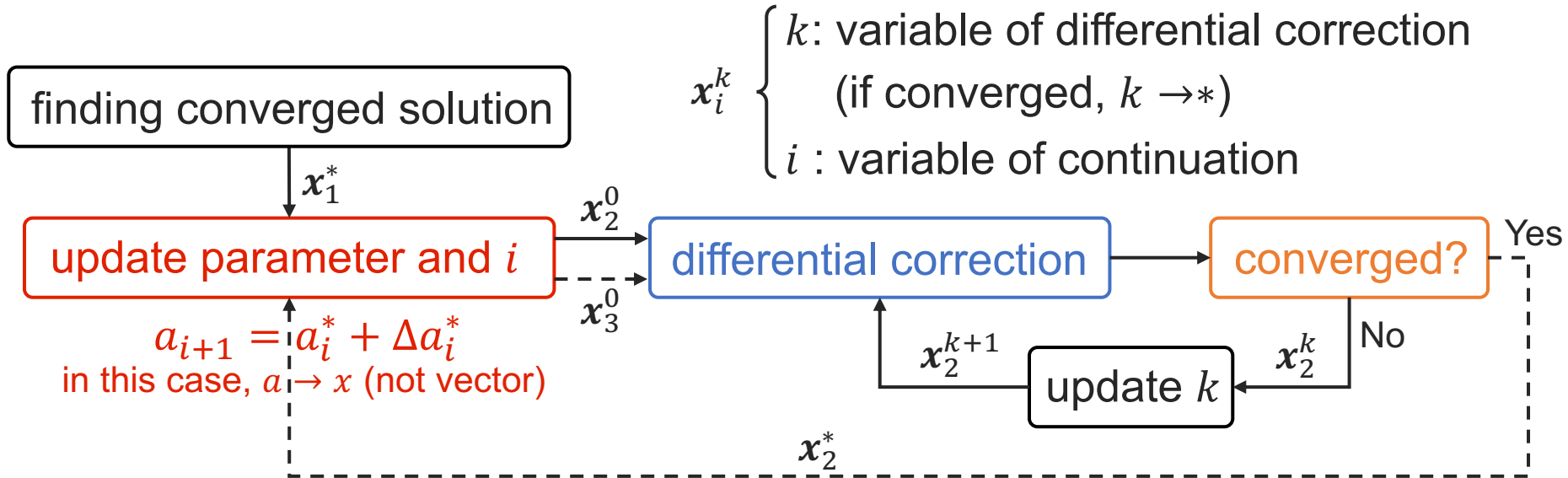
$$a_{i+1} = a_i^* + \Delta a_i^*$$

where a_{i+1} is the parameter of the $(i + 1)$ th solution, a_i^* is the parameter of the (i) th converged solution, and Δa_i^* is size of parameter update

Natural Parameter Continuation (NPC)

- A priori knowledge of the family of interest can **simplify the selection of parameters**.
- Example of parameter a is
 - Lyapunov orbit family: $a \rightarrow x$
 - halo orbit family: $a \rightarrow z$
 - extension from 2-body problem to 3-body problem: $a \rightarrow \mu$
- The initial guess for the perturbed solution is then converged to meet the constraints via a targeter, and this new converged solution then initializes the next solution.

NPC for Lyapunov orbit family



Example: NPC for Lyapunov orbit family

Ex9_natural_parameter_continuation.m

```

%% initial settings
[mu, a_1, ~, ~] = parameter(1); % Sun-Earth
[L1, L2, L3, L4, L5] = librationPoints(mu);
dim_L = a_1; % [-] -> [km]

options_ODE = odeset('RelTol', 3e-14, 'AbsTol', 1e-14);
iteration_max = 100;
threshold = 1e-10;
delta = 2e-6;
count = 0;
count_max = 4000;
plot_interval = 200;

Jacobi_min = 3.000040465479221;
Jacobi_max = 3.000881506800496;

color = jet;
Jacobi_lim = linspace(Jacobi_min, Jacobi_max, size(color,1));
Interp_c = griddedInterpolant(Jacobi_lim, color);

Jacobi = zeros(count_max/plot_interval, 1);
x0_corrected = zeros(count_max/plot_interval, 6);
t0_corrected = zeros(count_max/plot_interval, 1);

%% initial values
x0 = [1.01003168694194 0 0 0 8.61411407047802e-06 0]';
t0 = 1.47271346920646;
    
```

```

%% natural parameter continuation
f1 = figure();

while 1
    count = count + 1;
    disp(strcat('count = ', num2str(count)));

    % differential correction
    for iteration = 1:iteration_max
        [x_n, t_n, C] = fun_differential_correction_cr3bp(x0, t0, mu, options_ODE);

        tspan = [0 2*t_n];
        [t_corrected, x_corrected] = ode113(@(t, x) fun_cr3bp(t, x, mu), tspan, x_n, options_ODE);

        x_error = norm(x_corrected(end, :) - x_corrected(1, :));

        if x_error < threshold
            break;
        end

        if x_error > 1e+3
            disp('calculation diverged');
            return;
        end

        if iteration == iteration_max
            disp('do not finish');
            return;
        end

        x0 = x_n;
        t0 = t_n;
    end

    % plot orbit
    if (count >= plot_interval) && (mod(count, plot_interval) == 0)
        Jacobi(count/plot_interval, 1) = C;
        x0_corrected(count/plot_interval, :) = x_corrected(1, :);
        t0_corrected(count/plot_interval, 1) = t_corrected(end);
        rgb = Interp_c(C);
        plot(x_corrected(:, 1)*dim_L, x_corrected(:, 2)*dim_L, 'Color', rgb);
        hold on
    end

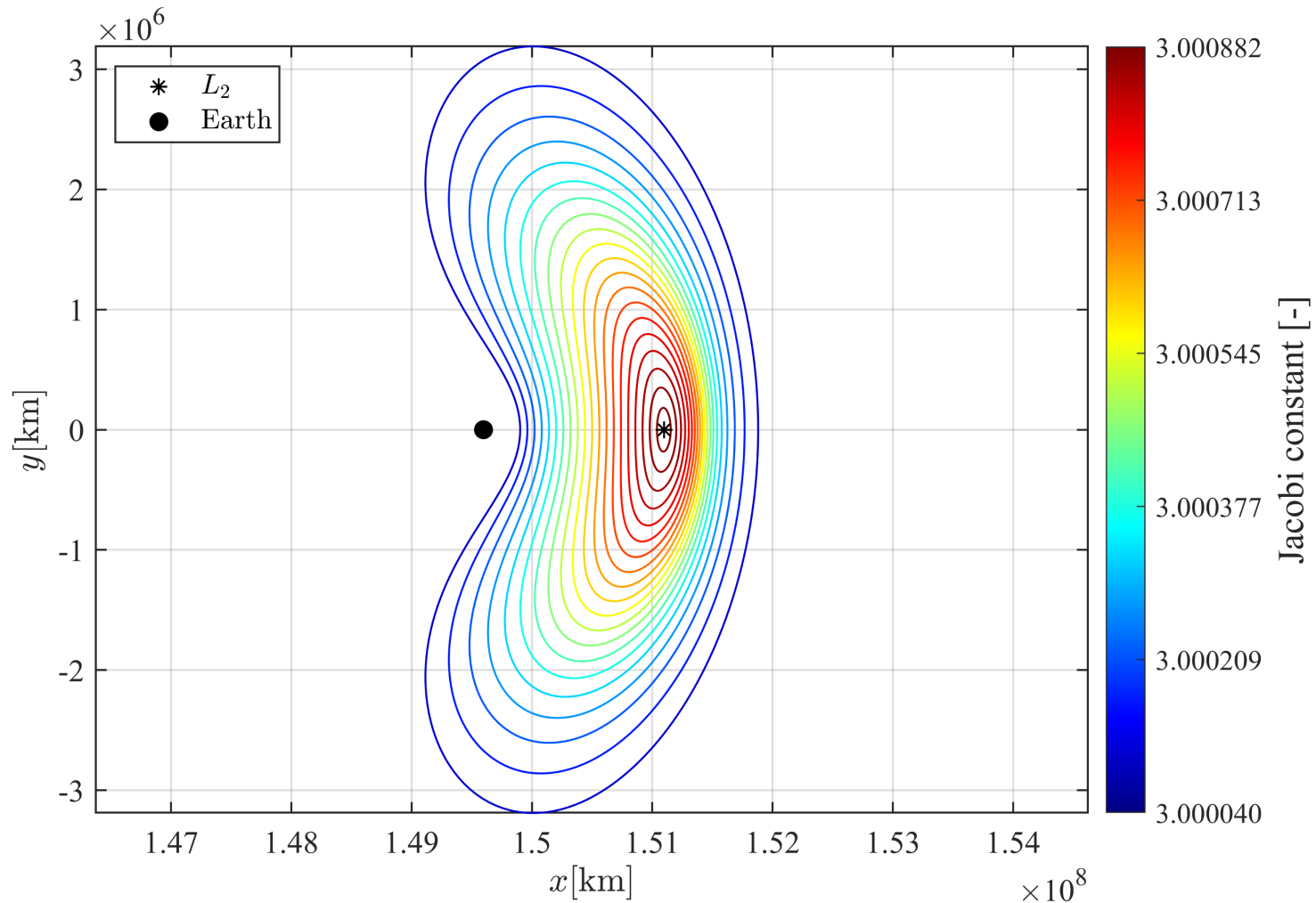
    x0(1) = x0(1) - delta;

    if count == count_max
        break
    end

    f1_p1 = plot(L2(1)*dim_L, L2(2)*dim_L, 'x', 'MarkerFaceColor', 'k', 'MarkerEdgeColor', 'k', 'MarkerSize', 7);
    f1_p2 = plot((1-mu)*dim_L, 0, 'o', 'MarkerFaceColor', 'k', 'MarkerEdgeColor', 'k', 'MarkerSize', 7);
    axis equal
    colormap jet;
    c = colorbar;
    ylabel(c, 'Jacobi constant [-]', 'FontSize', 15);
    caxis([Jacobi_min Jacobi_max]);
    c.Ticks = linspace(Jacobi_min, Jacobi_max, 6);
    xlabel('x$[km]$');
    ylabel('y$[km]$');
    grid on
    legend([f1_p1, f1_p2], {'L_2', 'Earth'}, 'Location', 'northwest');
    hold off

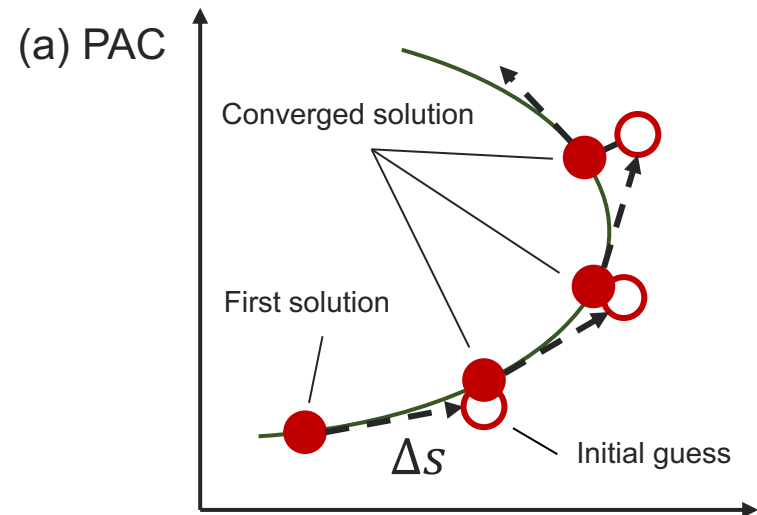
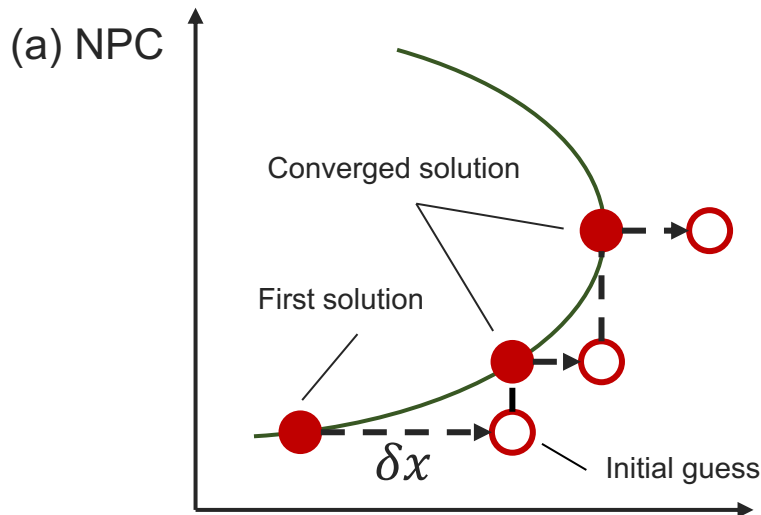
%% save results
f1_name = strcat('Ex_natural_parameter_continuation_mu', num2str(mu), '_x0=', num2str(x0(1)), '_ydot0=', num2str(x0(5)), '_t0=', num2str(t0), ...
    '_threshold=', num2str(threshold), '_delta=', num2str(delta), '_count_max=', num2str(count_max), '_plot_interval=', num2str(plot_interval));
f1_name = strrep(f1_name, '.', '_');
f1_name = strcat(f1_name, '.png');
print(f1, '-dpng', '-r600', f1_name);
    
```

Example: NPC for Lyapunov orbit family



Pseudo-arclength Continuation (PAC)

- The step implemented by PAC, denoted s , simultaneously updates all elements of the free variable vector for the next solution.
- PAC scheme ensures that this step is in the direction of the next family member by leveraging the Jacobian matrix of the previous solution.
- PAC is more robust to sudden slope changes along the parameter curve.
→ it can accommodate **larger step sizes** and require **fewer iterations**.



Pseudo-arclength Continuation (PAC)

- The update in the direction of the family is delivered by **the null space of the Jacobian matrix**, as vectors in the null space are tangent to the solution space of the family members.
- Define the previously converged solution by its free variable vector $\mathbf{X}_i^*$, such that $\mathbf{F}(\mathbf{X}_i^*) = \mathbf{0}$. The Jacobian matrix for this solution is $\mathbf{DF}(\mathbf{X}_i^*)$. The initial guess for the next solution is then defined

$$\mathbf{X}_{i+1}^k = \mathbf{X}_i^* + s\Delta\mathbf{X}_i^*$$

where $\Delta\mathbf{X}_i^*$, is the null space of the previously-converged Jacobian matrix, $\Delta\mathbf{X}_i^* = \mathcal{N}(\mathbf{DF}(\mathbf{X}_i^*))$ or $-\mathcal{N}(\mathbf{DF}(\mathbf{X}_i^*))$, $\|\Delta\mathbf{X}_i^*\| = 1$ and s is a scaling factor.

$$\mathbf{X}_i^k \begin{cases} k: \text{variable of differential correction} \\ \quad (\text{if converged, } k \rightarrow *) \\ i: \text{variable of continuation} \end{cases}$$

Pseudo-arclength Continuation (PAC)

- An additional constraint is appended to the original constraint vector. This tangent constraint is expressed

$$F_{PAC} = (\mathbf{X}_{i+1}^k - \mathbf{X}_i^*)^T \Delta \mathbf{X}_i^* - s = 0$$

* For derivation, click on the "Notes" button.

- The augmented constraint vector then has the form

$$\mathbf{G}(\mathbf{X}_{i+1}^k) = \begin{bmatrix} \mathbf{F}(\mathbf{X}_{i+1}^k) \\ F_{PAC} \end{bmatrix} = \mathbf{0}$$

- Thus, the augmented Jacobian is

$$\mathbf{DG}(\mathbf{X}_{i+1}^k) = \begin{bmatrix} \mathbf{DF}(\mathbf{X}_{i+1}^k) \\ \Delta \mathbf{X}_i^{*T} \end{bmatrix}$$

- Then, the differential correction process in PAC is expressed as

$$\mathbf{X}_{i+1}^{k+1} = \mathbf{X}_{i+1}^k - \mathbf{DG}^{-1}(\mathbf{X}_{i+1}^k) \mathbf{G}(\mathbf{X}_{i+1}^k)$$

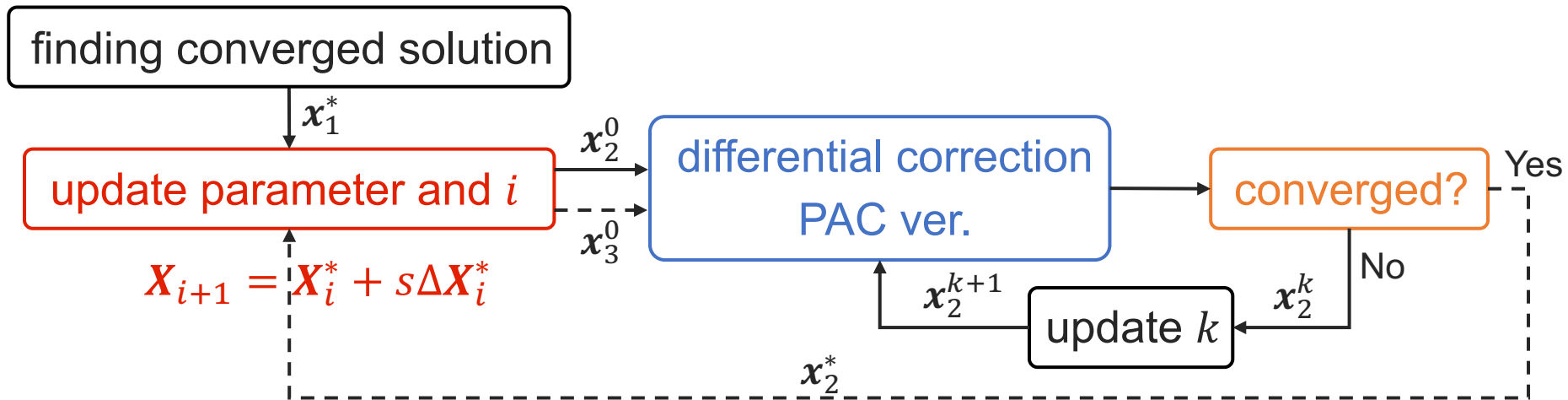
PAC for halo orbit family

- If $\mathbf{x}(t_f; \mathbf{x}_0) = [x_f \ y_f \ z_f \ \dot{x}_f \ \dot{y}_f \ \dot{z}_f]^T$, the objective function is defined as

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_0 \\ z_0 \\ \dot{y}_0 \\ t_f \end{bmatrix}, \quad \mathbf{F}(\mathbf{X}) = \begin{bmatrix} y_f - 0 \\ \dot{x}_f - 0 \\ \dot{z}_f - 0 \end{bmatrix}$$

- The Jacobian matrix for $\mathbf{X}$ in this situation is

$$\mathbf{DF}(\mathbf{X}) = \begin{bmatrix} \Phi_{2,1} & \Phi_{2,3} & \Phi_{2,5} & \dot{y}_f \\ \Phi_{4,1} & \Phi_{4,3} & \Phi_{4,5} & \ddot{x}_f \\ \Phi_{6,1} & \Phi_{6,3} & \Phi_{6,5} & \ddot{z}_f \end{bmatrix}$$



Sample Code: Pseudo-arclength Continuation

fun_differential_correction_cr3bp_PAC.m

```
% differential correction for halo family in CR3BP by PAC
function [x02_ast, t02_ast, C, G] = fun_differential_correction_cr3bp_PAC(x01_ast, t01_ast, x02, t02, scale, delta, mu, options)
% x01_ast : previously-converged x0 ( (i)th orbit )
% t01_ast : previously-converged period ( (i)th orbit )
% x02      : initial guess of x0 ( (i+1)th orbit )
% t02      : initial guess of period ( (i+1)th orbit )
% scale    : scaling factor of delta
% delta    : null space of the previously-converged Jacobian matrix
% mu       : mass ratio of the primaries
% options  : options for ode

X0 = [x02(1), x02(2), x02(3), x02(4), x02(5), x02(6), reshape(eye(6), 1, [])];
tspan = [0 t02];
[~, Y] = ode113(@(t, x) fun_stm_cr3bp(t, x, mu), tspan, X0, options);

X = Y(end, 1:6);
Phi = reshape(Y(end, 7:end), 6, 6);
f_x = fun_cr3bp([], X, mu);
F = [X(end, 2); X(end, 4); X(end, 6)];
F_PAC = ([x02(1); x02(3); x02(5); t02] - [x01_ast(1); x01_ast(3); x01_ast(5); t01_ast])'*delta - scale;
G = [F; F_PAC];
DF = [Phi(2, 1), Phi(2, 3), Phi(2, 5), f_x(2);
      Phi(4, 1), Phi(4, 3), Phi(4, 5), f_x(4);
      Phi(6, 1), Phi(6, 3), Phi(6, 5), f_x(6)];
DG = [DF; delta'];
X_ast = [x02(1); x02(3); x02(5); t02] - DG\G;

x02_ast = [X_ast(1); 0; X_ast(2); 0; X_ast(3); 0];
t02_ast = X_ast(4);
C = Jacobi_const(x02_ast, mu);
end
```

Sample Code: Pseudo-arclength Continuation

fun_null_cr3bp.m

```
function [N, R] = fun_null_cr3bp(x0, t0, mu, options)
% x0      : initial guess of x0
% t0      : initial guess of period
% mu      : mass ratio of the primaries
% options : options for ode

X0 = [x0(1), x0(2), x0(3), x0(4), x0(5), x0(6), reshape(eye(6), 1, [])];
tspan = [0 t0];
[~, Y] = ode113(@(t, x) fun_stm_cr3bp(t, x, mu), tspan, X0, options);

X = Y(end, 1:6);
Phi = reshape(Y(end, 7:end), 6, 6);
f_x = fun_cr3bp([], X, mu);
DF = [Phi(2, 1), Phi(2, 3), Phi(2, 5), f_x(2);
      Phi(4, 1), Phi(4, 3), Phi(4, 5), f_x(4);
      Phi(6, 1), Phi(6, 3), Phi(6, 5), f_x(6)];
R = rank(DF);
N = null(DF); % Jacobian matlix
end
```

Example: PAC for halo orbit family

Ex10_pseudo_arclength_continuation.m

```
% Initial settings
[mu, a_1, ~, ~] = parameter(1); % Sun-Earth
[L1, L2, L3, L4, L5] = librationPoints(mu);
dim_L = a_1; % [-] -> [km]

options_ODE = odeset('RelTol', 3e-14, 'AbsTol', 1e-14);
iteration_max = 100;
threshold = 1e-10;
scale = 1e-4;
count = 0;
count_max = 8400;
plot_interval = 100;

Jacobi_min = 3.000212571920089;
Jacobi_max = 3.000818466710653;

xlim_min = 1.485*10^8;
xlim_max = 1.525*10^8;
ylim_min = -2.5*10^6;
ylim_max = 2.5*10^6;
zlim_min = -23*10^5;
zlim_max = 10*10^5;

color = jet;
Jacobi_lim = linspace(Jacobi_min, Jacobi_max, size(color,1));
Interp_c = griddedInterpolant(Jacobi_lim, color);

Jacobi = zeros(count_max/plot_interval+1, 1);
x0_corrected = zeros(count_max/plot_interval+1, 6);
t0_corrected = zeros(count_max/plot_interval+1, 1);

% Initial values
x01_ast = [1.00837693674644 0 0.000200000000000000 0 0.00976542047036519 0]';
t01_ast = 1.55123023270505;
```

```
% pseudo arclength continuation
f1 = figure();
hold on

tspan1 = [0, 2*t01_ast];
[-, x1] = ode113(@(t, x) fun_cr3bp(t, x, mu), tspan1, x01_ast, options_ODE);

Jacobi(1) = Jacobi_const(x01_ast, mu);
x0_corrected(1,:) = x01_ast';
t0_corrected(1) = t01_ast;
rgb = Interp_c(Jacobi(1));
plot3(x1(1,:), dim_L, x1(1, 2:dim_L), x1(1, 3:dim_L), 'Color', rgb);
plot3(x1(1,:), dim_L, x1(1, 2:dim_L), zlim_minones(numel(x1(1, 1)), 1), 'Color', rgb + [1, 1, 1]/2);
plot3(x1(1,:), dim_L, ylim_maxones(numel(x1(1, 1)), 1), x1(1, 3:dim_L), 'Color', rgb + [1, 1, 1]/2);
plot3(xlim_maxones(numel(x1(1, 1)), 1), x1(1, 2:dim_L), x1(1, 3:dim_L), 'Color', rgb + [1, 1, 1]/2);

while 1
    count = count + 1;
    disp(sprintf('count = ', num2str(count)));
    [delta, ~] = fun_null_cr3bp(x01_ast, t01_ast, mu, options_ODE);
    delta = -delta;
    x02 = x01_ast;
    x02(1) = x01_ast(1) + scaledelta(1);
    x02(2) = x01_ast(2) + scaledelta(2);
    x02(5) = x01_ast(5) + scaledelta(3);
    t02 = t01_ast + scaledelta(4);

    % differential correction
    for iteration = 1:iteration_max
        [x00_ast, t02_ast, C, G] = fun_differential_correction_cr3bp_PAC(x01_ast, t01_ast, x02, t02, scale, delta, mu, options_ODE);
        tspan2 = [0, 2*t02_ast];
        [L_tcorrected, x_corrected] = ode113(@(t, x) fun_cr3bp(t, x, mu), tspan2, x02_ast, options_ODE);

        if norm(G) < threshold
            break;
        end

        if norm(G) > 1e3
            disp('calculation diverged');
            return;
        end

        if iteration == iteration_max
            disp('do not finish');
            return;
        end

        x02 = x02_ast;
        t02 = t02_ast;
    end

    % plot orbit
    if (count == plot_interval) && (mod(count, plot_interval) == 0)
        Jacobi(count/plot_interval+1) = Jacobi_const(x02_ast, mu);
        x0_corrected(count/plot_interval+1,:) = x_corrected(1,:);
        t0_corrected(count/plot_interval+1) = t02_ast;
        rgb = Interp_c(Jacobi(count/plot_interval+1));
        plot3(x_corrected(1,:), dim_L, x_corrected(1, 2:dim_L), x_corrected(1, 3:dim_L), 'Color', rgb);
        plot3(x_corrected(1,:), dim_L, x_corrected(1, 2:dim_L), zlim_minones(numel(x_corrected(1, 1)), 1), 'Color', rgb + [1, 1, 1]/2);
        plot3(x_corrected(1,:), dim_L, ylim_maxones(numel(x_corrected(1, 1)), 1), x_corrected(1, 3:dim_L), 'Color', rgb + [1, 1, 1]/2);
        plot3(xlim_maxones(numel(x_corrected(1, 1)), 1), x_corrected(1, 2:dim_L), x_corrected(1, 3:dim_L), 'Color', rgb + [1, 1, 1]/2);
    end

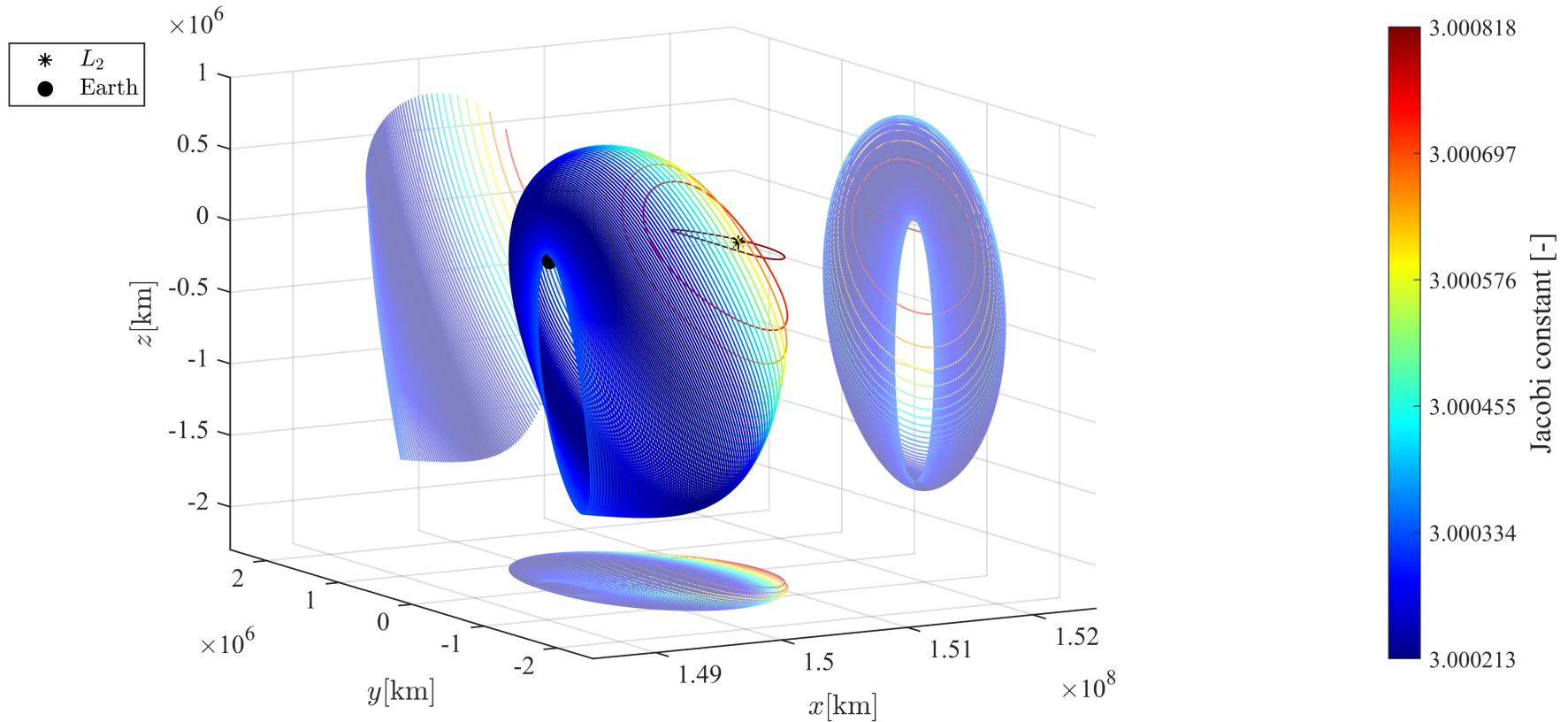
    x01_ast = x02_ast;
    t01_ast = t02_ast;

    if count == count_max
        break;
    end
end

f1.plt = plot3(L1(1:dim_L), L2(2:dim_L), L2(3:dim_L), 'r', 'MarkerFaceColor', 'k', 'MarkerEdgeColor', 'k', 'MarkerSize', 7);
f1.p2 = plot3(f1.mu*dim_L, 0, 0, 'b', 'MarkerFaceColor', 'k', 'MarkerEdgeColor', 'k', 'MarkerSize', 7);
view_angle = [-30 10];
view(gca, view_angle(1), view_angle(2));
axis equal
colormap jet;
G = colorbar;
xlabel(G, 'Jacobi constant [-]', 'FontSize', 15);
caxis([Jacobi_min Jacobi_max]);
G.Ticks = linspace(Jacobi_min, Jacobi_max, 6);
xlim([xlim_min xlim_max]);
ylim([ylim_min ylim_max]);
zlim([zlim_min zlim_max]);
xlabel('x(km)');
ylabel('y(km)');
zlabel('z(km)');
grid on
legend(f1.plt, f1.p2, 'L1_2D', 'Earth', 'Location', 'northwest');
hold off

% save results
f1_name = sprintf('Ex_pseudo_arclength_continuation_mu', num2str(mu), num2str(x01_ast), num2str(x02_ast), num2str(t01_ast), num2str(t02_ast), num2str(x0_corrected(1,1)), num2str(x0_corrected(1,2)), num2str(x0_corrected(1,3)), num2str(x0_corrected(1,4)), num2str(x0_corrected(1,5)), num2str(t02_ast), num2str(t02_ast), num2str(threshold), num2str(scale), num2str(count_max), num2str(plot_interval), num2str(plot_interval));
f1_name = sprintf(f1_name, '.png');
f1_name = sprintf(f1_name, '.png');
print(f1, f1_name, '-r300', f1_name);
```

Example: PAC for halo orbit family



Example: PAC for northern halo orbit family

Northern_Halo_family.m

```
%% LIBRATION POINT L2
[~,L2,~,~,~] = librationPoints(p("mu"));

%% INITIAL PERIODIC ORBIT
x0 = [1.01098476029841;0;0.00425821353748602;0;-0.0110580081896328;0];
T = 3.08315887305965;

%orbital period
t_n_L2 = T/2;
x_n_L2 = x0;

%% PSEUDO-ARCLENGTH CONTINUATION
hh = figure();
hold on
grid on
box on
view([-39 31]);
tspan1 = [0, 2*t_n_L2];
[~, x1] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan1, x_n_L2, options_ODE);
Jacobi1(1) = Jacobi_const(x_n_L2, p("mu"));
x0_corrected1(1, :) = x_n_L2';
t0_corrected1(1) = t_n_L2;
rgb = Interp_c(Jacobi1(1));
plot3(x1(:, 1), x1(:, 2), x1(:, 3), 'Color', rgb);

[delta, ~] = fun_null_cr3bp(x_n_L2, t_n_L2, p("mu"), options_ODE);%extract null space of initial values
delta = -delta;
x02 = x_n_L2;
x02(1) = x_n_L2(1) + p("scale")*delta(1);
x02(3) = x_n_L2(3) + p("scale")*delta(2);
x02(5) = x_n_L2(5) + p("scale")*delta(3);
t02 = t_n_L2 + p("scale")*delta(4);

while 1
    count = count + 1;
    disp(strcat('count = ', num2str(count)));

    % differential correction
    for iteration = 1:p("iteration_limit")
        [x02_ast, t02_ast, C, G] = fun_differential_correction_cr3bp_PAC(x_n_L2, t_n_L2, x02, t02, p("scale"), delta, p("mu"), options_ODE);

        tspan2 = [0, 2*t02_ast];
        [t_corrected, x_corrected] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan2, x02_ast, options_ODE);

        if norm(G) < p("threshold")
            break;
        end

        if norm(G) > 1e+3
            disp('calculation diverged');
            return;
        end

        if iteration == p("iteration_limit")
            disp('do not finish');
            return;
        end
    end
end
```

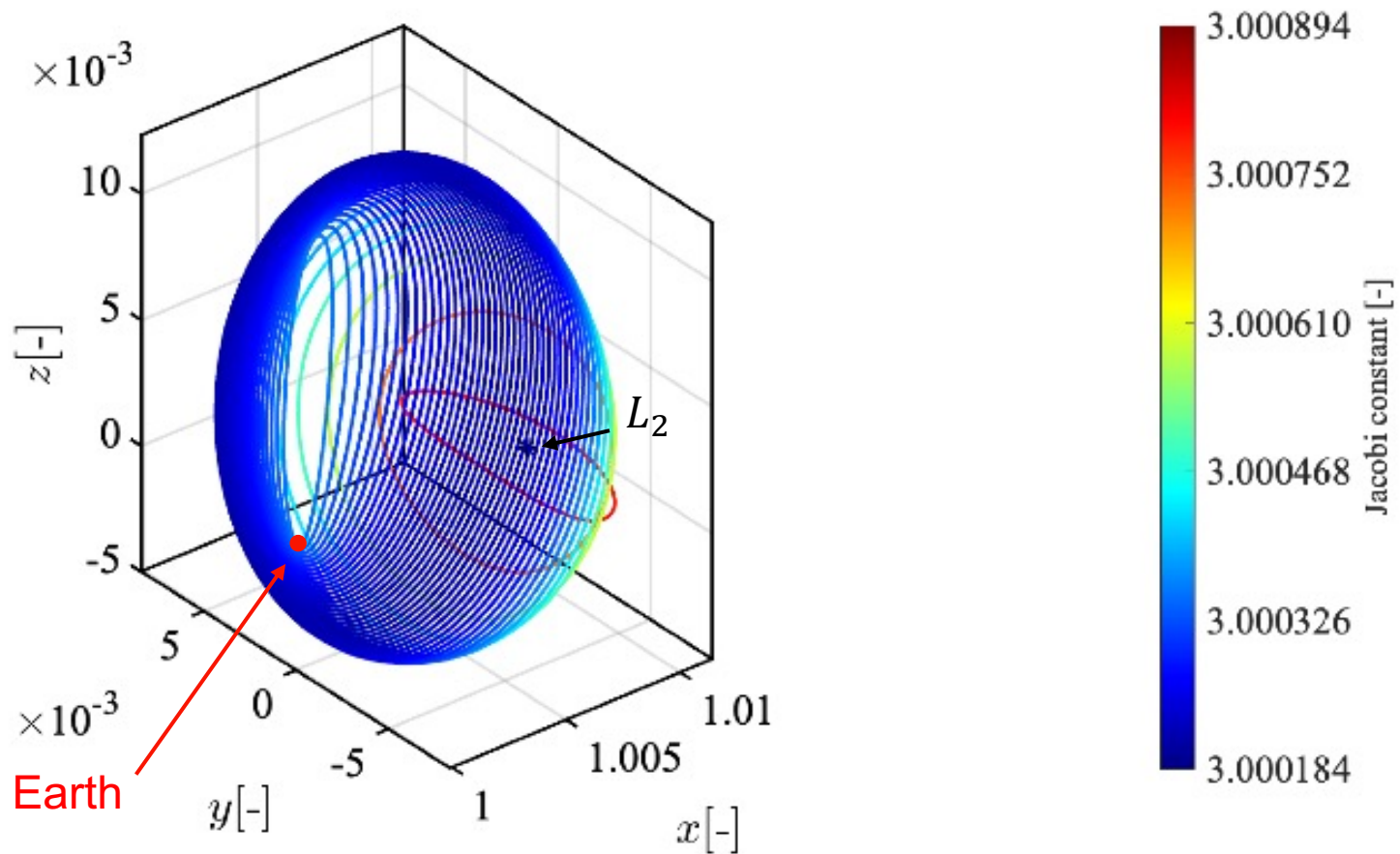
```
x02 = x02_ast;
t02 = t02_ast;
end

% plot orbit
if (count >= p("plot_interval")) && (mod(count, p("plot_interval")) == 0)
    Jacobi1(count/p("plot_interval")+1) = C;
    x0_corrected1(count/p("plot_interval")+1, :) = x_corrected(1, :); %RHS:initial cond
    t0_corrected1(count/p("plot_interval")+1) = t_corrected(end);
    rgb = Interp_c(C);
    plot3(x_corrected(:, 1), x_corrected(:, 2), x_corrected(:, 3), 'Color', rgb, 'LineWidth', 1.5);
end

x_n_L2 = x02_ast;
t_n_L2 = t02_ast;

if count == p("count_max")
    break
end
end
```

Example: PAC for northern halo orbit family



Example: PAC for southern halo orbit family

Southern_Halo_family.m

```
%% LIBRATION POINT L2
[~,L2,~,~,~] = librationPoints(p("mu"));

%% INITIAL PERIODIC ORBIT
x0 = [1.01041210799432;0;-0.00701710214085124;-0.0132251420919099;0];
T = 3.03765488729945;

%orbital period
t_n_L2 = T/2;
x_n_L2 = x0;

%% PSEUDO-ARCLENGTH CONTINUATION
hh = figure();
hold on
grid on
box on
view([-39 31]);
tspan1 = [0, 2*t_n_L2];
[~, x1] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan1, x_n_L2, options_ODE);
Jacobi1(1) = Jacobi_const(x_n_L2, p("mu"));
x0_corrected1(1, :) = x_n_L2';
t0_corrected1(1) = t_n_L2;
rgb = Interp_c(Jacobi1(1));
plot3(x1(:, 1), x1(:, 2), x1(:, 3), 'Color', rgb);

[delta, ~] = fun_null_cr3bp(x_n_L2, t_n_L2, p("mu"), options_ODE);%extract null space
delta = -delta;
x02 = x_n_L2;
x02(1) = x_n_L2(1) + p("scale")*delta(1);
x02(3) = x_n_L2(3) + p("scale")*delta(2);
x02(5) = x_n_L2(5) + p("scale")*delta(3);
t02 = t_n_L2 + p("scale")*delta(4);

while 1
count = count + 1;
disp(strcat('count = ', num2str(count)));
```

```
% differential correction
for iteration = 1:p("iteration_limit")
for x02_ast, t02_ast, C, G] = fun_differential_correction_cr3bp_PAC(x_n_L2, t_n_L2, x02, t02, p("scale"), delta, p("mu"), options_ODE);

tspan2 = [0, 2*t02_ast];
[t_corrected, x_corrected] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan2, x02_ast, options_ODE);

if norm(G) < p("threshold")
break;
end

if norm(G) > 1e+3
disp('calculation diverged');
return;
end

if iteration == p("iteration_limit")
disp('do not finish');
return;
end

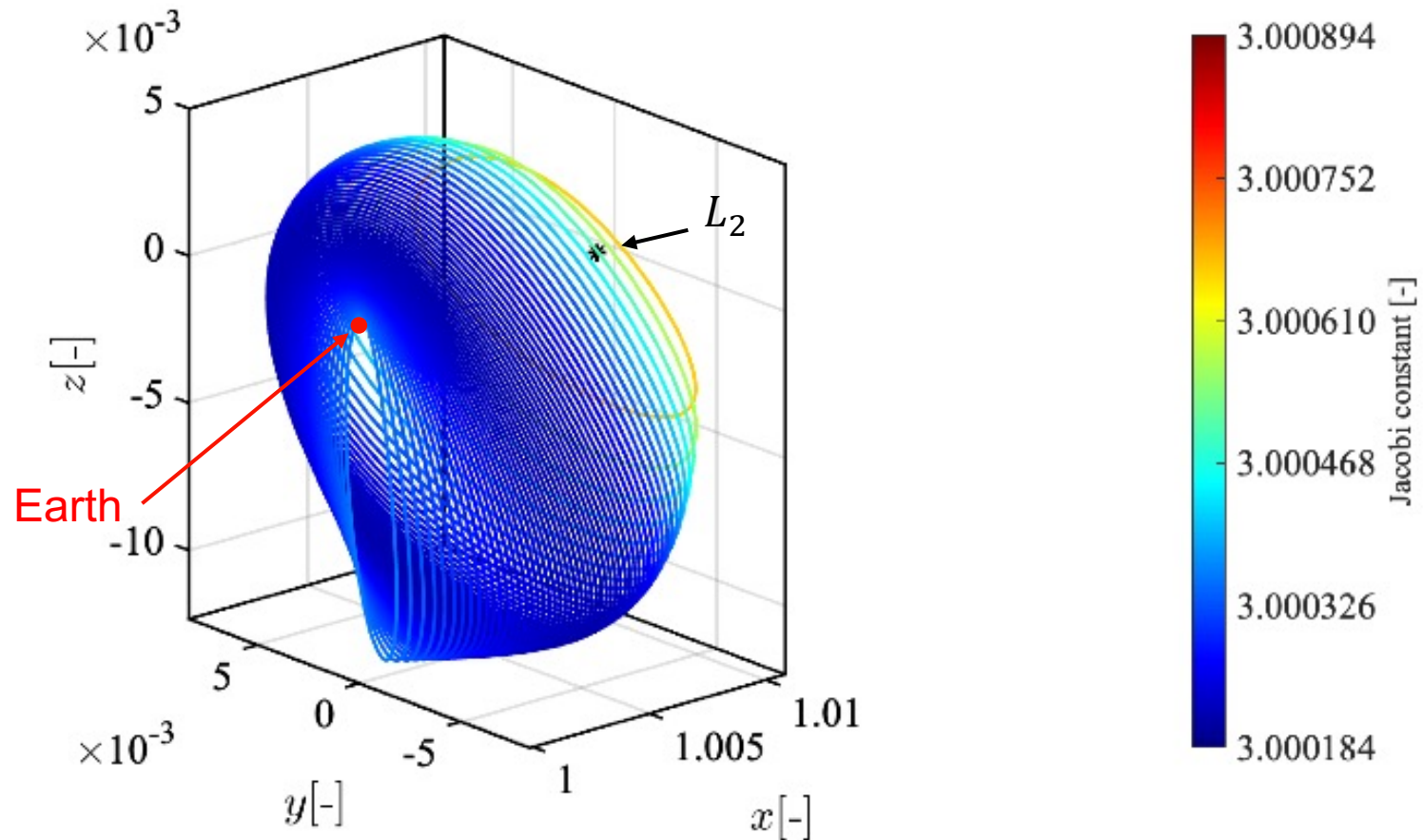
x02 = x02_ast;
t02 = t02_ast;
end

% plot orbit
if (count >= p("plot_interval")) && (mod(count, p("plot_interval")) == 0)
Jacobi1(count/p("plot_interval")+1) = C;
x0_corrected1(count/p("plot_interval")+1, :) = x_corrected(1, :); %RHS:initial cond
t0_corrected1(count/p("plot_interval")+1) = t_corrected(end);
rgb = Interp_c(C);
plot3(x_corrected(:, 1), x_corrected(:, 2), x_corrected(:, 3), 'Color', rgb,'LineWidth',1.5);
end

x_n_L2 = x02_ast;
t_n_L2 = t02_ast;

if count == p("count_max")
break
end
end
```

Example: PAC for southern halo orbit family



Example: PAC for vertical Lyapunov orbit family

Vertical_Lyapunov_family.m

```
%% INITIAL PERIODIC ORBIT
x0 = [1.0029239047032787;0;0;-4.8187208188144461e-2;-2.9416770480063559e-1];
T = 6.2808090883636165;
```

```
p = dictionary();
```

```
%% DICTIONARY OF SUN-EARTH CRTBP VARIABLES
```

```
p("Ms") = 1.9885e+30;
p("M1") = 5.9724e+24;
p("mu") = 3.00346093142064e-06;%SE
p("G") = 6.67300000000000e-11;
p("iteration_limit") = 100;
p("threshold") = 1e-10;
p("xpert") = 1e-7;
p("tf_L2") = 10;
p("scale") = 5e-5;
count = 0;
p("count_max") = 3000;
p("plot_interval") = 75;
p("Jacobi_min") = 2.913209300553006;
p("Jacobi_max") = 2.979596689425427;
```

```
p("chara_length_CR3BP_SE") = 149600000000.000;
dim_L = p("chara_length_CR3BP_SE")/1000;
p("chara_mass_CR3BP_SE") = 1.98850597240000e+30;
p("chara_time_CR3BP_SE") = 5023117.97170552;
p("N_CR3BP_SE") = 1.99079536979392e-07;
xpert_dim = p("xpert")*p("chara_length_CR3BP_SE")/p("chara_time_CR3BP_SE");
```

```
%% INITIALIZATION
```

```
options_ODE = odeset('RelTol', 3e-14, 'AbsTol', 1e-14);
color = jet;
Jacobi_lim = linspace(p("Jacobi_min"), p("Jacobi_max"), size(color,1));
Interp_c = griddedInterpolant(Jacobi_lim, color);
Jacobi = zeros(p("count_max")/p("plot_interval")+1, 1);
x0_corrected = zeros(p("count_max")/p("plot_interval")+1, 6);
t0_corrected = zeros(p("count_max")/p("plot_interval")+1, 1);
```

```
%% LIBRATION POINT L2
```

```
[~,L2,~,~,~] = librationPoints(p("mu"));
```

```
tspan1 = [0, 2*t_n_L2];
[~, x1] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan1, x_n_L2, options_ODE);
Jacobi(1) = Jacobi_const(x_n_L2, p("mu"));
x0_corrected(1, :) = x_n_L2;
t0_corrected(1) = t_n_L2;
rgb = Interp_c(Jacobi(1));
%plot3(x1(:, 1), x1(:, 2), x1(:, 3), 'Color', 'g');

[delta, ~] = fun_null_vertical_Lyapunov(x_n_L2, t_n_L2, p("mu"), options_ODE);%extract null space of initial values
delta = delta;
x02 = x_n_L2;
x02(1) = x_n_L2(1) + p("scale")*delta(1);
x02(5) = x_n_L2(5) + p("scale")*delta(2);
x02(6) = x_n_L2(6) + p("scale")*delta(3);
t02 = t_n_L2 + p("scale")*delta(4);

while 1
    count = count + 1;
    disp(strcat('count = ', num2str(count)));

    % differential correction
    for iteration = 1:p("iteration_limit")
        [x02_ast, t02_ast, C, G] = fun_differential_correction_vertical_Lyapunov_PAC(x_n_L2, t_n_L2, x02, t02, p("scale"), delta, p("mu"), options_ODE);

        tspan2 = [0, 2*t02_ast];
        [t_corrected, x_corrected] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan2, x02_ast, options_ODE);

        if norm(G) < p("threshold")
            break;
        end

        if norm(G) > 1e+3
            disp('calculation diverged');
            return;
        end

        if iteration == p("iteration_limit")
            disp('do not finish');
            return;
        end

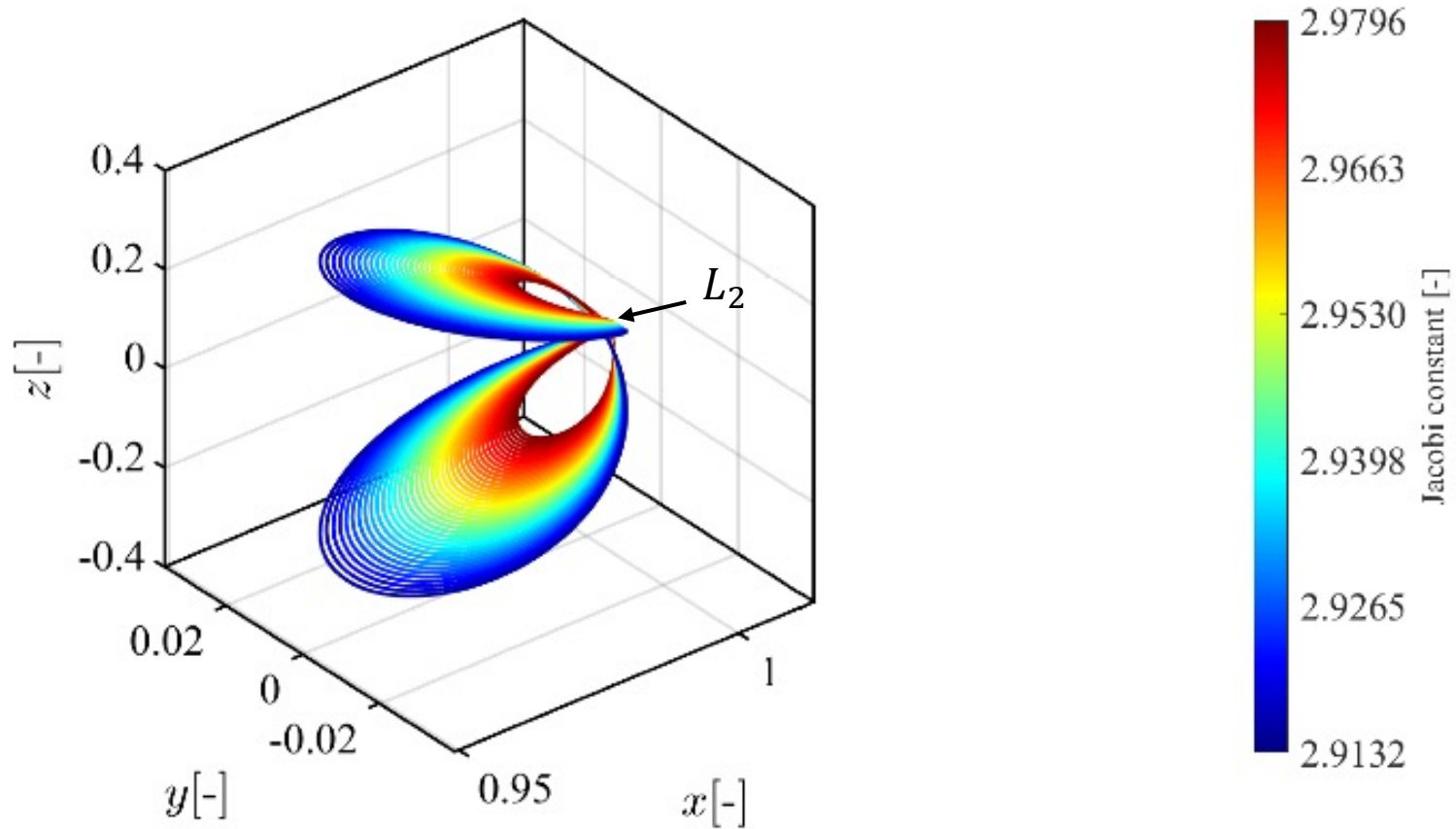
        x02 = x02_ast;
        t02 = t02_ast;
    end

    % plot orbit
    if (count >= p("plot_interval")) && (mod(count, p("plot_interval")) == 0)
        Jacobi(count/p("plot_interval")+1) = C;
        x0_corrected(count/p("plot_interval")+1, :) = x_corrected(1, :); %RHS:initial cond
        t0_corrected(count/p("plot_interval")+1) = t_corrected(end);
        rgb = Interp_c(C);
        plot3(x_corrected(:, 1), x_corrected(:, 2), x_corrected(:, 3), 'Color', rgb, 'Linewidth', 1.5);
    end

    x_n_L2 = x02_ast;
    t_n_L2 = t02_ast;

    if count == p("count_max")
        break
    end
end
```

Example: PAC for vertical Lyapunov orbit family



Example: PAC for axial orbit family

Axial_family.m

```
% INITIAL PERIODIC ORBIT
x0 = [1.01095593036708;0;0;-0.0198535102007301;0.0190210211349253];
T = 4.21697099424615;

p = dictionary();

%% DICTIONARY OF SUN-EARTH CRTBP VARIABLES
p("Ms") = 1.9885e+30;
p("M1") = 5.9724e+24;
p("mu") = 3.00346093142064e-06;%SE
p("G") = 6.67300000000000e-11;
p("iteration_limit") = 100;
p("threshold") = 1e-10;
p("xpert") = 1e-7;
p("tf_L2") = 10;
p("scale") = 1e-5;
count = 0;
p("count_max") = 2000;
p("plot_interval") = 100;
p("Jacobi_min") = 3.000091643722067;
p("Jacobi_max") = 3.000137845402229;

p("chara_length_CR3BP_SE") = 149600000000.000;
dim_L = p("chara_length_CR3BP_SE")/1000;
p("chara_mass_CR3BP_SE") = 1.98850597240000e+30;
p("chara_time_CR3BP_SE") = 5023117.97170552;
p("N_CR3BP_SE") = 1.99079536979392e-07;
xpert_dim = p("xpert")*p("chara_length_CR3BP_SE")/p("chara_time_CR3BP_SE");

%% INITIALIZATION
options_ODE = odeset('RelTol', 3e-14, 'AbsTol', 1e-14);
color = jet;
Jacobi_lim = linspace(p("Jacobi_min"), p("Jacobi_max"), size(color,1));
Interp_c = griddedInterpolant(Jacobi_lim, color);
Jacobi = zeros(p("count_max")/p("plot_interval")+1, 1);
x0_corrected = zeros(p("count_max")/p("plot_interval")+1, 6);
t0_corrected = zeros(p("count_max")/p("plot_interval")+1, 1);

%% LIBRATION POINT L2
[~,L2,~,~,~] = librationPoints(p("mu"));
```

```
tspan1 = [0, 2*t_n_L2];
[~, x1] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan1, x_n_L2, options_ODE);
Jacobi(1) = Jacobi_const(x_n_L2, p("mu"));
x0_corrected(1, :) = x_n_L2';
t0_corrected(1) = t_n_L2;
rgb = Interp_c(Jacobi(1));
%plot3(x1(:, 1), x1(:, 2), x1(:, 3), 'Color', 'g');

[delta, ~] = fun_null_vertical_Lyapunov(x_n_L2, t_n_L2, p("mu"), options_ODE);%extract null space of initial values
x02 = x_n_L2;
x02(1) = x_n_L2(1) + p("scale")*delta(1);
x02(5) = x_n_L2(5) + p("scale")*delta(2);
x02(6) = x_n_L2(6) + p("scale")*delta(3);
t02 = t_n_L2 + p("scale")*delta(4);

while 1
count = count + 1;
disp(strcat('count = ', num2str(count)));

% differential correction
for iteration = 1:p("iteration_limit")
[x02_ast, t02_ast, C, G] = fun_differential_correction_axial_PAC(x_n_L2, t_n_L2, x02, t02, p("scale"), delta, p("mu"), options_ODE);

tspan2 = [0, 2*t02_ast];
[t_corrected, x_corrected] = ode113(@(t, x) fun_cr3bp(t, x, p("mu")), tspan2, x02_ast, options_ODE);

if norm(G) < p("threshold")
break;
end

if norm(G) > 1e+3
disp('calculation diverged');
return;
end

if iteration == p("iteration_limit")
disp('do not finish');
return;
end

x02 = x02_ast;
t02 = t02_ast;
end

% plot orbit
if (count >= p("plot_interval")) && (mod(count, p("plot_interval")) == 0)
Jacobi(count/p("plot_interval")+1) = C;
x0_corrected(count/p("plot_interval")+1, :) = x_corrected(1, :); %RH5:initial cond
t0_corrected(count/p("plot_interval")+1) = t_corrected(end);
rgb = Interp_c(C);
plot3(x_corrected(:, 1), x_corrected(:, 2), x_corrected(:, 3), 'Color', rgb, 'Linewidth',1.5);
end

x_n_L2 = x02_ast;
t_n_L2 = t02_ast;

if count == p("count_max")
break
end
end
```

Example: PAC for axial orbit family

